

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\agent-a175e8d3638e28d46.jsonl`  
- SHA-256: `3f87c81abb7dc848e5c9490e99cf1d87cf2de68a5259371c1a46abef271cfbc4`  
- Source modified: `2026-06-16T07:07:03+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `subagents`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T06:41:12.777Z)

Harden the shipped Golden Regression Fixtures feature per a completed adversarial audit. Read `docs/GOLDEN\_REGRESSION\_FIXTURES.md` and the existing `backend/eval/golden\_fixtures.py`, `tests/test\_golden\_fixtures.py`, `tests/test\_golden\_real\_fixtures.py` first.

WORK ONLY IN THIS WORKTREE (isolated; do NOT touch the main checkout or other worktrees):

C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\
All paths relative to that root. Do NOT run git. Create/edit files + run tests only.

Implement ALL of the following confirmed fixes (each has a file:line anchor from the audit). Keep the existing public behavior for the COMMITTED synthetic fixtures byte-identical where possible ? the 3 synthetic fixtures' `expected.json` MUST still pass `--check` unchanged (you are hardening the freeze-real / edge paths + docs, not changing synthetic replay). If any change would alter a synthetic fixture's committed `expected.json`, STOP and report instead.

=== CODE ? backend/eval/golden_fixtures.py ===

1. ****[MUST-FIX] Free-text identifier leak via `source\_note`/`label`.** ****** `\_scan\_forbidden` only catches FORBIDDEN_KEYS/MD5/source-PATH; an identifying `--source-note`/`--label` (e.g. "Lin Dan vs Lee, India Open 2024") is written verbatim to provenance and passes silently. **FIX:** (a) validate `label` against a strict slug charset `[a-z0-9][a-z0-9\_\-]\*\$` (lowercase alnum + _/-, no spaces/punctuation) ? raise `RefusalError` with a clear message otherwise; (b) STOP persisting operator free-text `source\_note` into the committed `provenance.json` entirely (drop the `source\_note` key from the written provenance block; the fixed boilerplate `note` constant stays). Keep/deprecate the `--source-note` CLI flag as accepted-but-not-persisted (document it), OR remove it ? your call, but it must NOT reach committed bytes. Add tests: an identifying `label` (with spaces / a name) raises `RefusalError`; committed provenance has NO `source\_note` key.
2. ****Reset metric-invariance for non-whole-second t0**** (lines ~619-630). Currently subtracts full-precision `t0\_sec=t0\_frame/fps` while serializing at 6dp ? reset-vs-no-reset scores diverge on sub-second offsets (worst `|?|=1.0000000000001e-06`, past `\_FLOAT\_TOL`). **FIX:** round `t0\_sec` to 6dp (`\_round`) BEFORE subtracting from BOTH the labels and (consistently) deriving the trajectory shift, so the applied shift is an exact 6dp quantity ? bit-identical scores. Update the docstring/claim to match. Replace the single 9000-frame invariance test with a PARAMETRIZED test over fractional-second offsets (e.g. 7, 11, 13 frames) asserting score-equality AND `|?| <= \_FLOAT\_TOL`.
3. ****Partial-write + negative-label crash**** (writes at ~635-636,671 happen BEFORE `replay\_fixture`/scan). **FIX:** (a) BEFORE writing, validate every reset label start ≥ 0 ? raise `RefusalError` ("label timeline predates the trajectory") if any is negative; (b) write the fixture into a temp dir and promote (os.replace/rename) to the final slug dir only AFTER the scan passes, so a refused/failed freeze leaves NO partial fixture dir. Add a test: after a scan/validation refusal the fixture dir does not exist.
4. ****`\_scan\_forbidden` source-path skip over-skips**** (line ~530): `any(t in lit for lit in \_SCHEMA\_FILENAME\_LITERALS)` skips tokens like 'label'/'traj' that are substrings of schema literals ? a real source named `label.csv` leaking its stem is MISSED. **FIX:** use exact membership `t in \_SCHEMA\_FILENAME\_LITERALS`. Add a test (source path `/x/label.csv` + an identifying note ? caught).

5. **``compare\_expected` extra-key blind spot**\*\* (lines ~793-830): the ``score`` block and top-level keys are compared over FIXED field lists, so an EXTRA key on one side is ignored. FIX: add ``set(...)==set(...)`` key-set equality on the ``score`` dict AND the top-level dict (mirroring the shuttle-block check) before the per-field loops. Add a negative test (injecting an extra key returns a non-empty mismatch list).
6. **``refresh\_snapshot --reason`**: docstring/doc say it's REQUIRED but ``main()`` only WARNS (lines ~451, ~955-957). FIX: make ``--reason`` mandatory for ``--refresh-snapshot`` (``ap.error`` if absent, like the ``--freeze-real`` required-args check). Add a test.
7. **``load\_fixture` schema guard**\*\* (~761-764): guard ``expected`` AND ``provenance`` schema_version INDEPENDENTLY (raise if EITHER > CURRENT_SCHEMA). Add a negative test (provenance sv=99 while expected sv=1 ? raises).
8. **``max\_window\_duration` (W1) pinned-but-ignored**: ``PINNED_MAX_WINDOW`` is written to provenance/manifest but ``_pinned_proposal`` returns only (vt,mg,mwd) and ``build_candidates`` takes no max\_win. DO NOT modify ``distill_local.build_candidates`` (shared code). Instead add a clear code comment + docstring note that ``max_win`` is RECORDED but NOT applied by the replay (W1 bounded-windowing is OFF/uncharacterized by design; the fixtures characterize the W1-OFF path), so no one assumes it tracks a future SERVED W1 flip.
9. **Over-claim "pure-stdlib / no numpy"**** (docstring lines ~8-9,34-35 + ``eval_baselines/fixtures/README.md``): correct to "avoids numpy EXECUTION on the replay path (numpy is still imported transitively via experiment?classifier and must be installed)".
10. **Missing FAILURE-semantics negative tests**** (add to tests/test_golden_real_fixtures.py or test_golden_fixtures.py): ``compare_expected`` returns NON-empty for a changed-float-(>1e-6) / changed-int / candidate-reorder / extra-key case; ``_scan_forbidden`` FORBIDDEN_KEYS branch and MD5 branch each raise; ``refresh_snapshot`` no-target raises ValueError.
11. **test_synthetic_provenance_tags` forward-trap**** (tests/test_golden_fixtures.py:~70-77): it parametrizes over ALL manifest slugs and hard-asserts kind=='synthetic' ? a future ``cleared_real`` fixture would red the suite. FIX: assert by kind (synthetic tags for kind=='synthetic'; for kind=='cleared_real' assert owned/minors_free/non-blank license), and rename to reflect it gates ALL Tier-0 fixtures.

=== CI ? .github/workflows/ci.yml ===

12. Add ``python -m pytest tests/test_golden_real_fixtures.py -q`` to the ``golden-crossos`` job step (it currently runs only ``--check`` + test_golden_fixtures.py). The file is synthetic/tmp_path (no GPU/network) ? this gives the freeze-real/reset/M2 paths cross-OS coverage and surfaces fix #2 on win/mac.

=== DOC ? docs/GOLDEN_REGRESSION_FIXTURES.md (reconcile to shipped reality) ===

13. **``\$4c/\$4b/\$3/\$5 raw-CSV honesty**:\*\* the doc says "never commit the per-frame (Frame,Visibility,X,Y) CSV / only ~5 aggregate scalars" but the code COMMITS the full per-frame ``trajectory.csv`` (de-attributed via time-origin reset + random slug + metadata strip). Correct these sections to state the per-frame trajectory CSV IS committed (de-attributed), and add a prominent ? note in \$4c (and the changelog) that the **``owner's 2026-06-16 risk acceptance must be RE-CONFIRMED``** on this corrected basis (the per-frame stream ? the de Montjoye fingerprint ? is committed, not just 5 scalars). Do NOT claim minimization-to-5-scalars anywhere.
14. **``\$4a``** over-claims the replay reuses ``windowing.build_windows``/``WindowingPreset``/``"default SERVED"`` ? the shipped replay calls ``distill_local.build_candidates`` with a raw 3-tuple and imports none of windowing.py. Correct \$4a to match the module docstring's accurate map.
15. **``Metric-invariance wording``** (\$ ~94): update to match fix #2 (invariant after 6dp-rounded shift).
16. **``fps/fw quantization``** wording: reconcile \$4c with \$7 ? (quantization is a deferred P3 option, NOT active).
17. Add a row to the \$7 **``Progress log``** table for this PR (audit-hardening; note coverage).

=== VERIFY (run in the worktree; report verbatim) ===

1. ``python -m backend.eval.golden_fixtures --check`` ? the 3 synthetic fixtures MUST still pass (you didn't change synthetic replay). If they don't, STOP and report.
2. ``python -m pytest -q --cov=backend --cov=training --cov-fail-under=70`` ? FULL suite green; report pass/skip counts + total coverage (must stay >= 80.4%).
3. ``ruff check .`` and ``mypy backend training`` ? clean.
4. Demonstrate fix #1: ``freeze_real_fixture(label="mens singles final", ...)`` and a ``source_note`` with a name are rejected/not-persisted (show the test or a quick repro into a tmp dir, NOT eval_baselines/fixtures/).

Report: files changed, verbatim test/lint/type/coverage results, and any item you could NOT do (with the reason). Do NOT commit any real fixture. Do NOT run git.

2. assistant (2026-06-16T06:41:15.449Z)

I'll start by entering the worktree and reading the key files to understand the current state.

3. assistant (2026-06-16T06:41:19.660Z)

I'm instructed to work in the existing worktree. Let me enter it via its path.

4. user (2026-06-16T06:41:20.246Z)

Cannot enter worktree: the current working directory C:\Users\avidu\Projects\badminton-highlight-indexer is the repository root, not an isolated worktree ? switching is only available to sessions whose working directory is inside a worktree of this repository.

5. assistant (2026-06-16T06:41:24.361Z)

The EnterWorktree tool can't switch from root. I'll work directly with absolute paths to the worktree, as instructed (do NOT run git; create/edit files + run tests only). Let me start by reading the key files.

6. user (2026-06-16T06:41:25.783Z)

```
1      # Golden Regression Fixtures ? video-free, non-attributable regression suite
2
3      > **Status:** `IN PROGRESS` ? owner-gated . **Owner:** Avi . **Created:** 2026-06-16 .
**Last updated:** 2026-06-16 . **Legal:** owner **accepted the researched risk** 2026-06-16
(conditions §3 stand; this is the owner's informed acceptance, **not** a retained-counsel
opinion)
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE, per archive policy)
5      > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
6      > **Relation to existing docs:** EXTENDS (does not supersede)
`docs/GOLDEN_DATA_SHARING.md` (#175); reuses the eval stack documented in `docs/EVALUATION.md` /
`docs/EXPERIMENT_HARNESS.md`; quality floor ties to `docs/QUALITY_ITERATIONS.md`.
7      > **Honesty note:** §3 (Legal) is **researched general information, NOT legal advice** ?
it gates on retained counsel (§8). §2/§4/§7 mark which claims are **[verified]** against code vs
**[design]** proposals.
8
9      ---
10
11     ## 0. TL;DR
12
13     Let a contributor on **another machine with no video, GPU, WSL, or DB** run the rally-
segmentation regression suite from **committed structured files**, so refactors and non-quality
PRs get a red/green side-effect signal. Two layers, built once and grown by accretion:
14
15     1. **Mechanics** ? freeze the deterministic pipeline at the **post-detector trajectory
seam** and replay it through the *already-shipped* eval stack (`experiment.py` / `metrics.py` /
`windowing.py` / `regression.py`), with two auto-discovered gates: a
**characterization/snapshot** gate (catches any behaviour change) and a **quality-floor** gate
(catches quality drops).
16
17     2. **Privacy/legal filter (SEALED-FIXTURES)** ? only **de-attributed, minimized, owner-
source, biometric-free** numbers are ever committed; richer/sensitive streams stay key-gated or
out-of-repo. Non-attributability comes from **deletion-at-source**, not encryption.
18
19     **The convergence that makes this clean:** the *privacy* answer (publish shuttle-only,
drop person/pose streams) and the *determinism* answer (freeze only the pure-stdlib `CONTROL`
path, keep the numpy/BLAS learned scorer off the cross-machine gate) point at the **same minimal
public tier**. That coincidence is the spine of the design.
20
21     **Two findings already actionable, independent of the rest:**
- ? **Live de-attribution bug** *(verified)*: real player/venue names are committed
today in `eval_baselines/heuristic_lovo_n6.json`,
```

```

`eval_baselines/gemini_wrapper_2026-06-10.json`, and `backend/eval/eval_partial_labels.py` (the
last also has a personal `C:/Users/avidu/...` path) ? and in git history. ? **Deliverable P0.**
22 - ? **Determinism trap** *(verified, corroborated by two independent red-teams)*: the
learned variants (`shuttle_only`, `fusion`) train a numpy/OpenBLAS `DYNAMIC_ARCH` logistic
regression whose decisions flip near the 0.5 threshold across CPUs ? would false-RED on a
contributor's machine. ? the cross-machine gate must be **CONTROL/static-only**.
23
24 ---
25
26 ## 1. Problem & goal
27
28 The rally-quality measurement is split across two machines by capability
(`docs/GOLDEN_DATA_SHARING.md`):
29 - `backend/eval/fusion_golden.py` **extracts** `_golden_features.json` ? needs the
**video + trajectory CSV + labels + GPU/WSL** (GPU box only).
30 - `backend/eval/fusion_compare.py` / `backend/eval/ablation.py` **re-score** those JSONs
on **pure CPU, no video** ? any machine.
31
32 The CPU re-scoring is *already* video-free, but the inputs live in gitignored `output/`
/ private OneDrive, so the regression tests that depend on them **skip everywhere**
(`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`). The
committed `eval_baselines/` baselines exist but `regression.py` still scores DB-stored
predictions (needs a pipeline run on the video).
33
34 **Goal:** commit a tiny, version-tagged, **numeric-only, non-attributable** per-fixture
corpus + auto-discovered gates so a clean `git clone` / CI gets a real red/green regression
signal with **zero video/GPU/DB**, while never leaking attributable or sensitive data.
35
36 ---
37
38 ## 2. Decisions locked
39
40 | # | Decision | Source | Implication |
41 |---|-----|-----|-----|
42 | D1 | **Corpus is owner-recorded / owned** | owner answer 2026-06-16 | Legal analysis
takes **Fork A** (clean): no third-party copyright/ToS infringement; concern is trade-
secret/moat + privacy. Strongest non-attributability (no public reference to correlate against).
|
43 | D2 | **Repo private now, may open-source later** | owner answer 2026-06-16 | Design
for **public-grade hygiene from day one** ? git history is permanent; nothing scrubable-later. |
44 | D3 | **Adults, consent not formalized** | owner answer 2026-06-16 | **Person/pose/gait
streams stay OUT of the public tier** (DPDP/GDPR derived-personal-data risk without consent).
Public tier = **shuttle + abstract intervals + non-biometric features only**. |
45 | D4 | **Freeze seam = post-detector trajectory** *(design)* | workflow-1 synthesis |
Widest deterministic surface; reuse the shipped eval stack verbatim; the raw per-frame CSV is
**never committed** (only ~5 aggregate, fps/res-invariant shuttle scalars/window). |
46 | D5 | **Cross-machine gate is CONTROL/static-only** *(design)* | both red-teams
[verified cause] | Learned variants (numpy/BLAS) are **owner-box/Tier-1 only**, asserted with
tolerance, never as a cross-machine byte-snapshot. |
47 | D6 | **Non-attributability = deletion-at-source, encryption = public-only barrier**
*(design)* | workflow-2 synthesis | A contributor who runs the tests necessarily reads
plaintext+key (DRM/analog-hole). So we **delete** attribution, not hide it. Encryption only
stops the no-key public. |
48 | D7 | **Two tiers** *(design)* | workflow-2 synthesis | **Tier-0** public/no-key
(shuttle-only, de-attributed). **Tier-1** key-gated/out-of-repo (person/audio, full corpus),
**skips cleanly when absent**. |
49
50 ---
51
52 ## 3. Legal foundation (researched ? gates on counsel, §8)
53
54 **Core verdict (settled across IN/US/EU):** the derived numbers ? per-frame shuttle
(x,y), rally start/end intervals, audio/court/aggregate-person feature values ? **do NOT inherit
the source video's copyright.** They are uncopyrightable *facts/measurements*, not authored
expression, and not a §101 "derivative work."

```

55

56 | Jurisdiction | Copyright of the data | Database right | Contract / ToS | Net |

57 |---|---|---|---|---|

58 | ****India**** (primary) | Not copyrightable as facts (**EBC v. D.B. Modak**); but compilations get thin protection on a low "skill/labour" bar (**Burlington*, *OLX v. Padawan**) ? only bites if ****lifted from a 3rd-party dataset****, not self-derived | ****None**** (no sui-generis right) ? a real permissiveness edge | Anti-scraping ToS (untested in court) + ****IT Act s.43/s.66**** civil/criminal exposure for 3rd-party ingestion | ****Owned-source + minors-excluded + no person/pose = clean**** |

59 | ****US**** | Not copyrightable (**Feist**; §102(b); **NBA v. Motorola**) | None (**Feist** killed sweat-of-the-brow) | ****Dominant risk.**** ToS enforceable where copyright/CFAA fail (**ProCD**, **hiQ** ? injunction + ****forced deletion****). Remedy for redistribution is takedown, not nominal damages | Publishable as facts ****iff**** owned/permissioned source + no barring ToS + no person/biometric/minor stream |

60 | ****EU / UK**** | Not copyrightable (**Football Dataco v. Yahoo**) | ****Genuinely contestable.**** **Football Dataco v. Sportradar**: recording pre-existing on-court facts = ****obtaining****, not "creating" ? our "we created it" premise is ****overconfident****; harm test = **CV-Online Latvia** | *Ryanair v. PR Aviation*: ToS can ban extraction/redistribution even with no IP right | Get EU+UK advice before redistributing numeric streams at scale |

61

62 ****Provenance is the decisive fork.**** ****Fork A** (owner-recorded, unpublished) ? the only path safe to commit publicly; collapses to a trade-secret **choice** (Indian breach-of-confidence / Contract Act 1872 / IT Act s.72 ? ****not**** US DTSA) + privacy. ****Fork B** (broadcast/YouTube/scraped) ? do ****not**** commit; the extraction act needs a license or now-****contested**** US fair use (**Thomson Reuters v. Ross** 2025; USCO May-2025), and unlawful acquisition independently sinks it (**Bartz v. Anthropic**, ~\$1.5B settlement). Caveat: owner footage ****shot at a third party's event**** (tournament/club) carries ticket/venue/organiser/broadcaster terms ? treat as Fork B until cleared.

63

64 ****Publish-by-stream (privacy):****

65 - ? ****Publish-OK:**** shuttle (x,y)+Visibility (inanimate object), abstract rally/GT ****intervals**** (identity-stripped), non-biometric audio/court/aggregate-person features detached from identity.

66 - ?? ****Conditional (effectively never in v1):**** person bounding boxes that **persistently single out a player** ? flips to GDPR Art.9 the moment it identifies. This architecture is prone to that flip ? treat per-player tracking as a tripwire.

67 - ? ****Never publish:**** pose/skeleton/****gait**** (re-identifies at 80?95% even with faces blurred ? **Weiss et al. 2025**), face crops, any named-player?movement mapping. Matches the existing "gait/biometrics strictly future (GDPR Art.9)" guardrail.

68 - ? ****Minors flip everything:**** DPDP "child" = under 18; s.9(3) ****prohibits behavioural monitoring**** (the Rule-12 exemptions don't cover this project). A persistent per-player signature on a minor is, conservatively, prohibited. ****Exclude minors**** from any persisted stream beyond inanimate shuttle/interval data.

69

70 ****Non-attributability is relative, not absolute.**** A per-video trajectory is a fingerprint (de Montjoye: 4 points ? 95% re-ID). It's only non-attributable because the ****Fork-A source stays unpublished**** (no public reference to correlate against). Per **EDPS v. SRB** (CJEU C-413/23 P, 2025) it ****remains personal data to the holder**** who can re-run the detector. So: conditional, revocable (publish the source and the link snaps back), forward-looking (re-assess as the public-video environment grows). Never market it as "anonymous."

71

72 ***Key authorities:** Feist 499 U.S. 340; NBA v. Motorola 105 F.3d 841; 17 U.S.C. §101/§102(b); Sega v. Accolade 977 F.2d 1510; Football Dataco v. Yahoo (C-604/10) & v. Sportradar (C-173/11); BHB v. William Hill (C-203/02); CV-Online Latvia (C-762/19); Ryanair v. PR Aviation (C-30/14); Thomson Reuters v. Ross (D. Del. 2025); EBC v. D.B. Modak; DPDP Act 2023 s.3/s.9 + Rules 2025; EDPS v. SRB (C-413/23 P). Full URLs in the research artifact (§10).

73

74 ---

75

76 ## 4. Architecture (design)

77

78 ### 4a. Freeze seam + reuse map

79 Freeze at the ****trajectory CSV ? pure-Python replay****. One shared helper ``backend/eval/golden_fixtures.py::replay_fixture()`` is the **single** code path used by both the freeze tool and both gates:

```

80     `parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates
? rally_gate net_crossings ? experiment.predict(CONTROL) ? merge_overlaps ? metrics.score`.
81     Reuses verbatim: `metrics.score/interval_iou`, `segmentation_metrics`,
`experiment.predict/compare/Variant.spec_hash`, `windowing.WindowingPreset` (default SERVED),
`regression.gt_hash/save_baseline + incommensurability guards`, `eval_stats` (seed=0),
`eval_baselines/` committed-baseline precedent.
82
83     ### 4b. Tiered model (SEALED-FIXTURES)
84     | Tier | Where | Key? | Contents | Test behaviour |
85     |---|---|---|---|---|
86     | Tier-0 | `eval_baselines/fixtures/` (tracked) | none | opaque `fixture_id`,
fps/frame_width (quantized), relative `start`/`end`, 5 shuttle scalars, label, relative
`gts`. No person/audio/pose. | Unconditional ? runs on every PR/fork; turns today's
skipped litmus into a real public gate |
87     | Tier-1 | OneDrive (default) or SOPS-encrypted in-repo | token/age key | full
corpus with person/audio; authoritative ?F1/CI/sign-test + exact Gate-A litmus | Skips
cleanly when key/data absent (generalize `_golden_present()` ? `_tier1_present()` ) |
88
89     A fresh no-key clone is always green; coverage never depends on a secret.
90
91     ### 4c. Anti-attribution measures (Tier-0)
92     - Opaque RANDOM surrogate id (`os.urandom` + a private surrogate?source map kept
off-git) ? not the MD5 `video_id`, and not HMAC(salt, video_id?name) (red-team: brute-
forceable over a tiny known roster if the salt leaks). Strip the `name` field (today = source
filename).
93     - Metadata strip ? no filename, video_id, match/event/player identity, capture date.
94     - Time-origin reset ? subtract per-fixture `t0` from all start/end/gts. Provably
metric-invariant (`metrics.interval_iou` and start/end-error are pure differences) ? bit-
identical scores, severed absolute timeline.
95     - Stream minimization ? Tier-0 keeps shuttle block only; person/audio excluded;
pose/gait/face have no field in the schema by design.
96     - Raw-trajectory exclusion ? never commit the per-frame `(Frame,Visibility,X,Y)`
CSV.
97     - Quantize fps/frame_width ? red-team: at n=6 the raw values are a camera/venue
quasi-identifier.
98     - Provenance hard-fail gate ? the de-attribution pass refuses to emit a public
fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + biometric-excluded.
99
100    ### 4d. The two gates
101    - Characterization/snapshot ? replay fixture, parse-compare JSON (never byte-
compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields,
`approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints
(incommensurable ? loud fail). Catches any behaviour change even at constant F1. CONTROL path
only across machines.
102    - Quality-floor ? score vs committed labels; assert per-video + mean F1 ? floor in
`eval_baselines/regression_baseline.json` (created here; carries a windowing fingerprint so
a served-default change is flagged incommensurable, not silently compared); emit `eval_stats`
paired-delta-CI + sign test (NUMBERS INFORM); add a metrics-vs-base-branch paired delta.
Synthetic-tier floors measure correctness, not field quality (documented).
103
104    ### 4e. Serialization & optional protobuf
105    JSON with `sort_keys=True` + pre-quantized floats + LF is sufficient. Protobuf is
optional (01) ? adopt only if cross-OS byte-stable snapshots are wanted (proto3
`rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `*.pb` binary -text`).
Protobuf is encoding, not secrecy (trivially `--decode_raw`).
106
107    ### 4f. Encryption & keys (Tier-1 only)
108    Default = out-of-repo + token (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero
sensitive bytes in git history, sidesteps the legal "redistribution" act). Alternative = SOPS
+ age (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting
in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI
secret exposed only to the Tier-1 job on trusted branches, never to fork-PR runners. The
HMAC/surrogate salt (if used) and the surrogate?source map live with the private corpus,
rotated per release. Document explicitly that Tier-1 crypto defends only the no-key public.
109

```

```

110    ---
111
112    ## 5. Threat model (who's protected vs not)
113
114    | Actor | Protected against | NOT protected (by design / residual) |
115    |---|---|---|
116    | **TM1 no-key public** | map fixture?source/match/player; decrypt Tier-1; confirm a
source even holding the candidate file | read Tier-0 numbers + run shuttle-only harness
(intended; useless-in-isolation) |
117    | **TM2 keyed contributor** | recover filename/video_id/match/face/pose/gait ? *never
written* (survives the analog hole) | read every value the suite reads (unpreventable; not
pretended otherwise) |
118    | **TM3 CI runner** | leaked log can't leak attribution it never had | runs the suite
(intended); Tier-1 secret never on fork runners |
119    | **TM4 malicious keyed insider** | correlation needs a public reference; Fork-A source
is unpublished ? "not reasonably likely" | **residual:** conditional + revocable ? publish/leak
the source later and the link can snap back (forward-looking re-test duty) |
120    | **TM5 re-ID adversary** | source unpublished + only ~5 invariant scalars/window +
reset timeline ? no back-match | becomes live if a Fork-A source is ever published |
121    | **TM6 accidental plaintext commit** | required CI leak-scan rejects MD5/known-
name/non-surrogate/plaintext-SOPS | if the required check is disabled, a leak lands |
122
123    ---
124
125    ## 6. Honest limits ? what a green suite does NOT prove
126
127    - Green proves only that the **deterministic post-trajectory transforms** are unchanged
for the **frozen cases**. It does **not** cover: the WASB/TrackNet detector,
`_probe_fps_width`/cv2 frame-seek, the AI/provider handoff, chunking/re-encode, the ffmpeg
stitch, DB persistence/migrations, or the API filter. *A refactor that breaks decoding or the
detector passes these gates green.* The owner-box `regression.py` DB run remains the only full-
chain check.
128    - Characterization = **behaviour-locking, not correctness** ? it freezes current
behaviour *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour
correct". This caveat must ride in the **test pass/fail message**, not only the docs.
129    - **Synthetic-tier floors = plumbing correctness, NOT field quality.** Never cite
synthetic F1 in `QUALITY_ITERATIONS.md` / the paper as rally-detection performance (tag `kind:
synthetic`).
130    - Tier-0 public coverage is **shuttle-only** ; the person/audio fusion ?F1 and exact
Gate-A values need Tier-1 (skipped on no-key clones).
131    - Data is **not "anonymous" absolutely (EDPS v. SRB) ? relative, conditional,
revocable, forward-looking.
132    - Encryption gives **no secrecy from a running contributor** ; protobuf gives **no
secrecy at all**.
133    - De-attribution does **not** cure provenance/licensing ? it sits *behind* the per-
record provenance gate + counsel sign-off.
134    - `--refresh-snapshot` can rubber-stamp a real regression; mitigation is **review of the
visible diff** + a separate, deliberate floor-baseline update. Require a recorded reason.
135
136    ---
137
138    ## 7. Deliverables & progress tracker ? **source of truth**
139
140    Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One **small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
141
142    | ID | Deliverable | Depends on | Counsel-gated? | Status | PR |
143    |---|-----|-----|-----|-----|---|
144    | **P0** | **Leak remediation + required CI leak-scan.** Scrub real names/paths in
`eval_baselines/heuristic_lovo_n6.json`, `gemini_wrapper_2026-06-10.json`,
`backend/eval/eval_partial_labels.py`, `tests/test_cleanup_caches.py` ? opaque surrogate keys +
private map; add CI leak-scan (MD5 / known-name / non-surrogate / plaintext-SOPS) as a
**required status check** ; decide on git history (PR #77, accept vs `filter-repo`). | ? | No
(owned data hygiene) | ? | ? |
145    | **M1** | **Fixture framework + synthetic Tier-0 + shared replay helper** (also

```

delivered M2-core + M3 ? see below). `backend/eval/golden\_fixtures.py` (`replay\_fixture`, schema dispatch, `Path(\_\_file\_\_)`-relative) + 3 **tool-generated** synthetic fixtures + manifest + README + `.gitattributes` LF pin + `tests/test_golden_fixtures.py`. CONTROL-only / pure-stdlib; cross-OS verified. | ? | No (synthetic) | ? | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) |

146 | **M2** | **Freeze/refresh extractor.** Core (`freeze\_synthetic` / `refresh\_snapshot` / `--check` CLI + idempotence test) shipped in M1 (#189); the `--license`/`--minors-free`/`--owned`` **refusal gate** shipped with P1 (this PR). | M1 | No | ? |

[#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) · `_this PR` |

147 | **M3** | **Characterization gate** ? `tests/test_golden_fixtures.py`: parse-compared (exact-int / approx-float 1e-6), **CONTROL-only**, positive no-person/audio assertion, perturbation + idempotence. **Shipped in M1 (#189).** (Windowing/label fingerprint guard + row-count cap ? fold into M4.) | M1 | No | ? | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) |

148 | **MX** | **Cross-OS determinism guard (CI).** Lightweight `golden-crossos` CI job (ubuntu/windows/macOS) running `--check`` + the fixtures test ? locks the cross-OS guarantee (CONTROL replay is numpy-only, no GPU stack; a *separate fast job*, not full-suite×3). | M1 | No | ? | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) |

149 | **M4** | **Quality-floor gate + first `regression\_baseline.json`.** `tests/test_golden_quality_floor.py`; create `eval_baselines/regression_baseline.json` with **windowing fingerprint**; metrics-vs-base paired-delta; corpus-level (not per-slug) LOVO for runtime. | M1, M2 | No | ? | ? |

150 | **P1** | **De-attribution + minimization** ? `golden_fixtures.freeze_real_fixture()` (extends the M1 module, not a separate script): **opaque random `fx\_hex` surrogate slug**, metadata strip, metric-invariant **time-origin reset**, strict GT loader, **positive no-person/audio/MD5/source-path assertion**, manifest non-clobber; `--freeze-real`` CLI with the **refusal gate** (exit 2 unless owned + minors-free + license). Tooling-only, 12 synthetic tests; **no real data committed**. | M1 | No (tooling) | ? | `_this PR` |

151 | **P2** | **Owner-accepted (researched-risk), 2026-06-16.** Owner accepted the researched IP/privacy analysis (§3) to proceed **under its conditions** (owned/Fork-A source · minors excluded · biometric/person streams excluded · de-attributed). Owner's informed risk acceptance ? **NOT a retained-counsel opinion**; optional future counsel confirmation per §8. | P1 | Owner-accepted | ? | ? |

152 | **P3** | **Tier-0 REAL subset.** Commit de-attributed, minors/biometric-excluded, owned Fork-A real fixtures (unblocked by P2). Hard-gated **in code** by P1's refusal gate (owned + minors-free + license required) ? needs the owner's per-video clearance list. | P1, P2 | conditions §3 | ? | ? |

153 | **O1** | **(optional) Protobuf + deterministic serialization.** `rally.fixture.v1`` proto3, codegen + `protoc`/`git diff --exit-code`` guard, canonical writer, `.gitattributes`` binary. Only if cross-OS byte-stable snapshots are wanted. | M1 | No | ? | ? |

154 | **O2** | **(optional) Tier-1 key-gated path.** `scripts/fetch_golden.py`` (OneDrive token, default) and/or SOPS+age; `_tier1_present()`; `RALLY_GOLDEN_DIR`` wired; Tier-1 CI job gated on secret, never on fork runners. | M1 | No | ? | ? |

155 | **DOC** | **Discoverability cross-links** ? added the `eval_baselines/fixtures/`` row to `CODE_MAP §0`` + a `golden_fixtures.py`` entry to `CODE_MAP §2.3``; cross-linked `GOLDEN_DATA_SHARING.md`` (committed subset ? private OneDrive corpus). README indexed in #186; memory `current-state`` kept live. (EVALUATION/QUALITY_ITERATIONS cross-links **deferred** ? that doc surface is hot in the concurrent quality track; avoid collision.) | M1 | No | ? | `_this PR` |

156

157 **Must-fix-before-any-public-commit (red-team), folded into the rows above:** ? numeric-tolerance not byte-equality + multi-OS matrix + CONTROL-only (M3/D5/MX ?); ? scrub names in **all** files + history decision (P0); ? leak-scan a **required** check, hooks are convenience only (P0); ? person-optional loader + **positive** no-person assertion, don't trust the silent `0.0`` default (P1 ?); ? random surrogate id not HMAC-over-roster (P1 ?); ? quantize/drop fps/frame_width (P1 ? fps/fw retained as scale constants; quantize is a P3 option); ? document Tier-0 omits the fusion path (DOC); ? counsel + Fork-A + minors/biometric exclusion as hard preconditions (P2 owner-accepted; enforced in code by P1's refusal gate ?).

158

159 **Progress log (quantitative ? coverage must hold/improve, CI incl. the 3-OS cross-OS job green)**

160 | PR | Deliverable(s) | Suite | Coverage | Notes |

161 | ---|-----|-----|-----|-----|

162 | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) | M1 + M2-core + M3 | 746 pass / 7 skip | 80.41% | synthetic Tier-0 + CONTROL replay +


```

characterization; cross-OS proven (CI Linux replayed Windows-frozen fixtures) |
163 | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) | MX +
tracker/legal reconcile | full suite green | 80.41% | `golden-crossos` CI job green on
ubuntu+windows+macos ? cross-OS **enforced**; legal ? owner-accepted |
164 | _this PR_ | P1 + M2 refusal gate | 759 pass / 7 skip | **80.49%** (+0.08) |
`freeze_real_fixture` + refusal gate; **+13 tests**; tooling-only, no real data committed |
165
166 ---
167
168 ## 8. Open owner / counsel questions
169
170 **Counsel questions ? documented basis (the owner accepted the researched risk on
2026-06-16; see P2). These remain the record, and a retained-counsel confirmation is
optional/recommended before scaling beyond a small owned, minors-free, de-attributed subset:**
171 1. Does the EU/UK **sui-generis database right** subsist in our self-derived
shuttle/rally streams on the *Sportradar* "obtained vs created" reasoning, and does
redistributing a de-attributed Tier-0 subset cause *CV-Online* harm?
172 2. Post *Thomson Reuters v. Ross* + USCO-2025, is **US fair use for the extraction
step** defensible if the dataset could compete in a "data/analytics" market ? does publishing
only numbers (never footage) matter?
173 3. India: are anti-scraping ToS enforceable; **IT Act s.43/s.66** exposure for any 3rd-
party ingestion; does pending *ANI v. OpenAI* (~Apr 2026) move it? Confirm **DPDP s.9/Rule 10**
and whether per-player tracking is prohibited s.9(3) monitoring of a minor.
174 4. Sign-off on the **final committed Tier-0 subset + provenance log**; is non-
attributability-via-private-source a defensible posture to assert given its
conditional/revocable nature?
175
176 **Owner (strategic):**
177 5. Publishing the Fork-A subset **forfeits the trade-secret moat** (Indian breach-of-
confidence framing) ? weighed against community/benchmark/paper aims. Your call.
178 6. **Forward-looking commitment:** do not publish a highlight reel of the *same* source
whose fixtures are committed (without re-clearing), and accept a periodic re-assessment as the
public-video environment grows.
179 7. **Scope of first public commit:** synthetic-only (zero provenance risk, recommended)
vs a counsel-cleared real subset.
180 8. **Tier-1 mechanism:** out-of-repo+token (recommended) vs SOPS+age in-repo. And: is
protobuf (01) wanted, or is canonical-JSON enough?
181
182 ---
183
184 ## 9. Definition of done
185
186 The project is **DONE** (flip `Status ? DONE`, archive) when:
187 - [ ] **P0, M1?M4, P1, DOC** are ? and merged.
188 - [ ] A clean `git clone` on **win + ubuntu + mac** runs `python run_all_tests.py` and
the **Tier-0 gates pass green with no key / no video / no GPU / no DB**.
189 - [ ] The **CI leak-scan is a required status check** and passes (no MD5 / real-name /
non-surrogate / plaintext-SOPS in the tree).
190 - [ ] Characterization + quality-floor gates are **parse-compared, CONTROL-only,
windowing-fingerprinted**, and demonstrably catch a seeded behaviour change.
191 - [ ] `docs/CODE_MAP.md` + TEST MAP updated; this doc cross-linked; `NEXT_STEPS.md` +
memory `current-state` point here.
192 - [ ] **Either** the public commit is synthetic-only, **or** (**P2 owner-accepted ? +
P3**) for a real-data commit, under the §3 conditions enforced **in code** by P1's refusal gate.
193 - [ ] §6 honest-limits caveat rides in the test pass/fail messages, not only the docs.
194
195 *Optional 01/02 may remain ? without blocking DONE (they're earned, not assumed).*
196
197 ---
198
199 ## 10. References
200
201 **Internal code anchors (verified):** `backend/eval/{regression.py,fusion_golden.py,fusi
on_compare.py,ablation.py,experiment.py,metrics.py>windowing.py,distill_local.py,gt_loader.py,ev
al_stats.py}` · `backend/pipeline/detectors/{base.py,tracknet_runner.py,rally_gate.py}` ·

```

```

`backend/utils/hashing.py::compute_video_id` · `eval_baselines/{heuristic_lovo_n6.json,gemini_wra
pper_2026-06-10.json,experiment_leaderboard.json}` ·
`tests/test_ablation.py::test_litmus_reproduces_gate_a` · `.github/workflows/ci.yml`.
202    **Internal docs:** `GOLDEN_DATA_SHARING.md` (#175), `EVALUATION.md`,
`EXPERIMENT_HARNESS.md`, `QUALITY_ITERATIONS.md`, `MULTI_SIGNAL_FUSION_PLAN.md`, `CODE_MAP.md
$0`.
203    **Legal authorities:** see §3 (full URLs in the research workflow artifact). *This
section is researched general information, not legal advice.*
204
205    ---
206
207    ### Changelog
208    - **2026-06-16** ? Initial draft synthesized from two multi-agent workflows (mechanics
design + 2 critiques; legal memo IN/US/EU + SEALED-FIXTURES architecture + red-team). Verified
the live name-leak and the BLAS-determinism cause against code. Status: DRAFT, owner-gated.
209

```

7. user (2026-06-16T06:41:26.077Z)

```

1    """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3    `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4    replay code path** used by both the freeze tool and the characterization gate, and it
5    deliberately stays on the **pure-stdlib CONTROL path** (D5 in that doc): the learned
6    variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7    across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8    only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9    merges overlaps, and uses **no numpy**.
10
11    The replay reuses the already-shipped eval stack VERBATIM::
12
13        parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
14        ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
15        ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
16
17    Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
18
19    * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
20    provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
21    * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true`,
`license="synthetic"`.
22    * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
23    person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
24    * The windowing preset params ``(vt, mg, mwd, max_win)`` are **pinned inside each
fixture's
25    provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the
named
26    ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn
fixtures.
27
28    §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
29    suite proves only that the deterministic post-trajectory transforms are unchanged for
the
30    frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
31    that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
32    field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
33
34    Pure + offline + stdlib. The replay imports `tracknet_runner` (which is itself stdlib-

```

```

only
35     for the parse/window math ? no TF), but pulls in no numpy/cv2 on the CONTROL path.
36     """
37     from __future__ import annotations
38
39     import argparse
40     import json
41     import math
42     import os
43     import re
44     import sys
45     from dataclasses import dataclass
46     from pathlib import Path
47     from typing import Any, Dict, List, Optional, Sequence, Tuple
48
49     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
50     -----
51     from backend.eval import distill_local
52     from backend.eval.experiment import CONTROL, VideoCandidates, predict
53     from backend.eval.gt_loader import load_gt_intervals
54     from backend.eval.metrics import score
55     from backend.eval.regression import gt_hash
56     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
57
58     Interval = Tuple[float, float]
59
60     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
61     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
62     CURRENT_SCHEMA = 1
63
64     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
65     works
66     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
67     root.
68     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
69
70     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
71     merge_gap,
72     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
73     LITERAL
74     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
75     constant so a
76     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
77     fixture also
78     #: re-states these in its provenance; the replay reads them from there.
79     PINNED_VT = 0.02
80     PINNED_MERGE_GAP = 2.0
81     PINNED_MIN_WINDOW = 2.0
82     PINNED_MAX_WINDOW = 0.0 # 0 = bounded-windowing OFF (W1), matching the SERVED default
83
84     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
85     stated
86     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
87     exactly these.
88     SHUTTLE_FEATURES: Tuple[str, ...] = (
89         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
90     )
91     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
92         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
93     )
94
95     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
96     appear
97     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
98     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")

```

```

89
90     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
91     _FLOAT_TOL = 1e-6    # abs tolerance when parse-comparing recomputed vs committed floats
92
93     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
94     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
95     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
96
97
98     # =====
99     # Synthetic trajectory generation (deterministic ? NO random)
100    # =====
101
102
103    @dataclass(frozen=True)
104    class RallySpec:
105        """One in-flight rally segment: the shuttle oscillates across the net midline.
106
107        ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
108        period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
109        net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
110        """
111
112        start_frame: int
113        n_frames: int
114        period: float = 30.0
115        amplitude: float = 400.0
116
117
118    @dataclass(frozen=True)
119    class GapSpec:
120        """Dead-time between rallies ? what separates one window from the next.
121
122        ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
123        ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
124        end an active run so the next rally forms a distinct window.
125        """
126
127        start_frame: int
128        n_frames: int
129        kind: str = "still" # "still" | "occluded"
130
131
132    @dataclass(frozen=True)
133    class FixtureSpec:
134        """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
135
136        ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
137        "labels.csv". For a synthetic fixture they line up with the rallies by construction,
138        which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
139        field-quality one (§6).
140        """
141
142        slug: str
143        description: str
144        fps: float
145        frame_width: float
146        frame_height: float
147        segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
148        gts: Tuple[Interval, ...]
149        net_midline: float = 640.0
150

```

```

151
152     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
153                        ) -> List[Tuple[int, int, float, float]]:
154         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
155         rows: List[Tuple[int, int, float, float]] = []
156         y_mid = frame_height / 2.0
157         for i in range(spec.n_frames):
158             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
159             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
160             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
161             rows.append((spec.start_frame + i, 1, x, y))
162         return rows
163
164
165     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
166                    ) -> List[Tuple[int, int, float, float]]:
167         """Dead-time rows: resting-visible (still) or absent (occluded)."""
168         rows: List[Tuple[int, int, float, float]] = []
169         y_mid = frame_height / 2.0
170         for i in range(spec.n_frames):
171             if spec.kind == "occluded":
172                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
173             elif spec.kind == "still":
174                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
175             else: # pragma: no cover - guarded by builder
176                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
177         return rows
178
179
180     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
181         """Build the deterministic ``Frame, Visibility, X, Y`` rows for a synthetic
scenario."""
182         rows: List[Tuple[int, int, float, float]] = []
183         for seg in spec.segments:
184             if isinstance(seg, RallySpec):
185                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
186             elif isinstance(seg, GapSpec):
187                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
188             else: # pragma: no cover - guarded by dataclass shape
189                 raise TypeError(f"unknown segment type {type(seg).__name__}")
190         rows.sort(key=lambda r: r[0])
191         return rows
192
193
194     def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
195         """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
floats."""
196         lines = ["Frame,Visibility,X,Y"]
197         for frame, vis, x, y in rows:
198             lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
199         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
200
201
202     def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
203         """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
204         lines = ["start,end"]
205         for s, e in gts:
206             lines.append(f"{_fmt(s)},{_fmt(e)}")
207         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
208
209

```

```

210 # =====
211 # Built-in synthetic scenarios (the 3 committed slugs)
212 # =====
213
214
215 def _fr(seconds: float, fps: float) -> int:
216     return int(round(seconds * fps))
217
218
219 def builtin_specs() -> List[FixtureSpec]:
220     """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
221     windowing behaviour (one window / dead-time split / occlusion split)."""
222     fps = 30.0
223     fw, fh = 1280.0, 720.0
224     net = 640.0
225
226     # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
227     single = FixtureSpec(
228         slug="clean_single_rally",
229         description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
230         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
231         segments=(RallySpec(start_frame=0, n_frames=120),),
232         # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
233         gts=((0.0, 4.0),),
234     )
235
236     # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
237     # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
238     r0 = RallySpec(start_frame=0, n_frames=120)
239     gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
240     r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
241     multi = FixtureSpec(
242         slug="multi_rally_with_deadtime",
243         description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
244         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
245         segments=(r0, gap_still, r1),
246         gts=((0.0, 4.0), (7.0, 11.0)),
247     )
248
249     # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
250     # visibility break ? proves occlusion (not just low velocity) separates windows.
251     o0 = RallySpec(start_frame=0, n_frames=120)
252     gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
253     o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
254     occ = FixtureSpec(
255         slug="occlusion_break",
256         description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
257         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
258         segments=(o0, gap_occ, o1),
259         gts=((0.0, 4.0), (7.0, 11.0)),
260     )
261
262     return [single, multi, occ]
263
264
265 def _spec_by_slug() -> Dict[str, FixtureSpec]:
266     return {s.slug: s for s in builtin_specs()}
267

```

```

268
269 # =====
270 # The shared replay (CONTROL / pure-stdlib path)
271 # =====
272
273
274 def _fmt(x: float) -> str:
275     """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
276     ``0.0``."""
277     r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
278     if r == 0.0:
279         r = 0.0
280     return f"{r:.6f}"
281
282 def _round(x: float) -> float:
283     r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
284     return 0.0 if r == 0.0 else r
285
286
287 def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
288     """The windowing proposal tuple, read from the fixture provenance when present (the
289     pinned
290     params travel with the fixture), else the module-level pinned defaults. NEVER
291     windowing.SERVED."""
292     if prov and "windowing" in prov:
293         w = prov["windowing"]
294         return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
295     return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
296
297 def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
298     """Replay one fixture through the shipped CONTROL path; return its recomputed
299     snapshot.
300     Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
301     committed
302     ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
303     parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
304     shuttle
305     cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
306     Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
307     control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
308     then
309     merges overlaps; NO numpy is touched.
310     """
311     fdir = _resolve_dir(slug_or_dir)
312     slug = fdir.name
313     traj_csv = fdir / "trajectory.csv"
314     labels_csv = fdir / "labels.csv"
315     prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
316     else None
317
318     fps, fw = _fps_fw(prov, slug)
319     iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
320     proposal = _pinned_proposal(prov)
321
322     intervals, feature_rows = distill_local.build_candidates(
323         str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
324
325     # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
326     call
327     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']

```

```

unconditionally
324     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
325     features: Dict[str, List[float]] = {
326         name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
327     }
328     gts = load_gt_intervals(str(labels_csv), strict=True)
329     vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
330
331     preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
332     sc = score(preds, gts, iou)
333
334     candidates: List[Dict[str, Any]] = []
335     for j, (s, e) in enumerate(intervals):
336         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
337         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
338         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
339         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
340
341     return {
342         "slug": slug,
343         "schema_version": CURRENT_SCHEMA,
344         "iou": _round(iou),
345         "candidates": candidates,
346         "control_segments": [[_round(s), _round(e)] for s, e in preds],
347         "score": {
348             "iou_threshold": _round(sc.iou_threshold),
349             "num_pred": int(sc.num_pred),
350             "num_gt": int(sc.num_gt),
351             "true_positives": int(sc.true_positives),
352             "false_positives": int(sc.false_positives),
353             "false_negatives": int(sc.false_negatives),
354             "precision": _round(sc.precision),
355             "recall": _round(sc.recall),
356             "f1": _round(sc.f1),
357             "mean_iou": _round(sc.mean_iou),
358             "mean_start_error": _round(sc.mean_start_error),
359             "mean_end_error": _round(sc.mean_end_error),
360         },
361         "gt_hash": gt_hash(gts),
362     }
363
364
365     # =====
366     # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
367     # =====
368
369
370     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
371         ) -> Dict[str, Any]:
372         """The committed provenance block ? pins the windowing + privacy/licensing tags
($4c, $5)."""
373         desc = ""
374         if isinstance(spec_or_dir, FixtureSpec):
375             desc = spec_or_dir.description
376         return {
377             "slug": slug,
378             "schema_version": CURRENT_SCHEMA,
379             "kind": "synthetic",
380             "license": "synthetic",
381             "minors_free": True,

```



```

382         "description": desc,
383         "fps": _round(fps),
384         "frame_width": _round(fw),
385         "iou": _round(iou),
386         "windowing": {
387             "vt": PINNED_VT,
388             "mg": PINNED_MERGE_GAP,
389             "mwd": PINNED_MIN_WINDOW,
390             "max_win": PINNED_MAX_WINDOW,
391         },
392         "paths": {
393             "trajectory": "trajectory.csv",
394             "labels": "labels.csv",
395             "expected": "expected.json",
396         },
397         "note": (
398             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
399             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
($6)."
400     ),
401 }
402
403
404 def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
405                   iou: float = 0.5) -> Dict[str, Any]:
406     """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
407     ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
408
409     Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
410     + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
411     """
412     fdir = fixtures_dir / slug
413     traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
414     if not traj_csv.is_file() or not labels_csv.is_file():
415         raise FileNotFoundError(
416             f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
417
418     # Provenance first (it pins the windowing the replay reads), then replay ? expected.
419     spec = _spec_by_slug().get(slug)
420     fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
421     prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
422     _write_json(fdir / "provenance.json", prov)
423
424     expected = replay_fixture(fdir)
425     _write_json(fdir / "expected.json", expected)
426     return expected
427
428
429 def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
430     """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
431     provenance.json, and (re)write the manifest. Returns the slugs frozen."""
432     specs = builtin_specs()
433     fixtures_dir.mkdir(parents=True, exist_ok=True)
434     slugs: List[str] = []
435     for spec in specs:
436         fdir = fixtures_dir / spec.slug
437         fdir.mkdir(parents=True, exist_ok=True)
438         rows = make_synthetic_trajectory(spec)
439         write_trajectory_csv(rows, fdir / "trajectory.csv")

```

```

440         write_labels_csv(spec.gts, fdir / "labels.csv")
441         freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
442         slugs.append(spec.slug)
443     _write_manifest(specs, fixtures_dir)
444     return slugs
445
446
447     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
448                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
449         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
450         labels.csv ?
451         NO regeneration of the trajectory/labels. ? $6: this can rubber-stamp a real
452         regression; the
453         mitigation is reviewing the visible diff + recording a reason (the CLI requires
454         one)."""
455         if all_slugs:
456             targets = [d.name for d in sorted(fixtures_dir.iterdir())
457                       if d.is_dir() and (d / "trajectory.csv").is_file()]
458         elif slug:
459             targets = [slug]
460         else:
461             raise ValueError("refresh_snapshot needs --slug S or --all")
462         for s in targets:
463             freeze_fixture(s, fixtures_dir=fixtures_dir)
464         return targets
465
466     # =====
467     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
468     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
469     # $4c anti-attribution, $5 threat model, $7 row P1, $6 honest limits.)
470     # =====
471
472     class RefusalError(ValueError):
473         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
474
475         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
476         it) that
477         names *why* the unsafe public-fixture path was refused. The refusal is the code-
478         enforced
479         provenance gate ($4c "provenance hard-fail gate", $7 ?): the owner-recorded /
480         minors-excluded
481         / license flags are human-asserted, but the gate makes the unsafe path impossible to
482         reach
483         *silently* ? a fixture cannot be written unless all three are affirmatively set.
484         """
485
486     def _new_surrogate_slug() -> str:
487         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` ($4c).
488
489         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
490         random,
491         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
492         over a
493         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
494         off-git.
495         """
496         return f"fx_{os.urandom(8).hex()}"
497
498     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-
499     path tokens
500     #: that collide with these are NOT a leak ? exclude them so a short source stem like

```

```

``traj`` can't
494     #: false-positive against the literal
``trajectory.csv``/``labels.csv``/``expected.json``.
495     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
496         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
497         "provenance", "provenance.json",
498     )
499
500
501     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
502         """POSITIVE privacy assertion ($4c, red-team?): raise if ``text`` leaks anything
attributable.
503
504         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
505         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
506         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
507         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
508         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
509         """
510         low = text.lower()
511         for key in FORBIDDEN_KEYS:
512             if key in low:
513                 raise RefusalError(
514                     f"{where}: forbidden key substring {key!r} found in written output ? "
515                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
516                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md $4c).")
517         m = _MD5_RE.search(text)
518         if m:
519             raise RefusalError(
520                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
521                 f"video_id shape and an attribution back-channel; never write it ($4c/$5).")
522         for sp in source_paths:
523             if not sp:
524                 continue
525             p = Path(sp)
526             for token in (sp, p.name, p.stem):
527                 t = (token or "").strip().lower()
528                 # Skip a token that is itself a substring of a schema-literal the writers
always emit
529                 # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not
a leak.
530                 if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
531                     continue
532                 if t in low:
533                     raise RefusalError(
534                         f"{where}: source-path token {token!r} found in written output ? the
source "
535                         f"filename/path must never be persisted ($4c metadata strip).")
536
537
538     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
539                               ) -> List[Tuple[int, int, float, float]]:
540         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
541
542         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
543         pure time shift); only the absolute frame index moves toward 0, severing the match

```

```

timeline.
544     """
545     rows: List[Tuple[int, int, float, float]] = []
546     for p in points:
547         rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
548     rows.sort(key=lambda r: r[0])
549     return rows
550
551
552     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
553         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""
554         return [(s - t0_sec, e - t0_sec) for s, e in gts]
555
556
557     def freeze_real_fixture(
558         label: str,
559         trajectory_csv_path: Any,
560         labels_csv_path: Any,
561         *,
562         license: str,
563         minors_free: bool,
564         owned: bool,
565         fps: float,
566         frame_width: float,
567         fixtures_dir: Path = FIXTURES_DIR,
568         slug: Optional[str] = None,
569         time_origin_reset: bool = True,
570         source_note: str = "",
571     ) -> Dict[str, Any]:
572         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
573         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
row P1.)
574
575         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
576         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string.
577         The committed bytes carry NO filename / video_id / match / player / capture-date ?
only an
578         opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
579         reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
580         re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
attributable
581         slipped in.
582
583         NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
584         only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
585         attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
586         owner-asserted Fork-A / minors / biometric-exclusion flags.
587
588         Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
589         """
590         # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
591         reasons: List[str] = []
592         if minors_free is not True:

```

```

593         reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
594                         "of a minor is prohibited ? §3, guardrail 'exclude minors')")
595     if owned is not True:
596         reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
597                         "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
598     if not (isinstance(license, str) and license.strip()):
599         reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
600                         "'provenance hard-fail gate')")
601     if reasons:
602         raise RefusalError(
603             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
604             "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n - "
605             + "\n - ".join(reasons))
606
607     traj_path = Path(str(trajjectory_csv_path))
608     labels_path = Path(str(labels_csv_path))
609
610     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
611     out_slug = slug if slug is not None else _new_surrogate_slug()
612
613     # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
-----
614     points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
615     if not points:
616         raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
617     gts = load_gt_intervals(str(labels_path), strict=True)
618
619     # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c)
-----
620     # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
621     # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
622     # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
623     # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
624     if time_origin_reset:
625         t0_frame = min(int(p.frame) for p in points)
626         t0_sec = t0_frame / float(fps)
627     else:
628         t0_frame, t0_sec = 0, 0.0
629     rows = _reset_trajectory_rows(points, t0_frame)
630     reset_gts = _reset_labels(gts, t0_sec)
631
632     # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
-----
633     fdir = fixtures_dir / out_slug
634     fdir.mkdir(parents=True, exist_ok=True)
635     write_trajectory_csv(rows, fdir / "trajectory.csv")
636     write_labels_csv(reset_gts, fdir / "labels.csv")
637
638     iou = 0.5
639     prov = {
640         "slug": out_slug,
641         "schema_version": CURRENT_SCHEMA,
642         "kind": "cleared_real",
643         "label": str(label),

```

```

644         "license": license,
645         "minors_free": True,
646         "owned": True,
647         "fps": _round(fps),
648         "frame_width": _round(frame_width),
649         "iou": _round(iou),
650         "time_origin_reset": bool(time_origin_reset),
651         "windowing": {
652             "vt": PINNED_VT,
653             "mg": PINNED_MERGE_GAP,
654             "mwd": PINNED_MIN_WINDOW,
655             "max_win": PINNED_MAX_WINDOW,
656         },
657         "paths": {
658             "trajectory": "trajectory.csv",
659             "labels": "labels.csv",
660             "expected": "expected.json",
661         },
662         "gt_hash": gt_hash(reset_gts),
663         "source_note": str(source_note),
664         "note": (
665             "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
666             "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
667             "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
668             "Tier-0 / CONTROL-only. S6: behaviour-locking, not correctness; sits behind
counsel (P2).",
669         ),
670     }
671     _write_json(fdir / "provenance.json", prov)
672
673     expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction
snapshot
674     _write_json(fdir / "expected.json", expected)
675
676     # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, S4c / ?)
-----
677     source_paths = [str(traj_path), str(labels_path)]
678     _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
679                     where=f"{out_slug}/expected.json", source_paths=source_paths)
680     _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
681                     where=f"{out_slug}/provenance.json", source_paths=source_paths)
682
683     # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
684     _append_manifest_entry(prov, fixtures_dir)
685     return expected
686
687
688 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
689     """A manifest row for a single fixture, derived from its provenance block."""
690     slug = prov["slug"]
691     return {
692         "slug": slug,
693         "fps": _round(float(prov["fps"])),
694         "frame_width": _round(float(prov["frame_width"])),
695         "windowing": dict(prov.get("windowing", {})),
696         "kind": prov.get("kind", "synthetic"),
697         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
698         "paths": {
699             "dir": slug,
700             "trajectory": f"{slug}/trajectory.csv",
701             "labels": f"{slug}/labels.csv",

```

```

702         "expected": f"{slug}/expected.json",
703         "provenance": f"{slug}/provenance.json",
704     },
705 }
706
707
708 def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
709     """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
710     entries = load_manifest(fixtures_dir)
711     entries = [e for e in entries if e.get("slug") != prov["slug"]]
712     entries.append(_manifest_entry_for(prov))
713     entries.sort(key=lambda e: str(e.get("slug")))
714     _write_json(fixtures_dir / "manifest.json", entries)
715
716
717 # =====
718 # Manifest + loaders
719 # =====
720
721
722 def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
723     entries = [{
724         "slug": spec.slug,
725         "fps": _round(spec.fps),
726         "frame_width": _round(spec.frame_width),
727         "windowing": {
728             "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
729             "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
730         },
731         "kind": "synthetic",
732         "schema_version": CURRENT_SCHEMA,
733         "paths": {
734             "dir": spec.slug,
735             "trajectory": f"{spec.slug}/trajectory.csv",
736             "labels": f"{spec.slug}/labels.csv",
737             "expected": f"{spec.slug}/expected.json",
738             "provenance": f"{spec.slug}/provenance.json",
739         },
740     } for spec in specs]
741     _write_json(fixtures_dir / "manifest.json", entries)
742
743
744 def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
745     """Load the fixtures manifest (returns [] if absent)."""
746     path = fixtures_dir / "manifest.json"
747     if not path.is_file():
748         return []
749     data = _read_json(path)
750     if not isinstance(data, list): # pragma: no cover - corrupt manifest
751         raise ValueError(f"{path}: manifest must be a JSON array")
752     return data
753
754
755 def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
756     """Load one fixture's committed artifacts: `{provenance, expected, dir}`. Refuses
a fixture
757     whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
758     fdir = fixtures_dir / slug
759     prov = _read_json(fdir / "provenance.json")
760     expected = _read_json(fdir / "expected.json")
761     sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
762     if sv > CURRENT_SCHEMA:
763         raise ValueError(
764             f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade

```

```

the harness")
765     return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
766
767
768     # =====
769     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
770     # =====
771
772
773     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
774         """Return a list of human-readable mismatch messages (empty == match).
775
776         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
777         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
778         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
779         safe (the repo runs with core.autocrlf=true)."""
780         out: List[str] = []
781
782         def exact(path: str, a: Any, b: Any) -> None:
783             if a != b:
784                 out.append(f"{path}: {a!r} != {b!r} (exact)")
785
786         def approx(path: str, a: Any, b: Any) -> None:
787             try:
788                 if abs(float(a) - float(b)) > _FLOAT_TOL:
789                     out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
790             except (TypeError, ValueError):
791                 out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
792
793         exact("slug", recomputed.get("slug"), committed.get("slug"))
794         exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
795         exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
796         approx("iou", recomputed.get("iou"), committed.get("iou"))
797
798         rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
799         if len(rc) != len(cc):
800             out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
801         else:
802             for i, (r, c) in enumerate(zip(rc, cc)):
803                 approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
804                 approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
805                 rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
806                 if set(rs) != set(cs):
807                     out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
808             else:
809                 for k in rs:
810                     if k == "net_crossings":
811                         exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
812                     else:
813                         approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
814
815         rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
816         if len(rseg) != len(cseg):
817             out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
818         else:
819             for i, (r, c) in enumerate(zip(rseg, cseg)):
820                 approx(f"control_segments[{i}][0]", r[0], c[0])
821                 approx(f"control_segments[{i}][1]", r[1], c[1])

```



```

822
823     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
824     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
825 "false_negatives")
826     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
827 "mean_start_error", "mean_end_error")
828     for k in int_fields:
829         exact(f"score.{k}", rsc.get(k), csc.get(k))
830     for k in float_fields:
831         approx(f"score.{k}", rsc.get(k), csc.get(k))
832
833     return out
834
835 # =====
836 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
837 # =====
838
839
840 def _write_json(path: Path, obj: Any) -> None:
841     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
842     path.write_text(text + "\n", encoding="utf-8", newline="\n")
843
844
845 def _read_json(path: Path) -> Any:
846     with open(path, encoding="utf-8") as f:
847         return json.load(f)
848
849
850 def _resolve_dir(slug_or_dir: Any) -> Path:
851     if isinstance(slug_or_dir, Path):
852         return slug_or_dir
853     p = Path(str(slug_or_dir))
854     if p.is_dir():
855         return p
856     return FIXTURES_DIR / str(slug_or_dir)
857
858
859 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
860     if prov and "fps" in prov and "frame_width" in prov:
861         return float(prov["fps"]), float(prov["frame_width"])
862     spec = _spec_by_slug().get(slug)
863     if spec is not None:
864         return spec.fps, spec.frame_width
865     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
866
867
868 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
869     prov_path = fdir / "provenance.json"
870     prov = _read_json(prov_path) if prov_path.is_file() else None
871     return _fps_fw(prov, slug)
872
873
874 # =====
875 # CLI
876 # =====
877
878
879 def _cmd_check(fixtures_dir: Path) -> int:
880     manifest = load_manifest(fixtures_dir)
881     if not manifest:
882         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
883 file=sys.stderr)
884     return 1

```

```

884     failures = 0
885     for entry in manifest:
886         slug = entry["slug"]
887         committed = _read_json(fixtures_dir / slug / "expected.json")
888         recomputed = replay_fixture(fixtures_dir / slug)
889         mismatches = compare_expected(recomputed, committed)
890         if mismatches:
891             failures += 1
892             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
893                   f"(characterization = behaviour-locking, NOT correctness ? $6):")
894             for m in mismatches:
895                 print(f"    - {m}")
896         else:
897             sc = recomputed["score"]
898             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
899                   f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
900     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
901 "
902         f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
903     return 0 if failures == 0 else 1
904
905     def main(argv: Optional[Sequence[str]] = None) -> int:
906         ap = argparse.ArgumentParser(
907             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
908 harness.")
909         g = ap.add_mutually_exclusive_group(required=True)
910         g.add_argument("--freeze-synthetic", action="store_true",
911                        help="generate synthetic trajectory+labels for all built-in
912 scenarios, then "
913                            "freeze expected.json+provenance.json + write manifest.json")
914         g.add_argument("--refresh-snapshot", action="store_true",
915                        help="re-freeze expected.json from COMMITTED trajectory+labels (no
916 regeneration)")
917         g.add_argument("--check", action="store_true",
918                        help="replay all fixtures + parse-compare vs committed expected.json
919 (exit 0/1)")
920         g.add_argument("--freeze-real", action="store_true",
921                        help="(P1) de-attribute an OWNED real trajectory+labels into a
922 committable Tier-0 "
923                            "fixture; REFUSED without --owned --minors-free and a non-blank
924 --license")
925         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
926 slug for "
927                            "--freeze-real (default: a random fx_<hex> id)")
928         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
929 snapshot")
930         ap.add_argument("--reason", help="recorded reason for --refresh-snapshot (review the
931 diff!)")
932         # --freeze-real (P1 / M2 refusal gate) options:
933         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
934 (Frame,Visibility,X,Y)")
935         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
936 (start,end)")
937         ap.add_argument("--license", dest="license_", default=None,
938                        help="--freeze-real) the source license tag (required, non-blank)")
939         ap.add_argument("--owned", action="store_true",
940                        help="--freeze-real) assert owner-recorded Fork-A source
941 (required)")
942         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
943                        help="--freeze-real) assert the source contains NO minors
944 (required)")
945         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
946         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
947 real) source frame width (px)")

```

```

934         ap.add_argument("--label", default="cleared_real", help="--freeze-real) non-
identifying scenario label")
935         ap.add_argument("--source-note", dest="source_note", default="",
936                         help="--freeze-real) non-identifying note (NEVER a path/name/id)")
937         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
938                         help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
939         ap.set_defaults(time_origin_reset=True)
940         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
941                         help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
942         args = ap.parse_args(list(argv) if argv is not None else None)
943
944         fixtures_dir: Path = args.fixtures_dir
945
946         if args.freeze_synthetic:
947             slugs = freeze_synthetic(fixtures_dir)
948             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
949             print(f" -> {fixtures_dir}")
950             return 0
951
952         if args.refresh_snapshot:
953             if not args.slug and not args.all:
954                 ap.error("--refresh-snapshot requires --slug S or --all")
955             if not args.reason:
956                 print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can
"
957                     "rubber-stamp a real regression ? review the visible diff ($6).",
file=sys.stderr)
958             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
959             print(f"[golden-fixtures] refreshed snapshot for: {'', '.join(targets)}"
+ (f" (reason: {args.reason})" if args.reason else ""))
960             return 0
961
962         if args.freeze_real:
963             missing = [n for n, v in (("--trajectory", args.trajectory),
964                                     ("--labels", args.labels),
965                                     ("--fps", args.fps),
966                                     ("--frame-width", args.frame_width)) if v is None]
967
968             if missing:
969                 ap.error("--freeze-real requires " + ", ".join(missing))
970             try:
971                 expected = freeze_real_fixture(
972                     args.label, args.trajectory, args.labels,
973                     license=args.license_ if args.license_ is not None else "",
974                     minors_free=args.minors_free, owned=args.owned,
975                     fps=args.fps, frame_width=args.frame_width,
976                     fixtures_dir=fixtures_dir, slug=args.slug,
977                     time_origin_reset=args.time_origin_reset, source_note=args.source_note)
978             except RefusalError as e:
979                 # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
zero.
980                 print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
981                 return 2
982             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'"
"
983                 f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
984             print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
985                 "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")

```

```

986         return 0
987
988     # --check
989     return _cmd_check(fixtures_dir)
990
991
992 if __name__ == "__main__":
993     raise SystemExit(main())
994

```

8. assistant (2026-06-16T06:41:29.234Z)

Now let me read the test files.

9. user (2026-06-16T06:41:30.214Z)

```

1      """Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic
Tier-0.
2
3      Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate
is
4      **parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the
replay
5      path), and asserts the Tier-0 privacy invariant positively (no
person/audio/pose/gait/face keys).
6
7      $6 honest-limits caveat: green here proves only that the deterministic post-trajectory
transforms
8      are unchanged for the frozen synthetic cases ? characterization is behaviour-locking,
NOT
9      correctness, and synthetic-tier numbers measure plumbing, not field quality.
10     """
11     import json
12
13     import pytest
14
15     from backend.eval import golden_fixtures as gf
16
17     # Slugs discovered from the committed manifest ? the suite grows by accretion
automatically.
18     MANIFEST = gf.load_manifest()
19     SLUGS = [entry["slug"] for entry in MANIFEST]
20
21
22     def test_manifest_is_non_empty():
23         """A committed manifest with at least one fixture must exist (else nothing is
gated)."""
24         assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-
synthetic"
25
26
27     @pytest.mark.parametrize("slug", SLUGS)
28     def test_replay_matches_committed_expected(slug):
29         """Replay the fixture and parse-compare against its committed expected.json.
30
31         Characterization = behaviour-locking, NOT correctness ($6): a non-empty mismatch
list means
32         the deterministic post-trajectory behaviour CHANGED for this frozen case, not that
it is wrong.
33         """
34         committed = gf.load_fixture(slug)["expected"]
35         recomputed = gf.replay_fixture(slug)
36         mismatches = gf.compare_expected(recomputed, committed)
37         assert mismatches == [], (
38             f"{slug}: behaviour changed vs frozen snapshot (characterization, not

```

```

correctness ? $6):\n"
39         + "\n".join(f" - {m}" for m in mismatches)
40     )
41
42
43     @pytest.mark.parametrize("slug", SLUGS)
44     def test_schema_version_le_current(slug):
45         """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-
compatible)."""
46         fx = gf.load_fixture(slug)
47         assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
48         assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA
49
50
51     @pytest.mark.parametrize("slug", SLUGS)
52     def test_no_person_or_audio_keys(slug):
53         """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key
appears
54         anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
55         fdir = gf.FIXTURES_DIR / slug
56         raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
57         for forbidden in gf.FORBIDDEN_KEYS:
58             assert forbidden not in raw, (
59                 f"{slug}: forbidden key substring {forbidden!r} present in expected.json
(Tier-0 leak)")
60
61         expected = gf.load_fixture(slug)["expected"]
62         for i, cand in enumerate(expected["candidates"]):
63             # The only feature block is the shuttle block, and it holds EXACTLY the 5
shuttle cols.
64             assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}]
has extra keys"
65             assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
66                 f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")
67
68
69     @pytest.mark.parametrize("slug", SLUGS)
70     def test_synthetic_provenance_tags(slug):
71         """Tier-0 hard tags: kind=synthetic, license=synthetic, minors_free=true (constraint
5)."""
72         prov = gf.load_fixture(slug)["provenance"]
73         assert prov["kind"] == "synthetic"
74         assert prov["license"] == "synthetic"
75         assert prov["minors_free"] is True
76         # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
77         assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}
78
79
80     def test_control_is_not_learned():
81         """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression
? D5)."""
82         from backend.eval.experiment import CONTROL
83         assert CONTROL.is_learned() is False
84
85
86     def test_perturbing_trajectory_changes_segments(tmp_path):
87         """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving
the
88         windows are RE-DERIVED from the CSV each run, not merely echoed from
expected.json."""
89         slug = SLUGS[0]
90         src = gf.FIXTURES_DIR / slug
91         work = tmp_path / slug
92         work.mkdir()
93         # Copy the fixture, then TRUNCATE the trajectory to its first half so the derived

```

```

window's
94         # `end` shrinks ? a guaranteed structural change in the re-derived segments.
(Deleting an
95         # interior hole would not move start/end, since merge_gap coalesces across it and
the frame
96         # indices of the surviving rows are unchanged.)
97         lines = (src / "trajectory.csv").read_text(encoding="utf-8").splitlines()
98         header, body = lines[0], lines[1:]
99         mutated = [header] + body[: len(body) // 2]
100        (work / "trajectory.csv").write_text("\n".join(mutated) + "\n", encoding="utf-8",
newline="\n")
101        (work / "labels.csv").write_text((src / "labels.csv").read_text(encoding="utf-8"),
102                                         encoding="utf-8", newline="\n")
103        (work / "provenance.json").write_text((src /
"provenance.json").read_text(encoding="utf-8"),
104                                              encoding="utf-8", newline="\n")
105
106        original = gf.load_fixture(slug)["expected"]
107        perturbed = gf.replay_fixture(work)
108        assert perturbed["control_segments"] != original["control_segments"], (
109            "perturbed trajectory produced identical segments ? replay is echoing, not re-
deriving")
110
111
112    def test_freeze_is_idempotent(tmp_path):
113        """Re-freezing every built-in fixture into a temp dir produces expected.json
structurally
114        equal to the committed one (correct-by-construction + deterministic)."""
115        slugs = gf.freeze_synthetic(fixtures_dir=tmp_path)
116        assert slugs, "freeze_synthetic produced no fixtures"
117        for slug in slugs:
118            committed = gf.load_fixture(slug)["expected"]
119            refrozen = json.loads((tmp_path / slug /
"expected.json").read_text(encoding="utf-8"))
120            # Structural (parsed) equality ? no float churn, exact same shape/values.
121            assert gf.compare_expected(refrozen, committed) == [], (
122                f"{slug}: re-frozen expected.json differs from committed (non-idempotent
freeze)")
123            assert refrozen == committed, f"{slug}: re-frozen expected.json not structurally
identical"
124
125
126    def test_check_cli_passes(capsys):
127        """The --check CLI replays all fixtures and exits 0 when they match the committed
snapshots."""
128        rc = gf.main(["--check"])
129        assert rc == 0
130

```

10. user (2026-06-16T06:41:30.903Z)

```

1        """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3        These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4        minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5        its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7        Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8        We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
and the
9        ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"

```

```

CSVs, then
10     prove:
11         * the refusal gate blocks every unsafe combination (M2);
12         * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13         * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14         ``control_segments`` shift toward 0);
15         * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17     $6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18     are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
19     """
20     import pytest
21
22     from backend.eval import golden_fixtures as gf
23
24
25     # -----
26     # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27     # -----
28
29
30     def _write_source(tmp_path, *, frame_offset: int = 0):
31         """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33         Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34         ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35         capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36         """
37         spec = gf.builtin_specs()[0] # clean_single_rally
38         fps, fw = spec.fps, spec.frame_width
39         rows = gf.make_synthetic_trajectory(spec)
40         if frame_offset:
41             rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
42         gts = list(spec.gts)
43         if frame_offset:
44             off_sec = frame_offset / fps
45             gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
46
47         traj = tmp_path / "src_trajectory.csv"
48         labels = tmp_path / "src_labels.csv"
49         gf.write_trajectory_csv(rows, traj)
50         gf.write_labels_csv(gts, labels)
51         return traj, labels, fps, fw, gts
52
53
54     # -----
55     # M2 ? refusal gate
56     # -----
57
58
59     @pytest.mark.parametrize(
60         "kwargs, why",
61         [
62             ({"minors_free": False, "owned": True, "license": "owned"},
"minors_free=False"),
63             ({"minors_free": True, "owned": False, "license": "owned"}, "owned=False"),
64             ({"minors_free": True, "owned": True, "license": ""}, "license=' '"),
65             ({"minors_free": True, "owned": True, "license": "   "}, "license=blank"),
66             ({"minors_free": True, "owned": True, "license": None}, "license=None"),
67         ],

```

```

68     )
69     def test_refusal_gate_blocks_unsafe(tmp_path, kwargs, why):
70         """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
blank license.
71
72         The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
fixture path
73         must be impossible to reach silently.
74         """
75         traj, labels, fps, fw, _ = _write_source(tmp_path)
76         with pytest.raises(ValueError) as ei:
77             gf.freeze_real_fixture(
78                 "scenario", traj, labels,
79                 fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
80         # RefusalError is a ValueError subclass; the message must name a reason.
81         assert isinstance(ei.value, gf.RefusalError), why
82         assert "REFUSED" in str(ei.value), why
83         # Nothing was written for a refused freeze.
84         assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
"fixtures").iterdir()), (
85             f"{why}: a refused freeze must not write any fixture")
86
87
88     # -----
89     # P1 ? de-attribution
90     # -----
91
92
93     def test_real_fixture_is_deattributed(tmp_path):
94         """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
95         no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
96         fixtures = tmp_path / "fixtures"
97         traj, labels, fps, fw, _ = _write_source(tmp_path)
98         expected = gf.freeze_real_fixture(
99             "clean_rally", traj, labels,
100             license="owned", minors_free=True, owned=True,
101             fps=fps, frame_width=fw, fixtures_dir=fixtures)
102
103         slug = expected["slug"]
104         assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"
105
106         prov = gf._read_json(fixtures / slug / "provenance.json")
107         assert prov["kind"] == "cleared_real"
108         assert prov["minors_free"] is True and prov["owned"] is True
109         assert prov["license"] == "owned"
110         assert prov["slug"] == slug
111         # No attribution keys / no source filename anywhere in the provenance.
112         for banned in ("video_id", "name", "filename", "path", "match", "capture_date"):
113             assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
114         prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
115         for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
116             assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
117
118         # expected.json carries no biometric/identity/non-shuttle stream.
119         exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
120         for forbidden in gf.FORBIDDEN_KEYS:
121             assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
122
123         # The fixture is registered in the manifest with kind=cleared_real.
124         manifest = gf.load_manifest(fixtures)

```



```

125     rows = [e for e in manifest if e["slug"] == slug]
126     assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
127
128
129     def test_supplied_slug_is_honored(tmp_path):
130         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
131         concern)."""
132         fixtures = tmp_path / "fixtures"
133         traj, labels, fps, fw, _ = _write_source(tmp_path)
134         expected = gf.freeze_real_fixture(
135             "scenario", traj, labels, slug="fx_custom00",
136             license="owned", minors_free=True, owned=True,
137             fps=fps, frame_width=fw, fixtures_dir=fixtures)
138         assert expected["slug"] == "fx_custom00"
139         assert (fixtures / "fx_custom00" / "expected.json").is_file()
140
141     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
142         """Freezing a second real fixture must APPEND to the manifest, not wipe the
143         first."""
144         fixtures = tmp_path / "fixtures"
145         traj, labels, fps, fw, _ = _write_source(tmp_path)
146         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
147             license="owned", minors_free=True, owned=True,
148             fps=fps, frame_width=fw, fixtures_dir=fixtures)
149         e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
150             license="owned", minors_free=True, owned=True,
151             fps=fps, frame_width=fw, fixtures_dir=fixtures)
152         slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
153         assert {e1["slug"], e2["slug"]} <= slugs
154
155     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
156         """source_note is non-identifying by contract ? a path/name in it trips the positive
157         scan."""
158         fixtures = tmp_path / "fixtures"
159         traj, labels, fps, fw, _ = _write_source(tmp_path)
160         with pytest.raises(gf.RefusalError):
161             gf.freeze_real_fixture(
162                 "scenario", traj, labels, slug="fx_leak0001",
163                 license="owned", minors_free=True, owned=True,
164                 fps=fps, frame_width=fw, fixtures_dir=fixtures,
165                 source_note=str(traj)) # leaking the source path ? must be refused
166
167     # -----
168     # P1 (d) ? time-origin reset is metric-invariant
169     # -----
170
171     def test_time_origin_reset_is_metric_invariant(tmp_path):
172         """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source
173         frozen
174         WITHOUT reset ? but its absolute control_segments shift toward 0 (severed
175         timeline)."""
176         offset_frames = 9000 # +5 minutes at 30fps ? a large absolute capture-timeline
177         offset
178         traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
179
180         with_reset = gf.freeze_real_fixture(
181             "scenario", traj, labels, slug="fx_reset_on",
182             license="owned", minors_free=True, owned=True,
183             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
184             time_origin_reset=True)
185         without_reset = gf.freeze_real_fixture(

```

```

184         "scenario", traj, labels, slug="fx_reset_off",
185         license="owned", minors_free=True, owned=True,
186         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
187         time_origin_reset=False)
188
189     # (1) Score is bit-identical ? metrics.score is built from pure differences.
190     assert with_reset["score"] == without_reset["score"], (
191         "time-origin reset must be metric-invariant (score built from pure
differences)")
192
193     # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
194     seg_on = with_reset["control_segments"]
195     seg_off = without_reset["control_segments"]
196     assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
197     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
198     # The shift equals the offset in seconds (within rounding).
199     shift = seg_off[0][0] - seg_on[0][0]
200     assert abs(shift - offset_frames / fps) < 1e-3, (
201         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
202     # Reset trajectory starts at frame 0.
203     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
204                       .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
205     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
206
207
208     # -----
209     # P1 (e/f) ? round-trip / correct-by-construction
210     # -----
211
212
213     def test_freeze_real_round_trips_check(tmp_path):
214         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
215         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
216         fixtures = tmp_path / "fixtures"
217         traj, labels, fps, fw, _ = _write_source(tmp_path)
218         expected = gf.freeze_real_fixture(
219             "scenario", traj, labels, slug="fx_rt000001",
220             license="owned", minors_free=True, owned=True,
221             fps=fps, frame_width=fw, fixtures_dir=fixtures)
222
223         fdir = fixtures / expected["slug"]
224         committed = gf._read_json(fdir / "expected.json")
225         recomputed = gf.replay_fixture(fdir)
226         assert gf.compare_expected(recomputed, committed) == [], (
227             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
228
229
230     def test_freeze_real_uses_strict_label_loader(tmp_path):
231         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
232         never silently shrinks the GT set."""
233         fixtures = tmp_path / "fixtures"
234         traj, labels, fps, fw, _ = _write_source(tmp_path)
235         # Corrupt one label row (end <= start) ? strict loader must raise.
236         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
237         with pytest.raises(ValueError):
238             gf.freeze_real_fixture(
239                 "scenario", traj, labels, slug="fx_strict01",
240                 license="owned", minors_free=True, owned=True,
241                 fps=fps, frame_width=fw, fixtures_dir=fixtures)

```

11. assistant (2026-06-16T06:41:37.123Z)

Now I have a thorough understanding. Let me look at the current committed synthetic fixtures' provenance to confirm they have no `source\_note` (synthetic provenance doesn't include it ? good), and check the CI file and the eval_baselines README. Let me also check the relevant supporting code (`gt\_loader`, `distill\_local.build\_candidates`, `windowing`).

12. user (2026-06-16T06:41:38.559Z)

```

1      name: CI
2
3      on:
4        push:
5          branches: [master]
6        pull_request:
7
8      permissions:
9        contents: read
10
11     concurrency:
12       group: ci-${{ github.ref }}
13       cancel-in-progress: true
14
15     jobs:
16       # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17       # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18       # tracked .py, including scripts/ and training/, which the backend-only
19       # py_compile smoke sweep does not reach.
20       lint:
21         name: Ruff lint
22         runs-on: ubuntu-latest
23         steps:
24           - uses: actions/checkout@v6
25           - uses: actions/setup-python@v6
26             with:
27               python-version: "3.13"
28           - name: Install ruff
29             run: python -m pip install ruff
30           - name: Ruff check
31             run: ruff check . --output-format github
32
33       # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34       # per-module quarantine list ? ratchet by removing entries as modules clean
35       # up. Typed core deps are installed so mypy sees their real signatures
36       # (without them ignore_missing_imports would Any-stub half the checks).
37       typecheck:
38         name: Mypy (lenient baseline)
39         runs-on: ubuntu-latest
40         steps:
41           - uses: actions/checkout@v6
42           - uses: actions/setup-python@v6
43             with:
44               python-version: "3.13"
45           - name: Install mypy + typed core deps
46             run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
47             genai numpy
48           - name: Mypy
49             run: mypy backend training
50
51       # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
52       # module-level import error shipping while the suite stays green.
53       # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
54       # on machines without the GPU stack (test_import_smoke stubs what's absent).
```

```

54 import-smoke:
55     name: Syntax sweep + import smoke (no GPU stack)
56     runs-on: ubuntu-latest
57     steps:
58     - uses: actions/checkout@v6
59     - uses: actions/setup-python@v6
60       with:
61         python-version: "3.13"
62     - name: Install minimal deps
63       run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64     - name: Run import smoke tests
65       run: python -m pytest tests/test_import_smoke.py -v
66
67 full-suite:
68     name: Full test suite (offline, mocked)
69     runs-on: ubuntu-latest
70     steps:
71     - uses: actions/checkout@v6
72     - uses: actions/setup-python@v6
73       with:
74         python-version: "3.13"
75         cache: pip
76     - name: System libs for opencv-python
77       run: |
78         sudo apt-get update
79         sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80     - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81       # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82       # need an out-of-band index that PEP 621 can't express. Keep installing
83       # them separately here, before the editable install pulls the rest.
84       run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85     - name: Install package (editable) + dev tooling
86       # Editable PEP 621 install: runtime deps + the `dev` group
87       # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88       run: python -m pip install -e .[dev]
89       # obsreport (private, soft dep) is absent here: reporting tests skip via
90       # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91       # is therefore calibrated against THIS lane's number (local runs with
92       # obsreport installed measure a little higher).
93       # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94       # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95       # without an owner decision.
96     - name: Run full suite (with coverage floor)
97       run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99     # Cross-OS determinism guard for the committed golden regression fixtures
100     # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101     # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102     # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103     # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104     # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105     golden-crossos:
106       name: Golden fixtures cross-OS (${ matrix.os })
107       strategy:
108         fail-fast: false
109         matrix:
110           os: [ubuntu-latest, windows-latest, macos-latest]
111       runs-on: ${ matrix.os }
112       steps:
113       - uses: actions/checkout@v6
114       - uses: actions/setup-python@v6

```

```

115         with:
116             python-version: "3.13"
117         - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
119         - name: Golden-fixtures characterization (cross-OS determinism guard)
120           run: |
121             python -m backend.eval.golden_fixtures --check
122             python -m pytest tests/test_golden_fixtures.py -q
123

```

13. user (2026-06-16T06:41:38.818Z)

No files found

14. assistant (2026-06-16T06:41:41.797Z)

The fixtures dir doesn't exist yet in the worktree (it's gitignored or not generated). Let me check the eval_baselines structure and find the README.

15. user (2026-06-16T06:41:42.438Z)

```

eval_baselines\heuristic_lovo_n6.json
eval_baselines\gemini_wrapper_2026-06-10.json
eval_baselines\experiment_leaderboard.json

```

16. user (2026-06-16T06:41:44.396Z)

```

total 28
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 .
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 ..
-rw-r--r-- 1 avidu 197609 4696 Jun 16 12:10 experiment_leaderboard.json
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 fixtures
-rw-r--r-- 1 avidu 197609 1977 Jun 16 12:10 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609  689 Jun 16 12:10 heuristic_lovo_n6.json
---fixtures---
total 28
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 .
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 ..
-rw-r--r-- 1 avidu 197609 4018 Jun 16 12:10 README.md
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 clean_single_rally
-rw-r--r-- 1 avidu 197609 1557 Jun 16 12:10 manifest.json
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 multi_rally_with_deadtime
drwxr-xr-x 1 avidu 197609  0 Jun 16 12:10 occlusion_break

```

17. assistant (2026-06-16T06:41:47.489Z)

The fixtures exist (Glob may not traverse the dir due to gitignore patterns). Let me read the committed fixtures' provenance/expected and the README. Let me also check the supporting modules.

18. user (2026-06-16T06:41:48.440Z)

```

1      # `eval_baselines/fixtures/` ? Golden Regression Fixtures (Tier-0, synthetic)
2
3      Video-free, **CONTROL-only** characterization fixtures for the rally-segmentation
pipeline.
4      A contributor on **any machine ? no video, GPU, WSL, or DB** ? replays these committed
5      structured files through the already-shipped eval stack and gets a red/green regression
6      signal. See `docs/GOLDEN_REGRESSION_FIXTURES.md` (deliverable **M1**; $4, $6, $7).
7
8      ## What's here
9
10     Each fixture is one directory:

```

```

11
12 | File | Purpose |
13 |-----|-----|
14 | `trajectory.csv` | Synthetic shuttle trajectory (`Frame,Visibility,X,Y`) ? the freeze
seam input. |
15 | `labels.csv` | Synthetic ground-truth rally intervals (`start,end`). |
16 | `expected.json` | Frozen replay snapshot (candidates + 5 shuttle features + CONTROL
segments + score). |
17 | `provenance.json` | Pinned windowing params + privacy/licensing tags + description. |
18
19 `manifest.json` lists every fixture; `backend/eval/golden_fixtures.py` is the shared
replay
20 module + freeze/refresh/check CLI.
21
22 ## Guardrails (non-negotiable)
23
24 - **Synthetic / owned only.** Every fixture is generated by deterministic formulae (NO
random)
25 ? zero third-party footage, zero provenance/licensing risk. Tagged `kind:
"synthetic",
26 `license: "synthetic", `minors_free: true`.
27 - **No biometric / identity streams.** Tier-0 carries the **5 shuttle feature columns
only**
28 (`duration, peak_velocity, mean_velocity, active_ratio, net_crossings`). There is, by
design,
29 **no person / pose / gait / face / audio / player field anywhere in the schema** ? a
test
30 asserts this positively (deletion-at-source, $4c). Minors and biometrics are excluded.
31 - **CONTROL / pure-stdlib only.** The replay uses `experiment.CONTROL` (`is_learned() ==
False`)
32 ? no numpy / logistic-regression. Learned variants' near-threshold decisions flip
across CPUs,
33 so they can **never** anchor a cross-machine snapshot ($4d / D5).
34 - **Parse-compare, never byte-compare.** The gate `json.load`'s both sides: exact
equality on
35 int/structural fields, abs-tolerance `1e-6` on floats (CRLF/float-format safe).
36 - **Windowing is pinned per fixture.** The `(vt, mg, mwd, max_win)` params live in each
37 `provenance.json` and are passed explicitly to `build_candidates` ? the replay does
**not**
38 depend on the named `windowing.SERVED` constant, so a future SERVED retune can't
silently
39 churn fixtures.
40
41 ## Fixtures are GENERATED, never hand-edited
42
43 Do **not** hand-edit `expected.json` / `provenance.json` / the CSVs. They are produced
44 correct-by-construction by the CLI.
45
46 ```bash
47 # Generate the synthetic trajectory.csv + labels.csv for all built-in scenarios, then
48 # freeze expected.json + provenance.json + (re)write manifest.json:
49 python -m backend.eval.golden_fixtures --freeze-synthetic
50
51 # Replay all fixtures and parse-compare vs the committed expected.json (exit 0/1):
52 python -m backend.eval.golden_fixtures --check
53
54 # Re-freeze expected.json from the COMMITTED trajectory+labels (NO regeneration). ? This
can
55 # rubber-stamp a real regression ? review the visible diff and record a reason ($6):
56 python -m backend.eval.golden_fixtures --refresh-snapshot --all --reason "<why>"
57 python -m backend.eval.golden_fixtures --refresh-snapshot --slug clean_single_rally
--reason "<why>"
58 ```
59
60 ## Honest limits ($6 ? rides in the test pass/fail messages, not only here)

```

```

61
62     - A green suite proves only that the **deterministic post-trajectory transforms** are
unchanged
63     for the frozen cases. It does **not** cover the WASB/TrackNet detector, frame
decoding, the
64     AI/provider handoff, chunking, the ffmpeg stitch, DB persistence, or the API filter.
65     - Characterization is **behaviour-locking, not correctness** ? it freezes current
behaviour
66     *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour
correct".
67     - **Synthetic-tier F1 measures plumbing correctness, NOT field quality.** Never cite a
synthetic
68     fixture's F1 in `QUALITY_ITERATIONS.md` or the paper as rally-detection performance.
69

```

19. user (2026-06-16T06:41:49.425Z)

```

1      {
2      "description": "One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
3      "fps": 30.0,
4      "frame_width": 1280.0,
5      "iou": 0.5,
6      "kind": "synthetic",
7      "license": "synthetic",
8      "minors_free": true,
9      "note": "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only.
Plumbing-correctness fixture ? NOT a measure of rally-detection quality (§6).",
10     "paths": {
11         "expected": "expected.json",
12         "labels": "labels.csv",
13         "trajectory": "trajectory.csv"
14     },
15     "schema_version": 1,
16     "slug": "clean_single_rally",
17     "windowing": {
18         "max_win": 0.0,
19         "mg": 2.0,
20         "mwd": 2.0,
21         "vt": 0.02
22     }
23 }
24

```

20. user (2026-06-16T06:41:49.931Z)

```

1      [
2      {
3      "fps": 30.0,
4      "frame_width": 1280.0,
5      "kind": "synthetic",
6      "paths": {
7          "dir": "clean_single_rally",
8          "expected": "clean_single_rally/expected.json",
9          "labels": "clean_single_rally/labels.csv",
10         "provenance": "clean_single_rally/provenance.json",
11         "trajectory": "clean_single_rally/trajectory.csv"
12     },
13     "schema_version": 1,
14     "slug": "clean_single_rally",
15     "windowing": {
16         "max_win": 0.0,
17         "mg": 2.0,
18         "mwd": 2.0,
19         "vt": 0.02

```

```

20     }
21 },
22 {
23     "fps": 30.0,
24     "frame_width": 1280.0,
25     "kind": "synthetic",
26     "paths": {
27         "dir": "multi_rally_with_deadtime",
28         "expected": "multi_rally_with_deadtime/expected.json",
29         "labels": "multi_rally_with_deadtime/labels.csv",
30         "provenance": "multi_rally_with_deadtime/provenance.json",
31         "trajectory": "multi_rally_with_deadtime/trajectory.csv"
32     },
33     "schema_version": 1,
34     "slug": "multi_rally_with_deadtime",
35     "windowing": {
36         "max_win": 0.0,
37         "mg": 2.0,
38         "mwd": 2.0,
39         "vt": 0.02
40     }
41 },
42 {
43     "fps": 30.0,
44     "frame_width": 1280.0,
45     "kind": "synthetic",
46     "paths": {
47         "dir": "occlusion_break",
48         "expected": "occlusion_break/expected.json",
49         "labels": "occlusion_break/labels.csv",
50         "provenance": "occlusion_break/provenance.json",
51         "trajectory": "occlusion_break/trajectory.csv"
52     },
53     "schema_version": 1,
54     "slug": "occlusion_break",
55     "windowing": {
56         "max_win": 0.0,
57         "mg": 2.0,
58         "mwd": 2.0,
59         "vt": 0.02
60     }
61 }
62 ]
63

```

21. assistant (2026-06-16T06:42:02.921Z)

The synthetic provenance has no `source\_note` key already (good ? fix #1's "no source_note in committed provenance" claim is about the real-fixture path, not synthetic). Note: synthetic provenance has no `label` key either ? so fix #1's slug validation only applies to `freeze\_real\_fixture`'s `label`.

Let me check the supporting modules I need for the fixes: `gt\_loader.load\_gt\_intervals`, `distill\_local.build\_candidates` signature, and confirm there's no `windowing` import in golden_fixtures (there isn't). Let me also verify the env (python availability).

22. user (2026-06-16T06:42:07.107Z)

```

Python 3.13.13
---
numpy 2.4.6
---ruff---
ruff 0.15.16
---mypy---
mypy 2.1.0 (compiled: yes)

```


---pytest---
pytest 9.0.3

23. user (2026-06-16T06:42:07.117Z)

```
1      """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3      The repo historically carried two incompatible loaders:
4      - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5        ``curate export`` format), raises on bad rows;
6      - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7        (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9      A file in one convention fed to the other loader yielded zero/garbage intervals with no
10     error. This module accepts BOTH conventions (header-mapped, else positional), so every
11     quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12     flows through one reader. Both legacy entry points now delegate here.
13
14     Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15     - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16       columns ? rally_number, ending_reason, sport, ? ? are ignored);
17     - no header: columns 0,1 positionally.
18     Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19     (``annotations.parse_timestamp``).
20     """
21     import csv
22     import logging
23     from typing import List, Optional, Tuple
24
25     from backend.eval.annotations import parse_timestamp
26
27     logger = logging.getLogger(__name__)
28
29     Interval = Tuple[float, float]
30
31     _START_NAMES = ("start", "start_time")
32     _END_NAMES = ("end", "end_time")
33
34     def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35         """If `row` is a header naming start/end columns, return their indices, else
36         None."""
37         cells = [(c or "").strip().lower() for c in row]
38         start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39         end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40         if start_idx is not None and end_idx is not None:
41             return start_idx, end_idx
42         return None
43
44     def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45         """Load GT rally intervals from either golden-CSV convention.
46
47         strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48         (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49         strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50         skipped ? a silently shrinking GT set flatters precision.
51
52         Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53         (they break one-to-one matching and usually indicate a labeling error).
54         """
55         intervals: List[Interval] = []
56         skipped = 0
57         cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
```

```

58     saw_header = False
59
60     with open(csv_path, newline="", encoding="utf-8-sig") as f:
61         for lineno, row in enumerate(csv.reader(f), start=1):
62             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                 continue
64             if cols is None and not saw_header:
65                 header = _header_columns(row)
66                 if header is not None:
67                     cols = header
68                     saw_header = True
69                     continue # consume the header row
70             cols = (0, 1) # headerless: positional start,end
71             assert cols is not None # every branch above assigns it
72             si, ei = cols
73             try:
74                 if len(row) <= max(si, ei):
75                     raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                 start = parse_timestamp(row[si])
77                 end = parse_timestamp(row[ei])
78                 if start < 0:
79                     raise ValueError(f"negative start ({start})")
80                 if start >= end:
81                     raise ValueError(f"start ({start}) >= end ({end})")
82             except ValueError as e:
83                 if strict:
84                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                 skipped += 1
86                 continue
87             intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

24. assistant (2026-06-16T06:42:12.293Z)

Tooling all present. Important detail for fix #3: `load\_gt\_intervals` with `strict=True` already raises on `start < 0`. But fix #3(a) is about the **reset** labels being negative after subtracting `t0\_sec` ? i.e., a label that starts before `t0\_frame/fps`. The source labels load fine (positive), but after reset a label could go negative. So I need to validate the **reset** labels.

Now let me confirm `distill\_local.build\_candidates` signature (to write the W1 comment correctly) and check there's no `max\_win` param.

25. user (2026-06-16T06:42:13.981Z)

```

52: def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
53-                      proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
54-     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature rows)."""
55-     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
56-     vt, mg, mwd = proposal
57-     wins = TrackNetRunner.trajectory_to_action_windows(

```

```

58-         points, fps=fps, frame_width=frame_width,
59-         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
60-     intervals = [(w["start"], w["end"]) for w in wins]
61-     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
62-     X: List[List[float]] = []
63-     for w, g in zip(wins, gated):
64-         dur = max(1e-6, w["end"] - w["start"])
65-         win_frames = max(1.0, dur * fps)
66-         X.append([
67-             dur,                                     # rally length (sec) ? fps-
invariant
68-             float(w["peak_velocity"]),               # already normalized by
frame_width
69-             float(w["mean_velocity"]),
70-             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ? fps-
invariant
71-             float(g.crossings),                     # net crossings (count) ? the
rally signal
72-         ])
73-     return intervals, X
74-
75-
76-def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) -> List[Interval]:
77-     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)

```

26. assistant (2026-06-16T06:42:20.033Z)

Confirmed: `build\_candidates` takes a 3-tuple `(vt, mg, mwd)` and `trajectory\_to\_action\_windows` has no `max\_window`. So fix #8 is correct: `max\_win` is recorded but not applied.

Now I have everything I need. Let me start a chapter and begin implementing. I'll work through the code fixes in `golden\_fixtures.py` first.

27. user (2026-06-16T06:42:20.901Z)

Chapter marked: "Golden fixtures audit hardening". Continue your current work.

28. assistant (2026-06-16T06:42:31.672Z)

Let me start with the code fixes. First, fix #1 (slug-charset `label` validation + drop `source\_note` from committed provenance). I'll add a label-slug regex constant and a validator.

29. user (2026-06-16T06:42:37.057Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

30. assistant (2026-06-16T06:42:44.239Z)

Now let me update the docstring at the top of the module to fix the over-claim (#9) and the windowing-map (#8 / W1). Let me update the module docstring lines about numpy and the windowing pinning.

31. user (2026-06-16T06:42:50.643Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

32. user (2026-06-16T06:42:59.173Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-

audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

33. assistant (2026-06-16T06:43:02.050Z)

Now fix the `Pure + offline + stdlib` line at the end of the docstring (#9 again ? over-claims "pure stdlib").

34. user (2026-06-16T06:43:06.079Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

35. assistant (2026-06-16T06:43:09.412Z)

Now fix the `PINNED\_MAX\_WINDOW` comment to note it's recorded-not-applied (#8).

36. user (2026-06-16T06:43:14.171Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

37. assistant (2026-06-16T06:43:16.892Z)

Now fix #8 docstring on `\_pinned\_proposal` ? already says "NEVER windowing.SERVED" but let me add the max_win-not-applied note.

38. user (2026-06-16T06:43:21.192Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. assistant (2026-06-16T06:43:28.776Z)

Now fix #4: `\_scan\_forbidden` source-path skip over-skips. Change `any(t in lit for lit in ...)` to exact membership `t in \_SCHEMA\_FILENAME\_LITERALS`.

40. user (2026-06-16T06:43:33.145Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

41. assistant (2026-06-16T06:43:36.123Z)

Now also update the comment on `\_SCHEMA\_FILENAME\_LITERALS` to reflect exact-membership semantics.

42. user (2026-06-16T06:43:39.928Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

43. assistant (2026-06-16T06:43:48.708Z)

Now the big one: fixes #1, #2, #3 in `freeze\_real\_fixture`. Let me rewrite that function carefully. I need to:

- #1a: validate `label` against `\_LABEL\_SLUG\_RE` ? RefusalError
- #1b: drop `source\_note` from written provenance (keep `--source-note` flag accepted-but-not-

```

persisted)
- #2: round `t0_sec` to 6dp before subtracting (apply exact 6dp shift to labels AND trajectory consistently)
- #3a: validate reset label starts >= 0 ? RefusalError
- #3b: write to temp dir, promote via os.replace only after scan passes

```

Let me look at the current function body again to plan the edit precisely. I'll rewrite from the refusal-gate section through the manifest append.

For #2, the subtlety: trajectory is shifted by integer `t0\_frame` (frame indices), and labels by `t0\_sec`. Currently `t0\_sec = t0\_frame/fps` at full precision. The replay re-derives candidate times from the shifted *frame* CSV (so candidate times are `(frame - t0\_frame)/fps`), and labels are written as `(s - t0\_sec)`. For metric-invariance the label shift in seconds must equal the trajectory's effective shift in seconds. The trajectory shift in seconds is `t0\_frame/fps` (integer frames / fps), but that's then re-quantized through CSV writing at 6dp and re-parsed. The key fix: round `t0\_sec` to 6dp BEFORE subtracting from labels, so the applied label shift is an exact 6dp quantity, matching what's serialized. Let me apply it.

44. user (2026-06-16T06:43:49.860Z)

```

557             # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
558             # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
559             # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
560             # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
561             if not t or t in _SCHEMA_FILENAME_LITERALS:
562                 continue
563             if t in low:
564                 raise RefusalError(
565                     f"{where}: source-path token {token!r} found in written output ? the
source "
566                     f"filename/path must never be persisted ($4c metadata strip).")
567
568
569     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
570                               ) -> List[Tuple[int, int, float, float]]:
571         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
572
573         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
574         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
575         """
576         rows: List[Tuple[int, int, float, float]] = []
577         for p in points:
578             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
579         rows.sort(key=lambda r: r[0])
580         return rows
581
582
583     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
584         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""
585         return [(s - t0_sec, e - t0_sec) for s, e in gts]
586
587
588     def freeze_real_fixture(
589         label: str,
590         trajectory_csv_path: Any,
591         labels_csv_path: Any,
592         *,

```

```

593     license: str,
594     minors_free: bool,
595     owned: bool,
596     fps: float,
597     frame_width: float,
598     fixtures_dir: Path = FIXTURES_DIR,
599     slug: Optional[str] = None,
600     time_origin_reset: bool = True,
601     source_note: str = "",
602 ) -> Dict[str, Any]:
603     """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
604     Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
row P1.)
605
606     The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
607     ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string.
608     The committed bytes carry NO filename / video_id / match / player / capture-date ?
only an
609     opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
610     reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
611     re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
attributable
612     slipped in.
613
614     NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
615     only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
616     attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
617     owner-asserted Fork-A / minors / biometric-exclusion flags.
618
619     Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
620     """
621     # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
622     reasons: List[str] = []
623     if minors_free is not True:
624         reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
625             "of a minor is prohibited ? $3, guardrail 'exclude minors')")
626     if owned is not True:
627         reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
628             "broadcast/scraped = Fork B, do NOT commit ? $3 D1)")
629     if not (isinstance(license, str) and license.strip()):
630         reasons.append("license must be a non-blank string (per-record provenance tag ?
$4c "
631             "'provenance hard-fail gate')")
632     if reasons:
633         raise RefusalError(
634             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
635             "cure provenance/licensing; this gate sits behind counsel sign-off,
$6/P2):\n - "
636             + "\n - ".join(reasons))
637
638     traj_path = Path(str(trajectory_csv_path))
639     labels_path = Path(str(labels_csv_path))

```

```

640
641         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
642         out_slug = slug if slug is not None else _new_surrogate_slug()
643
644         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
-----
645         points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
646         if not points:
647             raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
648         gts = load_gt_intervals(str(labels_path), strict=True)
649
650         # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c)
-----
651         # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
652         # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
653         # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
654         # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
655         if time_origin_reset:
656             t0_frame = min(int(p.frame) for p in points)
657             t0_sec = t0_frame / float(fps)
658         else:
659             t0_frame, t0_sec = 0, 0.0
660         rows = _reset_trajectory_rows(points, t0_frame)
661         reset_gts = _reset_labels(gts, t0_sec)
662
663         # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
-----
664         fdir = fixtures_dir / out_slug
665         fdir.mkdir(parents=True, exist_ok=True)
666         write_trajectory_csv(rows, fdir / "trajectory.csv")

```

45. assistant (2026-06-16T06:43:52.709Z)

Now let me update the `\_reset\_labels` docstring to reflect the 6dp-rounded shift, then rewrite the function body. First, `\_reset\_labels`:

46. user (2026-06-16T06:43:58.039Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

47. assistant (2026-06-16T06:44:00.662Z)

Now the main rewrite. Let me add the label validation to the refusal gate, and rewrite the reset/write/scan/promote logic. First, the refusal gate + label validation:

48. user (2026-06-16T06:44:23.774Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

49. assistant (2026-06-16T06:44:27.592Z)

Now the reset / write / scan / promote section. Let me read the current state of that block to edit it precisely.

50. user (2026-06-16T06:44:28.800Z)

```
657         "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
658         "cure provenance/licensing; this gate sits behind counsel sign-off,
$6/P2):\n - "
659         + "\n - ".join(reasons))
660
661     traj_path = Path(str(trajecory_csv_path))
662     labels_path = Path(str(labels_csv_path))
663
664     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
665     out_slug = slug if slug is not None else _new_surrogate_slug()
666
667     # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
668     points = TrackNetRunner.parse_trajecory_csv(str(traj_path))
669     if not points:
670         raise ValueError(f"{traj_path}: no parseable trajecory rows
(Frame,Visibility,X,Y)")
671     gts = load_gt_intervals(str(labels_path), strict=True)
672
673     # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, $4c)
-----
674     # Subtract the SAME offset from BOTH the trajecory frames and the label seconds:
because
675     # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
676     # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
677     # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
678     if time_origin_reset:
679         t0_frame = min(int(p.frame) for p in points)
680         t0_sec = t0_frame / float(fps)
681     else:
682         t0_frame, t0_sec = 0, 0.0
683     rows = _reset_trajecory_rows(points, t0_frame)
684     reset_gts = _reset_labels(gts, t0_sec)
685
686     # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
-----
687     fdir = fixtures_dir / out_slug
688     fdir.mkdir(parents=True, exist_ok=True)
689     write_trajecory_csv(rows, fdir / "trajecory.csv")
690     write_labels_csv(reset_gts, fdir / "labels.csv")
691
692     iou = 0.5
693     prov = {
694         "slug": out_slug,
695         "schema_version": CURRENT_SCHEMA,
696         "kind": "cleared_real",
697         "label": str(label),
698         "license": license,
699         "minors_free": True,
700         "owned": True,
701         "fps": _round(fps),
702         "frame_width": _round(frame_width),
703         "iou": _round(iou),
704         "time_origin_reset": bool(time_origin_reset),
705         "windowing": {
706             "vt": PINNED_VT,
707             "mg": PINNED_MERGE_GAP,
708             "mwd": PINNED_MIN_WINDOW,
```



```

709         "max_win": PINNED_MAX_WINDOW,
710     },
711     "paths": {
712         "trajectory": "trajectory.csv",
713         "labels": "labels.csv",
714         "expected": "expected.json",
715     },
716     "gt_hash": gt_hash(reset_gts),
717     "source_note": str(source_note),
718     "note": (
719         "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
720         "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
721         "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
722         "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2).",
723     ),
724 }
725 _write_json(fdir / "provenance.json", prov)
726
727 expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction
snapshot
728 _write_json(fdir / "expected.json", expected)
729
730 # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, §4c / ?)
-----
731 source_paths = [str(traj_path), str(labels_path)]
732 _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
733                 where=f"{out_slug}/expected.json", source_paths=source_paths)
734 _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
735                 where=f"{out_slug}/provenance.json", source_paths=source_paths)
736
737 # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
738 _append_manifest_entry(prov, fixtures_dir)
739 return expected
740
741
742 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
743     """A manifest row for a single fixture, derived from its provenance block."""
744     slug = prov["slug"]
745     return {
746         "slug": slug,
747         "fps": _round(float(prov["fps"])),
748         "frame_width": _round(float(prov["frame_width"])),
749         "windowing": dict(prov.get("windowing", {})),
750         "kind": prov.get("kind", "synthetic"),
751         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
752         "paths": {
753             "dir": slug,
754             "trajectory": f"{slug}/trajectory.csv",
755             "labels": f"{slug}/labels.csv",
756             "expected": f"{slug}/expected.json",
757             "provenance": f"{slug}/provenance.json",
758         },
759     }
760
761
762 def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
763     """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
764     entries = load_manifest(fixtures_dir)
765     entries = [e for e in entries if e.get("slug") != prov["slug"]]

```

766 entries.append(_manifest_entry_for(prov))

51. assistant (2026-06-16T06:44:33.646Z)

Now I'll rewrite the reset/write/scan/promote block (lines 673-739). The plan:

1. Round `t0\_sec` to 6dp before subtracting (fix #2). Apply consistently.
2. Validate reset labels' starts >= 0 BEFORE writing (fix #3a) ? RefusalError.
3. Write into a temp dir, replay, scan, THEN promote to final via `os.replace` (fix #3b).
4. Drop `source\_note` from written provenance (fix #1b), but scan it defensively.

I need `tempfile` and `shutil` for temp dir + promotion. Let me add imports first.

52. user (2026-06-16T06:44:36.318Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

53. assistant (2026-06-16T06:44:38.426Z)

Now rewrite the reset/write/scan/promote block:

54. user (2026-06-16T06:45:13.367Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-06-16T06:45:16.401Z)

Now fix #5: `compare\_expected` extra-key blind spot ? add set-equality on `score` block AND top-level dict. Let me update `compare\_expected`.

56. user (2026-06-16T06:45:26.100Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

57. assistant (2026-06-16T06:45:28.181Z)

Now the `score` block key-set equality:

58. user (2026-06-16T06:45:33.210Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

59. assistant (2026-06-16T06:45:40.453Z)

Now fix #7: `load\_fixture` schema guard ? guard `expected` AND `provenance` independently.

60. user (2026-06-16T06:45:47.909Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

61. assistant (2026-06-16T06:45:51.608Z)

Now fix #6: `--refresh-snapshot --reason` mandatory. Change the WARN to `ap.error`.

62. user (2026-06-16T06:45:57.716Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

63. assistant (2026-06-16T06:46:01.343Z)

Now update the `--reason` help text and `--source-note` help text to document the new behavior.

64. user (2026-06-16T06:46:03.107Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

65. user (2026-06-16T06:46:10.399Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

66. assistant (2026-06-16T06:46:13.498Z)

Now let me verify the synthetic `--check` still passes (the most critical guardrail) before moving to tests. Let me run it.

67. user (2026-06-16T06:46:17.034Z)

```
[ok]   clean_single_rally: 1 candidates, 1 segments, F1=1.000000
[ok]   multi_rally_with_deadtime: 2 candidates, 2 segments, F1=1.000000
[ok]   occlusion_break: 2 candidates, 2 segments, F1=1.000000
[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ?
$6)
```

68. assistant (2026-06-16T06:46:21.347Z)

Synthetic `--check` still passes ? byte-identical behavior preserved. Now let me confirm the existing real-fixture tests still pass with my changes (they use `label="scenario"`, `"clean\_rally"`, `"a"`, `"b"` ? all valid slugs; and they don't assert `source\_note` presence). Let me also check the metric-invariance test at offset 9000 still works and confirm my temp-dir + negative-label refusal logic doesn't break the existing tests. Let me run the two golden test files.

69. user (2026-06-16T06:46:25.852Z)

```
..... [100%]
29 passed in 2.07s
```

70. assistant (2026-06-16T06:46:30.422Z)

All existing golden tests pass. Now let me verify fix #2 quickly with a fractional-second offset repro before writing the parametrized test, and verify fix #1 / #3 behave. Let me do a quick interactive repro for fixes #1, #2, #3.

71. user (2026-06-16T06:46:40.733Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 18, in <module>
    a = gf.freeze_real_fixture('scenario', traj, labels, slug='fx_on', license='owned',
minors_free=True, owned=True, fps=spec.fps, frame_width=spec.frame_width, fixtures_dir=tmp/'on',
time_origin_reset=True)
```

```

File "C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\gf-
audit\backend\eval\golden_fixtures.py", line 760, in freeze_real_fixture
    _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
    ~~~~~
    where=f"{out_slug}/expected.json", source_paths=source_paths)
    ~~~~~
File "C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\gf-
audit\backend\eval\golden_fixtures.py", line 566, in _scan_forbidden
    raise RefusalError(
        f"{where}: source-path token {token!r} found in written output ? the source "
        f"filename/path must never be persisted ($4c metadata strip).")
backend.eval.golden_fixtures.RefusalError: fx_on/expected.json: source-path token 't' found in
written output ? the source filename/path must never be persisted ($4c metadata strip).

```

72. assistant (2026-06-16T06:46:47.292Z)

Interesting ? this is a real consequence of fix #4. My repro used a source named `t.csv` (stem `t`), and the single-char `t` appears everywhere in the JSON. This is actually correct behavior now (exact-membership scan catches `t` as a leaked token, where `t` is not a schema literal). The old substring-skip would have skipped `t` because `t` in "trajectory". So my repro just used a pathological 1-char stem. Let me rename the source files to something not present in the output, like `srcabc123.csv`.

This actually surfaces a subtlety worth confirming: a single/short generic stem like `t` will now trip the scan. But that's the intended hardening ? the existing real-fixture tests use `src\_trajectory.csv` / `src\_labels.csv` (stems `src\_trajectory`/`src\_labels`), which won't appear in output. Let me re-run with a safe source name.

73. user (2026-06-16T06:46:59.782Z)

```

fix#2 frac-offset(7) score equal: False
fix#1a OK: free-text label refused; partial dir exists? False
fix#1b source_note key present? False
fix#3a OK: negative-label refused; partial dir exists? False
msg has predates: True

```

74. assistant (2026-06-16T06:47:04.406Z)

Fixes #1a, #1b, #3a all work. But fix #2 still shows score NOT equal at offset 7. Let me investigate why. The issue: I round `t0\_sec` to 6dp, but the trajectory is shifted by integer `t0\_frame` while labels are shifted by `t0\_sec`. The candidate intervals are derived from the `\*frame\*` timeline `(frame)/fps`, while labels use `t0\_sec`-subtracted seconds. The two shifts only match if `t0\_sec == t0\_frame/fps` exactly in the metric. Let me dig into where the divergence is.

75. user (2026-06-16T06:47:13.646Z)

```

on score: {'iou_threshold': 0.5, 'num_pred': 1, 'num_gt': 1, 'true_positives': 1,
'false_positives': 0, 'false_negatives': 0, 'precision': 1.0, 'recall': 1.0, 'f1': 1.0,
'mean_iou': 0.983333, 'mean_start_error': 0.033333, 'mean_end_error': 0.033333}
off score: {'iou_threshold': 0.5, 'num_pred': 1, 'num_gt': 1, 'true_positives': 1,
'false_positives': 0, 'false_negatives': 0, 'precision': 1.0, 'recall': 1.0, 'f1': 1.0,
'mean_iou': 0.983333, 'mean_start_error': 0.033334, 'mean_end_error': 0.033333}
DIFF mean_start_error 0.033333 0.033334 delta 1.000000000001e-06
on labels.csv: start,end
0.000000,4.000000
off labels.csv: start,end
0.233333,4.233333
on first traj rows: ['Frame,Visibility,X,Y', '0,1,640.000000,360.000000',
'1,1,723.164676,372.202099']
off first traj rows: ['Frame,Visibility,X,Y', '7,1,640.000000,360.000000',
'8,1,723.164676,372.202099']
t0_sec rounded = 0.233333 raw= 0.23333333333333334

```

76. assistant (2026-06-16T06:47:44.025Z)

Now I see the root cause precisely. The divergence is a **serialization-then-reparse** asymmetry, not a `t0_sec`` rounding issue:

```
- WITH reset: label start written as `0.000000`. Candidate start derived from frame 0 ?  
`0/30 = 0.0`. start_error = `|0.0 - 0.0|` ... actually the candidate start in distill comes from  
`trajectory_to_action_windows`. Let me think: candidate start = `(first_active_frame)/fps`.  
mean_start_error = 0.033333 (with reset) vs 0.033334 (without).  
- WITHOUT reset: label start written as `0.233333` (rounded from 0.2333...), candidate start  
derived from frame 7 ? `7/30 = 0.23333...`. start_error = `|0.23333... - 0.233333|`.
```

The asymmetry: the *labels* get re-quantized to 6dp on write (`0.233333``), but the *candidate* start is computed fresh from the frame index at full precision (`7/30 = 0.2333333...``). So in the no-reset case, label `0.233333`` vs candidate `0.2333333`` differ by $\sim 3.3e-7$, which perturbs the start_error in the 6th decimal. In the reset case, label `0.0`` vs candidate `0.0`` match exactly.

So the real fix: the *candidate* times also flow through `_round`` (6dp) in the snapshot, AND the start\_error is computed by `metrics.score`` from the raw (unrounded) candidate vs the parsed label. The parsed label is `0.233333`` (6dp, from the CSV) but candidate is full-precision `7/30``.

The cleanest fix consistent with the prompt ("round `t0_sec` to 6dp before subtracting from BOTH labels and consistently deriving the trajectory shift so the applied shift is an exact 6dp quantity"): the problem is the trajectory shift is applied as an *integer frame shift*, not a 6dp-second shift. When reset is OFF, no shift is applied, so candidate = `7/30`` (full precision) but label = `0.233333`` (6dp quantized at write). The label quantization-on-write is the asymmetry source.

The prompt's intent: make the applied shift an exact 6dp quantity for *both* sides. With reset ON, candidate frame ? `0/30=0`` and label ? `0``; both exact. With reset OFF, candidate = `7/30`` (full precision) but label CSV is quantized to `0.233333``. The mismatch is between full-precision candidate-from-frame and 6dp-quantized label.

The robust solution: round the candidate interval times to 6dp in `score()`` input ? but I shouldn't modify `metrics.score`` or `distill_local``. The real asymmetry is that labels get quantized through CSV round-trip while candidates don't.

Let me look at how `metrics.score`` uses these, and reconsider. The candidate intervals come back from `build_candidates`` at full precision and are fed directly to `predict`?`score``. The labels come from `load_gt_intervals`` reading the *written* CSV (6dp). So in the snapshot, candidate full-precision vs label 6dp.

Actually wait ? in *both* reset and no-reset, the candidate is full precision and the label is 6dp-from-CSV. The difference is whether the label happens to round-trip cleanly. With reset, label = exactly `0.0`` (clean). Without reset, label = `0.233333`` but candidate = `0.2333333...``.

The prompt says round `t0_sec`` before subtracting "so the applied shift is an exact 6dp quantity ? bit-identical scores." Let me reconsider: the key is that BOTH candidate and label, after reset, should be derived from the *same quantized origin*. Since the candidate is `frame/fps`` and the label is `(orig_sec - t0_sec)``, for these to match in the metric, I should ensure consistency.

Actually the cleanest interpretation that achieves bit-identical: the snapshot already `_round`s`` `control_segments`` and `candidates[].start/end`` to 6dp. But `metrics.score`` computes `mean_start_error`` from the *raw* preds and *raw* gts (before the snapshot's `_round``). The raw gts are read from CSV (6dp), raw preds are full precision.

So to make scores bit-identical regardless of offset, the *predicted/candidate* times that feed `score`` must be quantized to 6dp to match how labels are quantized through the CSV. I can't change `score``. But I CAN make the freeze write the trajectory such that frame indices reset to 0 ? which is what reset does, making candidate = `0/30 = 0.0`` exactly, matching label `0.0``.

The issue is only the *no-reset* path diverging from reset. The prompt's framing: "reset-vs-

no-reset scores diverge on sub-second offsets... round `t0_sec` to 6dp before subtracting from BOTH the labels and consistently deriving the trajectory shift."

The deepest consistent fix: when reset is ON, shift the trajectory by `t0_frame` (integer) AND the labels by `t0_sec` where `t0_sec = t0_frame/fps` rounded to 6dp. With reset ON the trajectory frame becomes 0 so `candidate=0.0` and `label = orig_label - t0_sec`. For metric-invariance vs no-reset, we need: `(candidate_reset - label_reset) == (candidate_noreset - label_noreset)`. `candidate_reset = candidate_noreset - t0_frame/fps`. `label_reset = label_noreset - t0_sec`. If `t0_sec == t0_frame/fps` exactly, the differences are identical *before quantization*. The divergence is purely from CSV 6dp quantization of labels.

Let me verify: is the divergence between reset and no-reset, or is it that no-reset itself is "wrong" due to label quantization? The test that matters (prompt fix #2) is **reset-vs-no-reset equality**. The cleanest way to guarantee that is to also quantize the trajectory shift to land frames on integers (already integer) AND ensure candidate times feeding the score are themselves 6dp-quantized the same way labels are.

The label is quantized because it's written to CSV then re-read. The candidate is NOT written to a CSV that's re-read ? it's computed live from the frame CSV. So the asymmetry is structural.

Let me check `metrics.score` to see if I can round at the snapshot's input boundary without touching shared code.

77. user (2026-06-16T06:47:44.937Z)

```
1      """Pure temporal interval-matching metrics for rally evaluation.
2
3      No I/O, no DB, no global state ? every function here is deterministic and unit
4      tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5      start < end. We score a list of *predicted* rally intervals against a list of
6      *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7      """
8
9      from dataclasses import dataclass, field
10     from typing import List, Tuple, Optional
11
12     Interval = Tuple[float, float]
13
14
15     def interval_iou(a: Interval, b: Interval) -> float:
16         """Temporal intersection-over-union of two intervals.
17
18         Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19         overlap / union, both measured as durations on the timeline.
20         """
21         a_start, a_end = a
22         b_start, b_end = b
23         if a_end <= a_start or b_end <= b_start:
24             return 0.0
25         inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26         if inter <= 0.0:
27             return 0.0
28         union = (a_end - a_start) + (b_end - b_start) - inter
29         return inter / union if union > 0 else 0.0
30
31
32     @dataclass
33     class Match:
34         """One matched predicted?ground-truth pair."""
35         pred_index: int
36         gt_index: int
37         iou: float
38
39
40     @dataclass
```

```

41     class MatchResult:
42         """Outcome of matching predictions to ground truth at a given IoU threshold."""
43         matches: List[Match] = field(default_factory=list)
44         unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45         unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
46     (misses)
47
48     def match_intervals(preds: List[Interval], gts: List[Interval],
49                       iou_threshold: float = 0.5) -> MatchResult:
50         """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52         Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53         are claimed highest-IoU first, and each prediction/GT can be used once. This
54         keeps a single prediction from being credited for two ground-truth rallies
55         (and vice versa).
56         """
57         candidates = []
58         for pi, p in enumerate(preds):
59             for gi, g in enumerate(gts):
60                 iou = interval_iou(p, g)
61                 # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
62                 # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63                 if iou > 0.0 and iou >= iou_threshold:
64                     candidates.append((iou, pi, gi))
65                 # Highest IoU first; ties broken by indices for determinism.
66                 candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68         used_pred, used_gt = set(), set()
69         matches: List[Match] = []
70         for iou, pi, gi in candidates:
71             if pi in used_pred or gi in used_gt:
72                 continue
73             used_pred.add(pi)
74             used_gt.add(gi)
75             matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77         unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78         unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79         return MatchResult(matches=matches,
80                           unmatched_pred_indices=unmatched_pred,
81                           unmatched_gt_indices=unmatched_gt)
82
83
84     @dataclass
85     class Score:
86         """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87         iou_threshold: float
88         num_pred: int
89         num_gt: int
90         true_positives: int
91         false_positives: int
92         false_negatives: int
93         precision: float
94         recall: float
95         f1: float
96         mean_iou: float # over matched pairs (0.0 if none)
97         mean_start_error: float # mean |pred_start - gt_start| over matches, seconds
98         mean_end_error: float # mean |pred_end - gt_end| over matches, seconds
99
100         def as_dict(self) -> dict:
101             return self.__dict__.copy()
102
103

```

```

104     def score(preds: List[Interval], gts: List[Interval],
105               iou_threshold: float = 0.5,
106               match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114
115         tp = len(mr.matches)
116         fp = len(mr.unmatched_pred_indices)
117         fn = len(mr.unmatched_gt_indices)
118
119         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122     else 0.0
123
124     if mr.matches:
125         mean_iou = sum(m.iou for m in mr.matches) / tp
126         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127 mr.matches) / tp
128         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129 mr.matches) / tp
130     else:
131         mean_iou = mean_start_err = mean_end_err = 0.0
132
133     return Score(
134         iou_threshold=iou_threshold,
135         num_pred=len(preds), num_gt=len(gts),
136         true_positives=tp, false_positives=fp, false_negatives=fn,
137         precision=precision, recall=recall, f1=f1,
138         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
139     )
140
141     def pr_curve(preds: List[Interval], gts: List[Interval],
142                 thresholds: Optional[List[float]] = None) -> List[Score]:
143         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
144
145         Lets you see strict-vs-lenient behaviour rather than a single operating point.
146         """
147         if thresholds is None:
148             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
149         return [score(preds, gts, iou_threshold=t) for t in thresholds]

```

78. assistant (2026-06-16T06:48:05.327Z)

So `score` computes `mean\_start\_error = |pred\_start - gt\_start|`. `pred` comes from `build\_candidates` (full precision frame/fps), `gt` comes from the CSV (6dp). The error is `|7/30 - 0.233333|` ? `|0.233333... - 0.233333|` = `3.3e-7`, which after the rally's natural start error (window starts at frame 8 not 7, giving the 0.033333 baseline) shifts the 6th decimal.

The root issue confirmed: in `replay\_fixture`, `score(preds, gts, iou)` is fed `gts` that came from the **written 6dp CSV**, but `preds` at full precision. The asymmetry between reset/no-reset is whether the labels happen to round-trip to clean values.

The cleanest fix that doesn't touch shared code and makes scores bit-identical: in `replay\_fixture`, round the prediction interval endpoints AND the gts to 6dp **before** calling `score`. But that would change the synthetic fixtures' committed `expected.json`? No ? the synthetic fixtures all produce clean values (F1=1.0, errors that are already stable), and

rounding to 6dp wouldn't change their already-6dp-clean scores. But the prompt explicitly says: "If any change would alter a synthetic fixture's committed expected.json, STOP and report." And "DO NOT modify distill_local.build_candidates."

But the prompt's prescribed fix for #2 is narrower: "round `t0\_sec` to 6dp BEFORE subtracting from BOTH the labels and (consistently) deriving the trajectory shift, so the applied shift is an exact 6dp quantity." This addresses the **reset** path. But the divergence I measured comes from the **no-reset** path's label quantization vs full-precision candidate, which `t0\_sec` rounding doesn't touch (no-reset has $t0_sec=0$).

Let me reconsider. The fundamental asymmetry is: **labels** round-trip through a 6dp CSV, candidates don't. The fix that makes reset==no-reset is to ALSO round the candidate intervals fed to `score` to 6dp ? matching the label quantization. This is the "consistently" part. Let me check whether rounding preds+gts to 6dp before `score` changes the synthetic `expected.json`.

The synthetic case: candidate from frame 1 (start) ? $1/30 = 0.033333...$, gt `0.0`. Without rounding preds: `score` sees $\text{pred_start} = 0.033333...$. With rounding: 0.033333 . The snapshot's `mean_start_error` is `_round(sc.mean_start_error)`. For synthetic: $|0.0333333 - 0.0| = 0.0333333$? `_round` ? 0.033333 . With pred rounded first: $|0.033333 - 0.0| = 0.033333$? `_round` ? 0.033333 . Same. So rounding preds to 6dp before score does NOT change synthetic snapshots (they're already producing clean 6dp). Let me verify this empirically rather than reason about it.

Actually, the safest and most targeted approach per the prompt: the prompt's fix #2 is specifically about the **reset** path and says round `t0\_sec`. The fix is to apply a 6dp-quantized shift consistently. Let me reconsider what "consistently deriving the trajectory shift" means: the trajectory is shifted by integer frames; the *effective second-shift* of the trajectory is $t0_frame/fps$. If I round `t0\_sec` to 6dp for labels but the trajectory's effective shift is the full-precision $t0_frame/fps$, they differ by up to $5e-7$? which is exactly the kind of drift. But with reset, $frame \neq 0$ means $\text{candidate} = 0/30 = 0$ exactly, and $\text{label} = \text{orig} - t0_sec(6dp)$.

Let me step back and test empirically what makes reset==no-reset. The clean approach: round preds and gts to 6dp at the `replay_fixture` score boundary. Let me test if that yields equality AND preserves synthetic.

79. user (2026-06-16T06:48:14.787Z)

```
off=7 equal=False diffs={'mean_start_error': 1.000000000001e-06}
off=11 equal=False diffs={'mean_end_error': 1.000000000001e-06}
off=13 equal=False diffs={'mean_start_error': 1.000000000001e-06}
```

80. assistant (2026-06-16T06:48:19.545Z)

Confirmed: worst $|?| = 1.000000000001e-06$, exactly as the audit reported, just past `_FLOAT_TOL=1e-6`. The fix: round preds and gts to 6dp before computing the score in `replay_fixture`, so the score uses the same 6dp-quantized values that the CSV serializes ? making reset and no-reset bit-identical. Let me verify this approach works and preserves synthetic.

Let me look at the exact lines in `replay_fixture` where `score` is called and apply the rounding at that boundary.

81. user (2026-06-16T06:48:20.631Z)

```
329         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
330
331         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
332         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
333
334         Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
335         control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
```

```

336 merges overlaps; NO numpy is touched.
337 """
338 fdir = _resolve_dir(slug_or_dir)
339 slug = fdir.name
340 traj_csv = fdir / "trajectory.csv"
341 labels_csv = fdir / "labels.csv"
342 prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
343
344 fps, fw = _fps_fw(prov, slug)
345 iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
346 proposal = _pinned_proposal(prov)
347
348 intervals, feature_rows = distill_local.build_candidates(
349     str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
350
351 # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
352 # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353 # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354 features: Dict[str, List[float]] = {
355     name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356 }
357 gts = load_gt_intervals(str(labels_csv), strict=True)
358 vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360 preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361 sc = score(preds, gts, iou)
362
363 candidates: List[Dict[str, Any]] = []
364 for j, (s, e) in enumerate(intervals):
365     shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
366     # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
367     shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
368     candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
369
370 return {
371     "slug": slug,
372     "schema_version": CURRENT_SCHEMA,
373     "iou": _round(iou),

```

82. assistant (2026-06-16T06:48:30.287Z)

Now I understand the wiring fully. The score is computed from `preds` (full precision from `merge\_overlaps`) and `gts` (6dp from CSV).

The cleanest fix consistent with the prompt's framing (and the snapshot already rounds everything to 6dp for serialization): compute the score from ****6dp-rounded**** preds and gts, so it matches the values that are actually serialized into the fixture and the CSV. This makes reset/no-reset bit-identical (both quantize to the same 6dp grid) AND is the honest computation ? the committed snapshot's candidate/segment endpoints are 6dp, so the score should reflect those same 6dp endpoints.

But the prompt's literal fix #2 instruction is to round `t0\_sec`. I already did round `t0\_sec`. The remaining 1e-6 comes from the candidate-vs-label quantization asymmetry, which the prompt anticipates with "consistently deriving the trajectory shift, so the applied shift is an exact 6dp quantity ? bit-identical scores."

I think the most faithful and robust implementation: round the score inputs (preds and gts) to

6dp in `replay\_fixture` before calling `score`. This guarantees bit-identical scores for any offset, is consistent with the 6dp serialization contract, and ? critically ? I must verify it does NOT change synthetic `expected.json`. Let me make that change and verify.

83. user (2026-06-16T06:48:36.914Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

84. assistant (2026-06-16T06:48:40.841Z)

Now verify: synthetic `--check` still passes (byte-identical), AND reset/no-reset now equal across offsets.

85. user (2026-06-16T06:48:49.548Z)

Exit code 1

[FAIL] clean_single_rally: 1 mismatch(es) (characterization = behaviour-locking, NOT correctness ? \$6):

- score.mean_iou: 0.983334 != 0.983333 (|?| > 1e-06)

[ok] multi_rally_with_deadtime: 2 candidates, 2 segments, F1=1.000000

[ok] occlusion_break: 2 candidates, 2 segments, F1=1.000000

[golden-fixtures] 2/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ? \$6)

86. assistant (2026-06-16T06:48:55.303Z)

Rounding preds+gts before score DID change a synthetic fixture's `mean\_iou` (0.983334 vs committed 0.983333). This violates the "byte-identical synthetic" constraint. The prompt explicitly forbids this approach if it alters committed synthetic `expected.json`. So I must NOT round both preds and gts at the score boundary.

Let me revert that and use the prompt's prescribed approach precisely: round `t0\_sec` to 6dp and "consistently deriving the trajectory shift." The synthetic fixtures are frozen at offset 0 with reset effectively a no-op (their trajectory starts at frame 0 already, t0_frame=0). So the synthetic path is the "no-reset / t0=0" path. The fix #2 must only affect the **real-fixture reset** path, leaving the no-reset/synthetic computation untouched.

Reverting:

87. user (2026-06-16T06:49:00.592Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

88. assistant (2026-06-16T06:49:19.372Z)

Now the correct, surgical fix for #2: the asymmetry is that the **labels** are quantized to 6dp by the CSV round-trip, while the **trajectory frames** shift by an exact integer. With reset, `t0\_frame` integer shift ? candidate times = `frame/fps`. Without reset, candidate times = `(frame+off)/fps`. For the boundary errors `|pred\_start - gt\_start|` to be identical between reset/no-reset, the **gt** quantization must be consistent.

The real fix the prompt prescribes: make the **label shift** exactly equal the trajectory's effective shift on the 6dp grid. The trajectory shifts by integer `t0\_frame`. The label should shift by the **same amount the candidate timeline shifts**, which is `t0\_frame/fps`. The divergence arises because:

- reset labels: `orig\_label - t0\_sec` then **written to CSV at 6dp** then re-read

- no-reset labels: `orig\_label` written at 6dp then re-read

- candidates: `frame/fps` (reset) vs `(frame+off)/fps` (no-reset), full precision, NOT quantized

So `pred\_start - gt\_start`:

```
- reset: `frame/fps - round6(orig - round6(t0_frame/fps))`  
- no-reset: `(frame+off)/fps - round6(orig)`
```

These differ because the reset label has `t0\_sec` subtracted (a 6dp value) while the candidate has `off` frames removed (exact). When `t0\_frame == off` (the reset offset equals the synthetic offset), `frame\_reset = frame\_noreset - off`, so `pred\_reset = pred\_noreset - off/fps`. And `gt\_reset = round6(orig - round6(off/fps))` vs `gt\_noreset = round6(orig)`. The difference `gt\_noreset - gt\_reset = round6(orig) - round6(orig - round6(off/fps))`, which only exactly equals `round6(off/fps)` if no double-rounding error ? and that's the 1e-6 slippage.

The prompt's exact prescription: round `t0\_sec` to 6dp before subtracting (done), AND "consistently deriving the trajectory shift, so the applied shift is an exact 6dp quantity." The trajectory shift in seconds must be an exact 6dp quantity matching `t0\_sec`. The way to make the candidate timeline shift by exactly `t0\_sec` (6dp): rather than shifting frames by integer `t0\_frame` (which shifts seconds by `t0\_frame/fps`, full precision), I cannot easily change candidate derivation without touching `build\_candidates`.

Let me reconsider ? the cleanest fix that does NOT touch synthetic: the issue is double-rounding `round6(orig - round6(t0))` vs `round6(orig) - round6(t0)`. If instead of subtracting `t0\_sec` from each label and letting the CSV round it, I compute the reset label as `round6(orig) - round6(t0)` ? i.e. round the **original** label to 6dp first, then subtract the 6dp `t0`, the subtraction of two 6dp quantities is exact (no double rounding). Let me verify that resolves it. The original labels come from `load\_gt\_intervals` (parsed from source CSV, already arbitrary precision). The synthetic path doesn't use `\_reset\_labels` (no real freeze), so this only touches the real path ? synthetic untouched.

Let me update `\_reset\_labels` to round each original endpoint to 6dp before subtracting the 6dp `t0\_sec`:

89. user (2026-06-16T06:49:26.440Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

90. user (2026-06-16T06:49:39.908Z)

```
[ok] clean_single_rally: 1 candidates, 1 segments, F1=1.000000  
[ok] multi_rally_with_deadtime: 2 candidates, 2 segments, F1=1.000000  
[ok] occlusion_break: 2 candidates, 2 segments, F1=1.000000  
[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ? S6)  
=== invariance ===  
off= 7 equal=False worst|delta|=1e-06 <=TOL:False  
off= 11 equal=False worst|delta|=1e-06 <=TOL:False  
off= 13 equal=False worst|delta|=1e-06 <=TOL:False  
off= 1 equal=False worst|delta|=1e-06 <=TOL:False  
off= 29 equal=False worst|delta|=1e-06 <=TOL:False  
off= 9000 equal=True worst|delta|=0 <=TOL:True
```

91. assistant (2026-06-16T06:50:28.463Z)

Synthetic preserved, but the rounded-label approach didn't fix the sub-second offsets ? still 1e-06 (now exactly at the boundary, `<=TOL` is False because the comparison is `> \_FLOAT\_TOL` strictly... actually `1e-06 <= 1e-06` is True in Python, but it printed False ? that's float repr; the actual value is `1.000000000001e-06`, still just over). The remaining drift is on the **candidate side**, not labels.

The candidate `pred\_start` = `frame/fps`. With reset, frame is shifted by integer `t0\_frame`, so `pred\_start\_reset = (frame - t0\_frame)/fps`. Without reset, `pred\_start\_noreset = frame/fps`. The difference is exactly `t0\_frame/fps`. But the **gt** difference is `t0\_sec` (= `round6(t0\_frame/fps)`). So `(pred - gt)`:
- reset: `(frame-t0\_frame)/fps - (round6(orig) - t0\_sec)`
- noreset: `frame/fps - round6(orig)`

- difference: `-t0_frame/fps + t0_sec` = `t0_sec - t0_frame/fps` = `round6(t0_frame/fps) - t0_frame/fps` ? up to 5e-7.`

So the candidate timeline shifts by the **exact** `t0_frame/fps`` (full precision) while the label shifts by `t0_sec`` (6dp-rounded). That mismatch is the residual. To make them consistent, the candidate shift must also be exactly `t0_sec`` ? but the candidate shift is dictated by the integer frame shift, which is `t0_frame/fps`` exactly.

The prompt says: "round `t0_sec` to 6dp BEFORE subtracting from BOTH the labels and (consistently) deriving the trajectory shift." This means: the trajectory shift should ALSO be derived from the 6dp `t0_sec``. But the trajectory shifts by integer **frames** ? the only frame-quantized shift available. So "consistently deriving the trajectory shift" must mean: choose `t0_frame`` such that `t0_frame/fps == t0_sec`` exactly. Since `t0_sec = round6(t0_frame/fps)``, they're only equal if `t0_frame/fps`` is already 6dp-clean.

This means the residual is fundamentally because the candidate timeline lives on the `frame/fps`` grid (e.g. multiples of `1/30 = 0.0333...`), which is NOT 6dp-clean, while labels are 6dp-clean. The boundary error `|pred - gt|`` therefore has the candidate at full `frame/fps`` precision.

The honest resolution: **also round the candidate (pred) endpoints to 6dp before scoring** ? but ONLY in a way that doesn't change synthetic. We saw rounding both `preds+gts` changed synthetic `mean_iou``. But what if I round only at the `_reset_labels`/freeze` path? No ? `replay_fixture`` is shared.

Let me reconsider what actually changed synthetic when I rounded `preds`: it was `mean_iou`` going 0.983333 ? 0.983334. That's because rounding the `pred` endpoints changes the IoU. The synthetic candidate endpoints are at `frame/fps`` (e.g. `0.033333...`, `3.966667``). Rounding to 6dp gives `0.033333``, `3.966667`` ? barely different, but enough to flip `mean_iou`` in the 6th place.

So I cannot round `preds` globally. The prompt's fix #2 is specifically scoped to the **reset path being metric-invariant** ? the comparison is reset-vs-no-reset of the SAME real source. Both go through `replay_fixture`` identically; the only difference is the frozen CSVs. So whatever rounding I apply must be in `replay_fixture`` (shared) and must be a no-op for synthetic.

Key realization: the residual 1e-6 is **inherent to comparing a frame-grid candidate against a 6dp label across two different absolute frame origins**. There's no way to make `frame/fps`` and `(frame+off)/fps`` differ by exactly a 6dp quantity unless I quantize the candidate.

Given the constraint "byte-identical synthetic", the resolution the prompt actually wants: relax the test to `assert `|?| <= _FLOAT_TOL`` (which is what the characterization gate uses ? `1e-6`` tolerance), AND make the worst-case land **at or under** `_FLOAT_TOL``. Currently it's `1.000000000001e-06`` (floating point repr of what should be 1e-6). Let me check: is the residual actually `> _FLOAT_TOL`` in the `compare_expected`` sense? The audit said "past `_FLOAT_TOL``".

Let me reconsider the math. Actually, the cleanest fix that the prompt describes and that works: instead of shifting the trajectory by integer frames and labels by seconds (two different grids), shift **both in the seconds domain on the 6dp grid**. But the trajectory **MUST** be integer frames (it's a frame-indexed CSV).

Hold on ? let me reconsider. What if the candidate-side residual can be eliminated by making `t0_sec`` exactly `t0_frame/fps`` WITHOUT rounding, but rounding **ONLY** the label endpoints (not `t0`)? Let me test: `t0_sec = t0_frame/float(fps)`` (NOT rounded), and `_reset_labels`` rounds each endpoint. Then:

- `gt_reset = round6(round6(orig) - t0_frame/fps)``... no, the CSV re-rounds anyway.

Let me step back and think about what the score actually needs. `mean_start_error = |pred_start - gt_start|``. For `reset==no-reset`:
`pred_start_reset - gt_start_reset == pred_start_noreset - gt_start_noreset`` (assuming same sign, which holds when `pred>gt` or `pred<gt` consistently).

`pred_start_reset = (af - t0_frame)/fps`` where `af`` = active start frame.
`pred_start_noreset = af/fps``.
`pred_reset - pred_noreset = -t0_frame/fps``.

`gt_start_reset`` = value read back from reset CSV = `round6(round6(orig) - t0_sec)``.

```
`gt_start_noreset` = `round6(orig)`.  
`gt_reset - gt_noreset` = round6(round6(orig) - t0_sec) - round6(orig)`.
```

For invariance: ``pred_reset - pred_noreset == gt_reset - gt_noreset``, i.e.
``-t0_frame/fps == round6(round6(orig) - t0_sec) - round6(orig)``.

If ``t0_sec = round6(t0_frame/fps)``, and ``round6(orig) - t0_sec`` is already 6dp (diff of two 6dp = exact 6dp), then ``gt_reset - gt_noreset = (round6(orig) - t0_sec) - round6(orig) = -t0_sec = -round6(t0_frame/fps)``.

So we need ``-t0_frame/fps == -round6(t0_frame/fps)``, i.e. ``t0_frame/fps`` must be 6dp-clean. It isn't (e.g. ``7/30``). So the residual = ``t0_frame/fps - round6(t0_frame/fps)`` ? $\pm 5e-7$, which doubles into the ``|.|`` difference up to $1e-6$.

****The candidate (pred) timeline is the problem**** ? it's NOT serialized/quantized, so it carries full ``frame/fps`` precision. The ONLY way to make it 6dp-consistent without changing synthetic is... to realize the synthetic case ALSO has this exact same ``frame/fps`` precision but reset is a no-op there (`t0_frame=0`), so the synthetic never compares two origins.

Given the hard constraint, the right move is: ****the test asserts ``|?| <= _FLOAT_TOL``, and I make the implementation get the worst case to ``exactly ? 1e-6``. The current ``1.000000000001e-06`` is ``> 1e-6``. I can eliminate the doubling by rounding the ``score's`` boundary-error inputs**** ? but only in ``replay_fixture`` and only such that synthetic is unchanged.

Let me reconsider the synthetic breakage: rounding preds changed ``mean_iou`` (an IoU computation), not ``mean_start_error``. What if I round preds+gts to 6dp ****only for the score computation**** but the synthetic ``mean_iou`` change was real... Let me actually check what synthetic ``mean_iou`` SHOULD be and whether the committed 0.983333 is itself a rounding artifact.

92. user (2026-06-16T06:50:36.666Z)

```
preds full prec: [(0.03333333333333333, 3.966666666666667)]  
gts: [(0.0, 4.0)]  
iou full: 0.9833333333333334  
iou 6dp-rounded preds/gts: 0.9833335000000001  
committed mean_iou: 0.983333
```

93. assistant (2026-06-16T06:51:02.671Z)

Now I see it clearly. The committed synthetic ``mean_iou`` is ``_round(0.9833333...) = 0.983333``. If I round preds/gts to 6dp ****before**** computing IoU, I get ``0.9833335`` ? ``_round`` ? ``0.983334``. So rounding the ``inputs`` to IoU changes the result because IoU is a nonlinear ratio sensitive to endpoint quantization. That's why the global pre-round breaks synthetic.

So I cannot pre-round the IoU inputs. The boundary-error fix must be ****separate**** from the IoU computation. ``mean_iou`` must use full-precision endpoints (preserving synthetic), but ``mean_start_error`/`mean_end_error`` are the ONLY fields that diverge (linear differences).

The cleanest fix honoring all constraints: leave ``score()`` exactly as-is for IoU/matching, and in ``replay_fixture``, after computing ``sc``, ****round the boundary-error inputs consistently**** ? but ``score()`` is one call. I can't selectively round.

Alternative: the divergence is purely in ``mean_start_error`/`mean_end_error``. These are already ``_round``-ed in the snapshot. The snapshot rounds ``_round(sc.mean_start_error)``. The reset gives ``_round(X)`` and no-reset gives ``_round(Y)`` where ``|X-Y| ? 5e-7``, and they round to adjacent 6dp values (e.g. 0.033333 vs 0.033334) ? ``|?_snapshot| = 1e-6``.

The fundamental fix per the prompt: make `X==Y` by ensuring the ****trajectory shift**** applied is the exact same 6dp quantity as the label shift. The trajectory shift in the ``score's`` eyes is ``t0_frame/fps``. To make it ``t0_sec`` (6dp), I'd need the candidate start to be quantized.

****The correct, constraint-satisfying fix:**** quantize the ****predicted interval endpoints to 6dp before scoring**** for the boundary-error computation, but NOT for IoU. Since I can't split ``score()``, and rounding inputs breaks IoU ? the answer is to round preds/gts to 6dp ****only when**

they would be serialized**, i.e. the score should be computed on the 6dp grid for boundary errors but full precision for IoU.

Given that `score()` can't be split and the prompt forbids changing synthetic bytes, let me re-read the prompt's exact wording for fix #2 once more: `"round t0_sec to 6dp (_round)` BEFORE subtracting from BOTH the labels and (consistently) deriving the trajectory shift, so the applied shift is an exact 6dp quantity ? bit-identical scores."

The phrase "deriving the trajectory shift" ? the trajectory shift is `t0_frame`. If I derive `t0_frame` from the 6dp `t0_sec` such that the trajectory's effective second-shift equals `t0_sec` exactly... but frames are integers.

Wait ? maybe the intended reading is simpler: shift the `trajectory frames` by `t0_frame`, but compute the `labels` shift using `t0_sec = _round(t0_frame/fps)`, AND ALSO the candidate intervals are derived from frames that were shifted by `t0_frame` ? so in the RESET fixture, the candidate start = `(af - t0_frame)/fps`. When this is `serialized into expected.json`, it's `_round`-ed. But `score()` uses the unrounded value.

The test asserts `|?| <= _FLOAT_TOL`. The prompt says: "asserting score-equality AND `|?| <= _FLOAT_TOL`." So the test accepts EITHER exact equality OR within tolerance. If I can't get exact equality without breaking synthetic, the `|?| <= _FLOAT_TOL` clause is the realistic target. The current worst is `1.000000000001e-06` which is `> 1e-6`.

Let me get it to `<= 1e-6` exactly. The doubling happens because `_round(X)` and `_round(Y)` land on opposite sides of a 6dp boundary. If I compute boundary errors on a consistent grid... Let me try: `round t0_sec` AND derive the label shift so the reset label is computed as `round6(orig) - round6(t0_frame/fps)` (already done), AND additionally round the `candidate` start/end to 6dp `before passing to score`, but keep IoU full-precision by passing a precomputed `match_result` based on full-precision intervals. `score()` accepts `match_result`! So I can compute matches+IoU on full-precision intervals, then call `score` with 6dp-rounded preds/gts for the boundary errors and the precomputed full-precision match_result for IoU.

Wait ? but `score` recomputes `mean_iou` from `mr.matches` (the match IoUs), and boundary errors from `preds[idx]`/`gts[idx]`. If I pass `match_result` computed on full-precision intervals, `mean_iou` stays full-precision (synthetic-safe), but boundary errors use whatever preds/gts I pass. So: compute `mr = match_intervals(preds_full, gts_full, iou)`, then `score(preds_rounded, gts_rounded, iou, match_result=mr)`. The `mr.matches` carry full-precision IoU ? `mean_iou` unchanged. Boundary errors use rounded ? bit-identical reset/no-reset.

But does this change synthetic boundary errors? Synthetic `mean_start_error`: full pred `0.033333...`, gt `0.0` ? `0.033333...` ? `_round` ? `0.033333`. Rounded pred `0.033333`, gt `0.0` ? `0.033333` ? `_round` ? `0.033333`. Same. Let me verify this approach preserves synthetic AND fixes invariance.

94. user (2026-06-16T06:51:04.188Z)

```
49     ""
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import os
56     import re
```

95. user (2026-06-16T06:51:07.727Z)

```
58     import sys
59     import tempfile
60     from dataclasses import dataclass
61     from pathlib import Path
62     from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
65     -----
```

```
65     from backend.eval import distill_local
66     from backend.eval.experiment import CONTROL, VideoCandidates, predict
67     from backend.eval.gt_loader import load_gt_intervals
68     from backend.eval.metrics import score
69     from backend.eval.regression import gt_hash
```

96. assistant (2026-06-16T06:51:13.148Z)

Hmm, this introduces complexity and a subtle behavior. Let me reconsider whether this is worth it versus the simpler, prompt-faithful path. Actually, re-reading the prompt: it says round `t0\_sec` "so the applied shift is an exact 6dp quantity ? bit-identical scores. Update the docstring/claim to match." And the test asserts "score-equality AND $|?| \leq \text{_FLOAT_TOL}$."

The simplest robust fix that achieves true bit-identical scores AND preserves synthetic: pass a full-precision `match\_result` to `score` (so IoU is untouched) while feeding 6dp-rounded intervals for boundary errors. This is the precise, principled fix. Let me implement it. First, import `match\_intervals`.

97. user (2026-06-16T06:51:15.139Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

98. assistant (2026-06-16T06:51:17.186Z)

Now update the score call in `replay\_fixture`:

99. user (2026-06-16T06:51:24.113Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

100. user (2026-06-16T06:51:41.459Z)

```
[ok]   clean_single_rally: 1 candidates, 1 segments, F1=1.000000
[ok]   multi_rally_with_deadtime: 2 candidates, 2 segments, F1=1.000000
[ok]   occlusion_break: 2 candidates, 2 segments, F1=1.000000
[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ?
$6)
=== invariance ===
off=    1 equal=False worst|delta|=1e-06
off=    7 equal=False worst|delta|=1e-06
off=   11 equal=False worst|delta|=1e-06
off=   13 equal=False worst|delta|=1e-06
off=   17 equal=False worst|delta|=1e-06
off=   23 equal=False worst|delta|=1e-06
off=   29 equal=False worst|delta|=1e-06
off=  100 equal=False worst|delta|=1e-06
off=  900 equal=True worst|delta|=0
```

101. assistant (2026-06-16T06:51:46.041Z)

Still 1e-06. The boundary-error rounding alone isn't enough because the rounded preds still differ between reset/no-reset. Let me diagnose: with reset, `pred\_start = round6((af - t0\_frame)/fps)`; without reset, `pred\_start = round6(af/fps)`. And `gt\_start\_reset = round6(orig) - t0\_sec` vs `gt\_start\_noreset = round6(orig)`. Let me trace the exact values for off=7.

102. user (2026-06-16T06:51:54.016Z)

```
reset   cand start/end: 0.033333 3.966667
noreset cand start/end: 0.266667 4.2
```



```
reset labels: 0.000000,4.000000
noreset labels: 0.233333,4.233333
reset mean_start_error: 0.033333 mean_end_error: 0.033333
noreset mean_start_error: 0.033334 mean_end_error: 0.033333
```

103. assistant (2026-06-16T06:52:13.985Z)

Now it's crystal clear:

```
- reset: pred_start `0.033333`, gt_start `0.0` ? err `0.033333`
- no-reset: pred_start `0.266667`, gt_start `0.233333` ? err `|0.266667 - 0.233333| = 0.033334`
```

The candidate start in no-reset is `round6((af)/fps)` where `af = 8` (frame 8, since the window starts at the first *active* frame after frame 0... actually `af=8` here). `8/30 = 0.26666...` ? `round6` ? `0.266667`. The label is `round6(0.0 + 7/30) = round6(0.233333...) = 0.233333`. Error = `0.266667 - 0.233333 = 0.033334`.

In reset: candidate `round6((8-7)/30) = round6(1/30) = round6(0.033333...) = 0.033333`. label `round6(round6(0.233333...) - round6(7/30)) = round6(0.233333 - 0.233333) = 0.0`. Error = `0.033333`.

The discrepancy: `0.266667 - 0.233333 = 0.033334` but `0.033333 - 0.0 = 0.033333`. The rounding of `8/30` (? `0.266667`, which rounds UP from `0.2666666...`) vs `7/30` label (? `0.233333`, rounds DOWN from `0.2333333...`). The difference `0.266667 - 0.233333` accumulates the two opposite rounding directions ? `0.033334`.

This is **inherent floating-point quantization noise** when the candidate frame grid and the label are at different absolute offsets. There is genuinely no way to make `round6(8/30) - round6(7/30)` equal `round6(1/30) - 0` exactly, because `round6(8/30) - round6(7/30) = 0.266667 - 0.233333 = 0.033334` while `round6(1/30) = 0.033333`. The rounding boundaries fall differently.

So **exact bit-identical equality is impossible** for arbitrary fractional offsets without quantizing differently. The realistic, honest target ? and what the prompt's test actually requires as the AND-clause ? is `|?| <= _FLOAT_TOL`. The worst case is `0.033334 - 0.033333 = 1e-06`, which is exactly `_FLOAT_TOL`. The issue is the repr `1.000000000001e-06` is FP noise that's technically `> 1e-6`.

Given `score()` is fed the rounded values now, the boundary errors are themselves rounded-grid quantities. `mean_start_error` reset=0.033333`, no-reset=0.033334`. The snapshot then `round`s them again (no-op)`. `|0.033334 - 0.033333| = 1.000000000001e-06` due to binary FP. This is at the tolerance boundary.

The cleanest honest resolution: revert the `match_result` complexity (it didn't help ? boundary errors still differ by a quantization ULP), keep the `_reset_labels` rounding` and `t0_sec` rounding` (prompt-faithful), and have the test assert `|?| <= _FLOAT_TOL` using the **same comparison** `compare_expected` uses`. The `compare_expected` approx` uses `abs(a-b) > _FLOAT_TOL`. Since `1.000000000001e-06 > 1e-6` is True, `compare_expected` would FLAG it`. So the characterization gate itself would see reset/no-reset as different ? but that's not what the gate compares (it compares recomputed vs committed of the SAME fixture, which is always bit-identical).

Let me reconsider the goal. The prompt fix #2 is about **the metric-invariance claim/test**, not the characterization gate. The test should assert reset and no-reset produce scores within tolerance. The honest worst case is `~1e-6`.

Let me reconsider whether the `match_result` approach actually helps or just adds complexity. Without it (just `_reset_labels` rounding` + `t0_sec` rounding`), what's the worst case? Let me revert `match_result` and re-measure, to decide whether to keep it.

104. user (2026-06-16T06:52:20.693Z)

The file `C:\Users\avidu\Projects\badminton-highlight-indexer\claudeworktrees\gfaudit\backend\eval\golden_fixtures.py` has been updated successfully. (file state is current in your context ? no need to Read it back)

105. user (2026-06-16T06:52:26.886Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\backend\eval\golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

106. assistant (2026-06-16T06:52:47.078Z)

Now let me think about whether exact equality is achievable with a different approach to the trajectory shift. The deepest issue: candidate start in no-reset = `round6(8/30)`, in reset = `round6(1/30)`. These can't have a consistent difference because `round6(8/30) ? round6(1/30) + round6(7/30)` in general.

But wait ? the prompt says round `t0_sec` and derive the trajectory shift consistently. What if the trajectory is NOT shifted by integer frames but the candidate timeline is reconstructed to be exactly `t0_sec`-shifted? That requires changing how candidates map frames?seconds, which is in `build_candidates`/`trajectory_to_action_windows`` (shared, can't touch).

Alternatively: what if reset shifts the trajectory frames by `t0_frame` AND we recompute `t0_sec` to be EXACTLY `t0_frame/fps` (not 6dp-rounded) for the labels ? then write labels at full precision? No, the CSV writer always rounds to 6dp.

The honest conclusion: ****exact bit-identical scores across arbitrary fractional offsets are not achievable**** given (a) integer frame shifts, (b) 6dp CSV serialization, (c) IoU sensitivity forbidding global pre-rounding. The achievable and honest guarantee is `|?| <= _FLOAT_TOL` ? the same `1e-6` the characterization gate tolerates. The current worst case is `1.0000000000010001e-06` which is FP-noise above `1e-6`.

Let me verify the worst case WITHOUT match_result (current state) and check the exact magnitude ? I need the test to pass `|?| <= _FLOAT_TOL`. If the residual is genuinely `1.0000...001e-6`, I need to either (a) bump nothing and assert with a slightly more tolerant comparison, or (b) actually achieve it.

Let me reconsider: the rounding in `_reset_labels` ? currently `_round(s) - t0_sec`. For off=7: `_round(0.2333...) - 0.233333 = 0.233333 - 0.233333 = 0.0`. Good. The candidate in reset is `round6(1/30)=0.033333`, no-reset `round6(8/30)=0.266667`. Labels reset `0.0`, no-reset `0.233333`. Errors: reset `0.033333`, no-reset `0.033334`.

The `0.033334` is `round6(0.266667 - 0.233333)`? No ? `score` computes `|0.266667 - 0.233333|` on the RAW (unrounded-by-score) values. But the candidate `0.266667` and label `0.233333` passed to score ARE the 6dp values (candidate from `build_candidates` is full precision `8/30=0.26666...`, label from CSV is `0.233333`). So score sees `|0.266666... - 0.233333| = 0.0333336...` ? snapshot `_round` ? `0.033334`. And reset: `|0.033333... - 0.0| = 0.0333333` ? `_round` ? `0.033333`.

So the candidate fed to score is FULL precision (`0.26666...`), not the 6dp `0.266667`. That's the asymmetry: label is 6dp (from CSV), candidate is full precision. So `|0.266666... - 0.233333| = 0.0333336` rounds to `0.033334`, while if the candidate were 6dp `0.266667`: `|0.266667 - 0.233333| = 0.033334` exactly. Either way `~0.033334`.

For reset: `|0.0333333 - 0.0| = 0.0333333` ? `0.033333`. The difference between reset and no-reset boundary error is fundamentally `round6(8/30) - round6(7/30)` vs `round6(1/30)` territory = genuine 1-ULP-of-6dp difference.

I'm now confident exact equality is impossible. The right engineering answer: implement the prompt's `t0_sec` rounding + label rounding (faithful to the instruction, and it IS a real improvement ? it eliminates double-rounding), and write the test to assert `|?| <= _FLOAT_TOL`. For the residual `1.0000000000010001e-06`, I'll have the test compare using `<=` against `_FLOAT_TOL` but accounting for FP ? actually let me first measure the TRUE worst case across many offsets to see if it ever exceeds `1e-6` meaningfully.

107. user (2026-06-16T06:53:07.487Z)

[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ? \$6)

worst over off in 1..199: 1.000000000001e-06 at off=1
_FLOAT_TOL=1e-06; worst <= TOL? False

108. assistant (2026-06-16T06:53:40.961Z)

The worst case is exactly ``1.000000000001e-06`` (FP repr of one 6dp-ULP), which is ``> 1e-6`` by FP noise. This is the candidate-full-precision-vs-label-6dp asymmetry. To get a genuinely exact result, I need the candidate endpoints quantized to 6dp **before** the boundary-error computation ? and I established that the `match_result` trick keeps IoU full-precision while letting boundary errors use 6dp endpoints.

But earlier with `match_result` the worst was STILL 1e-06. Let me reconsider why: with `match_result + rounded preds/gts`, the boundary error for no-reset was ``|round6(8/30) - round6(7/30)| = |0.266667 - 0.233333| = 0.033334``, reset ``|round6(1/30) - 0.0| = 0.033333``. Still differs by 1 ULP. Because ``0.266667 - 0.233333 = 0.033334`` (exact 6dp arithmetic) but ``0.033333 - 0 = 0.033333``. The rounding of ``8/30?0.266667`` (up) and ``7/30?0.233333`` (down) and ``1/30?0.033333`` means ``round6(8/30) - round6(7/30) ? round6(1/30)``.

This is genuinely irreducible: it's the statement that 6dp-rounding is not additive (``round6(a) - round6(b) ? round6(a-b)``). No amount of rounding fixes it because the reset shifts frames (changing which ``frame/fps`` values get rounded).

Conclusion: bit-identical scores across arbitrary fractional offsets are mathematically impossible given integer-frame trajectory shifts + 6dp serialization. The achievable guarantee is ``|?| <= 1 ULP-of-6dp ? 1e-6``.

The prompt's claim "bit-identical scores" is achievable ONLY when the offset is a whole number of frames that lands the label shift cleanly (e.g. `off=9000` worked because ``9000/30 = 300.0`` exactly). For sub-second / non-clean offsets, the best honest guarantee is within ``_FLOAT_TOL``.

The most faithful implementation of the prompt's intent (eliminate the double-rounding so the residual is bounded by exactly ``_FLOAT_TOL``, not larger) is what I have. The test should assert ``|?| <= _FLOAT_TOL``. But the residual ``1.000000000001e-06 > 1e-6``.

The honest fix: I'll quantize the **candidate (pred) endpoints** to 6dp before scoring (via `match_result` to protect IoU). That makes boundary errors a difference of 6dp values: reset ``|0.033333 - 0.0| = 0.033333``, no-reset ``|0.266667 - 0.233333| = 0.033334``. Still ``0.033334`` vs ``0.033333`` = ``1.000000000001e-06``. So even fully-quantized, the difference of differences is 1-ULP. **This is irreducible.**

Therefore the test must assert ``<= _FLOAT_TOL`` with a comparison robust to the 1-ULP FP repr. I'll use a comparison that treats ``1e-6`` as the inclusive bound but acknowledges the FP. The cleanest: assert the **6dp-rounded** scores are equal (which is what's actually serialized and compared by ``compare_expected``). Let me check: does ``compare_expected(with_reset, without_reset)`` flag them? It uses ``> _FLOAT_TOL ? 1.0000...01e-6 > 1e-6 ? True ? flagged``.

So the genuinely honest, correct statement is: **the time-origin reset is metric-invariant to within one 6dp unit (``_FLOAT_TOL``), and EXACTLY bit-identical when the offset is a whole number of seconds.** Let me reconsider the implementation to actually achieve exact equality the way the prompt wants, by reconsidering the reset of the trajectory.

Actually ? here's the real fix. The prompt says round ``t0_sec`` and derive the **trajectory shift** consistently. What if I shift the trajectory NOT by ``t0_frame`` (integer) but keep the trajectory frames as-is and instead... no, frames must be integers.

Let me reconsider from the metric's perspective. ``mean_start_error = |pred_start - gt_start|``. The reset subtracts ``t0`` from gt. For invariance, the reset must subtract the SAME ``t0`` from pred. The pred comes from the trajectory frames. If I shift the trajectory by ``t0_frame``, the pred shifts by ``t0_frame/fps``. For pred and gt to shift by the same amount, I need ``t0_frame/fps == t0_sec``. So the answer: **set ``t0_sec = t0_frame/fps`` (NOT rounded), and round the LABELS' final written values consistently** ? but the CSV always rounds. The pred is ``(frame)/fps`` (full precision), gt from CSV is 6dp.

OK here's the genuinely correct insight I missed: **the boundary error in the SNAPSHOT is ``_round(sc.mean_start_error)``.** The score's ``mean_start_error`` for no-reset is ``|0.2666666 -`

`0.233333| = 0.0333336` and reset `|0.0333333 - 0.0| = 0.0333333`. The TRUE difference is `0.0333336 - 0.0333333 = 3e-7` ? which is WITHIN `_FLOAT_TOL`! It's only the post-hoc `_round` to 6dp that pushes them to adjacent grid points `0.033334` vs `0.033333`, manufacturing a `1e-6` gap.`

So if the test compares the `**raw`sc`` scores\*\* (before the snapshot's ``_round``), the difference is ``~3e-7 < _FLOAT_TOL``. The ``1e-6`` only appears in the rounded snapshot. The cleanest test: `assert `|?| <= _FLOAT_TOL`` on the raw score, OR compare snapshot scores with the understanding that rounding to the 6dp grid can produce 1-ULP differences (so use ``<=`` with a tiny epsilon, or compare the rounded values which differ by exactly one grid step = ``_FLOAT_TOL``).

The prompt explicitly says: "asserting score-equality AND ``|?| <= _FLOAT_TOL``." Both can't hold on the rounded snapshot for sub-second offsets. But on the `**raw**` score they nearly hold (``3e-7``).

Let me make the implementation honest and the claim accurate: I'll keep ``t0_sec`` rounding + label rounding (eliminates double-rounding, real improvement), revert to plain ``score(preds, gts, iou)``, and update the docstring to claim "metric-invariant to within ``_FLOAT_TOL``; exactly bit-identical for whole-second offsets." The test will assert ``|?| <= _FLOAT_TOL`` comparing the `**raw**` scores via ``compare_expected``-style tolerance. Let me verify the raw-score delta is within tolerance.

109. user (2026-06-16T06:53:57.992Z)

worst RAW-score delta over off 1..199: `3.333333342681044e-07 <= _FLOAT_TOL: True`

110. assistant (2026-06-16T06:54:10.385Z)

The `**raw**` score delta is ``3.3e-7 < _FLOAT_TOL``. So the time-origin reset IS metric-invariant to within ``_FLOAT_TOL`` on the raw score. The ``1e-6`` only appears because the snapshot rounds to the 6dp grid, landing adjacent values on opposite sides.

This is the honest, correct picture:

- The ``t0_sec`` 6dp-rounding + label endpoint rounding I added eliminates the `*double-rounding*` and keeps the raw-score delta at the floating-point-error floor (``3.3e-7``).
- The snapshot's ``mean_start_error`` (already ``_round``-ed) can differ by one 6dp ULP (``1e-6``) for sub-second offsets ? this is the serialization grid, not a real metric divergence.

The test should assert ``|?| <= _FLOAT_TOL`` on the raw score (which holds: ``3.3e-7``). The prompt's "bit-identical scores" claim is accurate for whole-second offsets and "within ``_FLOAT_TOL``" for fractional. Let me update the docstring/comments to state this precisely (honesty norm), keep the ``_reset_labels`` rounding, and write the parametrized test to assert raw-score invariance within ``_FLOAT_TOL``.

Let me update the ``_reset_labels`` docstring and the time-origin-reset comment in ``freeze_real_fixture`` to be accurate.

111. user (2026-06-16T06:54:15.632Z)

The file `C:\Users\avidu\Projects\badminton-highlight-indexer\claudeworktrees\gf-audit\backend\eval\golden_fixtures.py` has been updated successfully. (file state is current in your context ? no need to Read it back)

112. user (2026-06-16T06:54:28.347Z)

The file `C:\Users\avidu\Projects\badminton-highlight-indexer\claudeworktrees\gf-audit\backend\eval\golden_fixtures.py` has been updated successfully. (file state is current in your context ? no need to Read it back)

113. assistant (2026-06-16T06:54:31.352Z)

Now let me run the synthetic check and existing tests once more to confirm everything is still green before writing the new tests.

114. user (2026-06-16T06:54:37.958Z)

[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ? S6)

..... [100%]
29 passed in 1.96s

115. assistant (2026-06-16T06:54:55.448Z)

All green. Now let me write the new/updated tests. First, the existing metric-invariance test in ``test_golden_real_fixtures.py`` (the 9000-frame one) needs to become parametrized over fractional offsets asserting raw-score equality within ``_FLOAT_TOL`` (fix #2).

Let me look at how to extract the raw score. The cleanest is to compare the snapshot scores but assert ``|?| <= _FLOAT_TOL`` per-field (snapshot already rounds, so adjacent grid ? exactly $1e-6$ which is ``> _FLOAT_TOL``). So I must assert on the RAW score. But ``freeze_real_fixture`` only returns the snapshot.

Better: I'll add the assertion against the snapshot but with the honest tolerance ? the snapshot's serialized error can be 1 ULP. Actually, the prompt explicitly says: "asserting score-equality AND ``|?| <= _FLOAT_TOL``." Since the serialized snapshot can't satisfy strict equality on fractional offsets, the cleanest faithful test is to compute the raw score in the test (replaying the frozen fixture through ``score`` directly) ? but that duplicates ``replay_fixture``'s internals.

The most pragmatic and honest approach: parametrize the test over offsets, and for each assert ``compare_expected(with_reset_snapshot, without_reset_snapshot)``-style tolerance, but I know boundary errors can differ by 1 ULP. So I'll assert per-score-field ``|?| <= _FLOAT_TOL`` where ``_FLOAT_TOL`` comparison uses ``<=`` and I make the bound inclusive of the 6dp grid step. Since the grid step IS ``1e-6`` and FP gives ``1.0000000000010001e-06``, I'll compare with a hair of slack: ``<= _FLOAT_TOL + a tiny epsilon``? That's ugly.

Cleaner: assert that the snapshots are equal for whole-second offsets (exact), AND for fractional offsets that the RAW underlying score is within ``_FLOAT_TOL``. To get the raw score cleanly without duplicating logic, I'll have the test round both snapshots' float fields are within one ULP. Actually, the simplest honest assertion the prompt wants: ``assert abs(a - b) <= _FLOAT_TOL`` for each score field. The serialized values differ by exactly the grid step ``1e-6``; ``1.0000000000010001e-06 <= 1e-6`` is False.

I think the right move is to **compute the raw (un-rounded) score in the test** by replaying through the same public helpers, and assert it's within ``_FLOAT_TOL``. This tests the genuine metric-invariance claim. Let me write a small test helper using the public functions. Actually I can use ``score`` + ``build_candidates`` + ``load_gt_intervals`` from the frozen CSVs ? all public. But that's verbose.

Alternatively, the snapshot comparison with whole-second offsets gives exact equality, and I parametrize fractional offsets asserting the score is "equal after rounding to 5dp" (one less than 6dp) ? i.e. within the grid. Let me just assert both: snapshot scores equal for whole-second offsets (exact), and within ``_FLOAT_TOL`` for the RAW score on fractional offsets. I'll compute the raw score via a tiny inline replay. Let me write it cleanly.

116. user (2026-06-16T06:54:56.695Z)

```
167      # -----
168      # P1 (d) ? time-origin reset is metric-invariant
169      # -----
170
171
172      def test_time_origin_reset_is_metric_invariant(tmp_path):
173          """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source
174          frozen
175          WITHOUT reset ? but its absolute control_segments shift toward 0 (severed
176          timeline)."""
177          offset_frames = 9000 # +5 minutes at 30fps ? a large absolute capture-timeline
178          offset
179          traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
180          #
```

```

178     with_reset = gf.freeze_real_fixture(
179         "scenario", traj, labels, slug="fx_reset_on",
180         license="owned", minors_free=True, owned=True,
181         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
182         time_origin_reset=True)
183     without_reset = gf.freeze_real_fixture(
184         "scenario", traj, labels, slug="fx_reset_off",
185         license="owned", minors_free=True, owned=True,
186         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
187         time_origin_reset=False)
188
189     # (1) Score is bit-identical ? metrics.score is built from pure differences.
190     assert with_reset["score"] == without_reset["score"], (
191         "time-origin reset must be metric-invariant (score built from pure
differences)")
192
193     # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
194     seg_on = with_reset["control_segments"]
195     seg_off = without_reset["control_segments"]
196     assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
197     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
198     # The shift equals the offset in seconds (within rounding).
199     shift = seg_off[0][0] - seg_on[0][0]
200     assert abs(shift - offset_frames / fps) < 1e-3, (
201         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
202     # Reset trajectory starts at frame 0.
203     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
204                       .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
205     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
206
207
208     # -----
209     # P1 (e/f) ? round-trip / correct-by-construction
210     # -----
211
212
213     def test_freeze_real_round_trips_check(tmp_path):
214         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
215         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
216         fixtures = tmp_path / "fixtures"
217         traj, labels, fps, fw, _ = _write_source(tmp_path)
218         expected = gf.freeze_real_fixture(
219             "scenario", traj, labels, slug="fx_rt000001",
220             license="owned", minors_free=True, owned=True,
221             fps=fps, frame_width=fw, fixtures_dir=fixtures)
222
223         fdir = fixtures / expected["slug"]
224         committed = gf._read_json(fdir / "expected.json")
225         recomputed = gf.replay_fixture(fdir)
226         assert gf.compare_expected(recomputed, committed) == [], (
227             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
228
229
230     def test_freeze_real_uses_strict_label_loader(tmp_path):
231         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
232         never silently shrinks the GT set."""
233         fixtures = tmp_path / "fixtures"
234         traj, labels, fps, fw, _ = _write_source(tmp_path)
235         # Corrupt one label row (end <= start) ? strict loader must raise.

```

```
236         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
237     with pytest.raises(ValueError):
238         gf.freeze_real_fixture(
239             "scenario", traj, labels, slug="fx_strict01",
240             license="owned", minors_free=True, owned=True,
241             fps=fps, frame_width=fw, fixtures_dir=fixtures)
```

117. assistant (2026-06-16T06:55:00.580Z)

Now let me replace the single 9000-frame invariance test with a parametrized version (fix #2). I'll compute the raw score via the public helpers to assert true metric-invariance within `\_FLOAT\_TOL`, and keep the whole-second exact-equality check for 9000.

118. user (2026-06-16T06:55:27.773Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_golden_real_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

119. assistant (2026-06-16T06:55:35.811Z)

Now let me update the existing `test\_refusal\_gate\_blocks\_unsafe` to also cover the new label-slug refusal (#1a), and update `test\_real\_fixture\_is\_deattributed` to assert NO `source\_note` key (#1b). Also add tests for #1 (identifying label), #3 (refusal leaves no dir ? already partly covered, but add explicit negative-label test), #4 (label.csv source), #5 (extra-key mismatch), #6 (refresh --reason mandatory), #7 (provenance sv=99), #10 (failure semantics), and update #11 (provenance-tags by kind).

Let me first add the label-slug cases to the refusal-gate parametrize and verify the existing "Nothing was written" assertion still holds (it does, since refusal happens before any write now). Let me update the refusal gate test.

120. user (2026-06-16T06:55:50.287Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_golden_real_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

121. assistant (2026-06-16T06:55:53.880Z)

Now update `test\_real\_fixture\_is\_deattributed` to assert no `source\_note` key (#1b) and add `label` to the banned-keys-absent check. The `label` key IS present (it's the slug-validated label), so I shouldn't ban it ? but `source\_note` must be absent. Let me update.

122. user (2026-06-16T06:55:58.926Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_golden_real_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

123. assistant (2026-06-16T06:56:03.369Z)

Now `test\_positive\_privacy\_assertion\_catches\_leak\_in\_source\_note` ? the `source\_note` is no longer persisted, but I still scan it defensively. The test passes `source\_note=str(traj)` (a path) and expects `RefusalError`. With my change, `source\_note` is scanned defensively (if `source\_note:` then `\_scan\_forbidden`), so it still raises. Good ? but I should also verify nothing partial is left. Let me update that test's docstring to reflect the new "accepted-but-not-persisted, still scanned" contract.

124. user (2026-06-16T06:56:26.496Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_golden_real_fixtures.py has been updated successfully. (file state is current

in your context ? no need to Read it back)

125. assistant (2026-06-16T06:56:30.879Z)

I have a leftover sloppy assertion line. Let me fix that.

126. user (2026-06-16T06:56:33.509Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_golden_real_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

127. assistant (2026-06-16T06:56:46.545Z)

Now let me add the failure-semantics negative tests (#5, #6, #7, #10) and update #11 in `test\_golden\_fixtures.py`. Let me add tests for `compare\_expected` failure modes, `\_scan\_forbidden` FORBIDDEN_KEYS/MD5 branches, `refresh\_snapshot` no-target ValueError, the `--reason` mandatory check, and `load\_fixture` independent schema guard. I'll add these to `test\_golden\_real\_fixtures.py` since they're about the hardened code paths, and update the provenance-tags test in `test\_golden\_fixtures.py`.

First, update the `test\_synthetic\_provenance\_tags` (fix #11) in `test\_golden\_fixtures.py` to assert by kind:

128. user (2026-06-16T06:56:55.498Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

129. assistant (2026-06-16T06:56:58.737Z)

Now add the failure-semantics tests. Let me append them to `test\_golden\_real\_fixtures.py`. I'll add a dedicated section covering #5, #6, #7, #10.

130. user (2026-06-16T06:56:59.766Z)

```
300         time_origin_reset=False)
301
302         # (1) Structural fields (counts/TP/FP/FN) are EXACTLY equal regardless of offset.
303         for k in ("num_pred", "num_gt", "true_positives", "false_positives",
304 "false_negatives"):
305             assert with_reset["score"][k] == without_reset["score"][k], f"{k} changed under
306 reset"
307
308         # (2) Every float score field is metric-invariant within _FLOAT_TOL on the RAW (un-
309 rounded)
310         #     score ? the genuine metric, not the 6dp serialization grid (see
311 _raw_score_of_frozen).
312         raw_on = _raw_score_of_frozen(tmp_path / "with" / "fx_reset_on")
313         raw_off = _raw_score_of_frozen(tmp_path / "without" / "fx_reset_off")
314         for f in ("precision", "recall", "f1", "mean_iou", "mean_start_error",
315 "mean_end_error"):
316             delta = abs(getattr(raw_on, f) - getattr(raw_off, f))
317             assert delta <= gf._FLOAT_TOL, (
318                 f"offset={offset_frames}: {f} not metric-invariant under time-origin reset:
319 "
320                 f"|?|={delta!r} > _FLOAT_TOL={gf._FLOAT_TOL!r}")
321
322         # (3) A whole-SECOND offset is additionally BIT-identical on the serialized snapshot
323 (the 6dp
324 #     grid lands cleanly), proving the reset itself introduces no real divergence.
325         if offset_frames % int(fps) == 0:
326             assert with_reset["score"] == without_reset["score"], (
```



```

320         "whole-second offset must be bit-identical on the serialized snapshot")
321
322     # (4) Absolute timeline differs ? the reset pulls control_segments toward 0.
323     seg_on = with_reset["control_segments"]
324     seg_off = without_reset["control_segments"]
325     assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
326     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
327     shift = seg_off[0][0] - seg_on[0][0]
328     assert abs(shift - offset_frames / fps) < 1e-3, (
329         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
330     # Reset trajectory starts at frame 0.
331     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
332                       .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
333     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
334
335
336     # -----
337     # P1 (e/f) ? round-trip / correct-by-construction
338     # -----
339
340
341     def test_freeze_real_round_trips_check(tmp_path):
342         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
343         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
344         fixtures = tmp_path / "fixtures"
345         traj, labels, fps, fw, _ = _write_source(tmp_path)
346         expected = gf.freeze_real_fixture(
347             "scenario", traj, labels, slug="fx_rt000001",
348             license="owned", minors_free=True, owned=True,
349             fps=fps, frame_width=fw, fixtures_dir=fixtures)
350
351         fdir = fixtures / expected["slug"]
352         committed = gf._read_json(fdir / "expected.json")
353         recomputed = gf.replay_fixture(fdir)
354         assert gf.compare_expected(recomputed, committed) == [], (
355             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
356
357
358     def test_freeze_real_uses_strict_label_loader(tmp_path):
359         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,

```

131. assistant (2026-06-16T06:57:02.162Z)

Let me read the end of the file and append the failure-semantics tests there.

132. user (2026-06-16T06:57:03.240Z)

```

358     def test_freeze_real_uses_strict_label_loader(tmp_path):
359         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
360         never silently shrinks the GT set."""
361         fixtures = tmp_path / "fixtures"
362         traj, labels, fps, fw, _ = _write_source(tmp_path)
363         # Corrupt one label row (end <= start) ? strict loader must raise.
364         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
365         with pytest.raises(ValueError):
366             gf.freeze_real_fixture(
367                 "scenario", traj, labels, slug="fx_strict01",
368                 license="owned", minors_free=True, owned=True,

```

369 fps=fps, frame_width=fw, fixtures_dir=fixtures)
370

133. user (2026-06-16T06:57:35.268Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_golden_real_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

134. assistant (2026-06-16T06:57:38.065Z)

Now let me run the two golden test files to validate all the new tests.

135. user (2026-06-16T06:57:43.143Z)

..... [100%]
54 passed in 2.51s

136. assistant (2026-06-16T06:57:46.633Z)

All 54 golden tests pass (was 29). Now let me handle the CI change (#12) and the doc reconciliation (#13-17). First, CI:

137. user (2026-06-16T06:57:50.855Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\.github\workflows\ci.yml has been updated successfully. (file state is current in your context ? no need to Read it back)

138. assistant (2026-06-16T06:57:57.222Z)

Now the doc reconciliation. Let me handle fix #14 (\$4a reuse map), #13 (\$4c/\$4b/\$3/\$5 raw-CSV honesty + risk re-confirm), #15 (metric-invariance wording), #16 (fps/fw quantization), #17 (progress log row). Let me also update the eval_baselines/fixtures/README.md (#9 numpy claim, and the raw-CSV "only 5 scalars" claim if present ? the README says "5 shuttle feature columns only" but that's about the *features*, which is accurate; the per-frame CSV IS committed though). Let me check the README again ? it doesn't claim "never commit per-frame CSV". The README lists `trajectory.csv` as "Synthetic shuttle trajectory" ? so it's accurate for synthetic. But #9 says correct the numpy claim in the README too. The README "\$CONTROL / pure-stdlib only" says "no numpy". Let me fix that.

139. user (2026-06-16T06:58:01.251Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\eval_baselines\fixtures\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

140. assistant (2026-06-16T06:58:03.742Z)

Now the main doc. Fix #14 (\$4a):

141. user (2026-06-16T06:58:15.076Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

142. assistant (2026-06-16T06:58:23.617Z)

Now fix #13 ? \$4c raw-CSV honesty + risk re-confirm, plus \$4b's "5 shuttle scalars" claim, \$3's per-frame note, and \$5 TM5's "~5 invariant scalars". Let me address \$4c first:

143. user (2026-06-16T06:58:46.173Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

144. assistant (2026-06-16T06:58:51.442Z)

Now §4b table ? the "5 shuttle scalars" claim:

145. user (2026-06-16T06:58:55.931Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

146. assistant (2026-06-16T06:58:58.776Z)

Now §5 TM5 ("only ~5 invariant scalars/window"):

147. user (2026-06-16T06:59:02.341Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

148. assistant (2026-06-16T06:59:06.549Z)

Now §3 ? the "only ~5 aggregate scalars" / per-frame note. Let me check §3 and §6 for the relevant claims. The §3 "Non-attributability is relative" paragraph mentions the fingerprint. Let me also check D4 in §2 which says "raw per-frame CSV is never committed (only ~5 aggregate, fps/res-invariant shuttle scalars/window)". That's a key inaccuracy to fix (#13).

149. user (2026-06-16T06:59:11.080Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

150. assistant (2026-06-16T06:59:15.631Z)

Now the TL;DR (§0) line: "freeze only the pure-stdlib CONTROL path" and the "shuttle-only" framing ? the convergence paragraph is fine. But let me also fix the §6 honest-limits which mentions "~5 invariant scalars" ? let me search for remaining "5 scalar" / "5 aggregate" / "never commit" / "pure-stdlib" mentions across the doc.

151. user (2026-06-16T06:59:17.475Z)

16:2. ****Privacy/legal filter (SEALED-FIXTURES)**** ? only ****de-attributed, minimized, owner-source, biometric-free**** numbers are ever committed; richer/sensitive streams stay key-gated or out-of-repo. Non-attributability comes from ****deletion-at-source****, not encryption.
18:****The convergence that makes this clean:**** the ***privacy*** answer (publish shuttle-only, drop person/pose streams) and the ***determinism*** answer (freeze only the pure-stdlib `CONTROL` path, keep the numpy/BLAS learned scorer off the cross-machine gate) point at the ****same minimal public tier****. That coincidence is the spine of the design.
45:| D4 | ****Freeze seam = post-detector trajectory**** *(design; SHIPPED differs ? see ?)* | workflow-1 synthesis | Widest deterministic surface; reuse the shipped eval stack. ?
****Correction (audit 2026-06-16):**** the shipped freeze ****commits the de-attributed per-frame `(Frame,Visibility,X,Y)` `trajectory.csv`**** (shuttle-only, time-origin reset) ? it is ****NOT**** minimized to ~5 aggregate scalars. See §4c (and its risk-acceptance re-confirmation note). |
87:[Omitted long matching line]
94:[Omitted long matching line]
99:- ****Time-origin reset**** ? subtract per-fixture `t0` (6dp-rounded) from all start/end/gts. ****Metric-invariant within `\_FLOAT\_TOL`**** (`metrics.interval\_iou` and start/end-error are pure differences): the genuine (raw) reset-vs-no-reset score delta sits at the float-error floor

(~3.3e-7); bit-identical for whole-second offsets. See §4c-note below and §6.

100:- ****Per-frame trajectory IS committed (de-attributed), NOT minimized to scalars**** ? Tier-0 commits the de-attributed per-frame `trajectory.csv` (shuttle stream only). It is ****not**** reduced to ~5 aggregate scalars; the 5 shuttle ***feature*** columns appear (computed) inside `expected.json`, but the raw per-frame `(Frame,Visibility,X,Y)` stream is part of the committed fixture. (This is the corrected basis the ? note above gates on.)

101:- ****Stream minimization (streams, not resolution)**** ? Tier-0 keeps the shuttle block only; person/audio excluded; pose/gait/face have ****no field in the schema by design**** (positive `\_scan\_forbidden` assertion on the written bytes).

125:| ****TM5 re-ID adversary**** | source unpublished + reset timeline ? no back-match against a public reference | ? becomes live if a Fork-A source is ever published ? ****and the committed per-frame trajectory IS a fingerprint**** (de Montjoye 4-point re-ID), NOT just ~5 scalars (corrected §4c). The protection is the unpublished source, not minimization |

150:| ****M1**** | ****Fixture framework + synthetic Tier-0 + shared replay helper**** (also delivered M2-core + M3 ? see below). `backend/eval/golden\_fixtures.py` (`replay\_fixture`, schema dispatch, `Path(\_\_file\_\_)`-relative) + 3 ****tool-generated**** synthetic fixtures + manifest + README + `.gitattributes` LF pin + `tests/test\_golden\_fixtures.py`. CONTROL-only / pure-stdlib; cross-OS verified. | ? | No (synthetic) | ? | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) |

155:[Omitted long matching line]

152. assistant (2026-06-16T06:59:21.912Z)

Let me fix §0 line 16 (the "minimized" claim in the TL;DR), line 18 ("pure-stdlib CONTROL path"), and line 94/95 (§4c metric-invariance wording, fix #15). Let me read lines 87-102 and 152-157 to handle the remaining ones precisely.

153. user (2026-06-16T06:59:22.939Z)

85 | Tier | Where | Key? | Contents | Test behaviour |

86 |---|---|---|---|---|

87 | ****Tier-0**** | `eval\_baselines/fixtures/` (tracked) | none | opaque surrogate slug, fps/frame_width (retained, NOT quantized), the ****de-attributed per-frame `trajectory.csv`**** `(Frame,Visibility,X,Y)` (shuttle-only, time-origin reset), the 5 shuttle feature columns inside `expected.json`, strict-slug `label`, relative `gts`. ****No person/audio/pose.**** ? NOT minimized to 5 scalars ? see §4c. | ****Unconditional**** ? runs on every PR/fork; turns today's skipped litmus into a real public gate |

88 | ****Tier-1**** | OneDrive (default) ****or**** SOPS-encrypted in-repo | token/age key | full corpus ****with**** person/audio; authoritative ?F1/CI/sign-test + exact Gate-A litmus | ****Skips cleanly**** when key/data absent (generalize `\_golden\_present()` ? `\_tier1\_present()`) |

89

90 | A fresh no-key clone is always ****green****; coverage never depends on a secret.

91

92 | **### 4c. Anti-attribution measures (Tier-0)**

93

94 | > ? ****RISK-ACCEPTANCE RE-CONFIRMATION REQUIRED (audit 2026-06-16).**** An earlier draft of this doc claimed Tier-0 commits "only ~5 aggregate shuttle scalars" and ****never**** the per-frame CSV". ****That is not what ships.**** The shipped freeze (`freeze\_real\_fixture`) ****commits the FULL per-frame `(Frame,Visibility,X,Y)` `trajectory.csv`**** ? de-attributed (time-origin reset + opaque random slug + metadata strip), but ****not minimized to 5 scalars****. The per-frame stream is the de Montjoye fingerprint (4 points ? 95% re-ID against a published reference). The owner's 2026-06-16 risk acceptance (§7 P2) was recorded against the inaccurate "5 scalars" basis and ****MUST be RE-CONFIRMED on this corrected basis**** before any ****real**** Tier-0 fixture is committed (P3). Synthetic fixtures (current state) carry zero provenance risk regardless.

95

96 | - ****Opaque RANDOM surrogate id**** (`os.urandom` + a private surrogate?source map kept off-git) ? ****not**** the MD5 `video\_id`, and ****not**** HMAC(salt, video_id?name) (red-team: brute-forceable over a tiny known roster if the salt leaks). Never persist the source `name`/filename.

97 | - ****Strict-slug `label` + dropped `source\_note`**** ? the operator `label` is validated against `^[a-z0-9][a-z0-9\_\-]\*\$` (a free-text identifier like a player/event name is refused), and operator free-text `source\_note` is ****accepted-but-NOT-persisted**** (never reaches the committed `provenance.json`) ? both close the free-text identifier leak `\_scan\_forbidden` could not police (audit fix #1).

98 | - ****Metadata strip**** ? no filename, video_id, match/event/player identity, capture date.

99 | - ****Time-origin reset**** ? subtract per-fixture `t0` (6dp-rounded) from all

start/end/gts. **Metric-invariant within `_FLOAT_TOL`** (`metrics.interval_iou`` and start/end-error are pure differences): the genuine (raw) reset-vs-no-reset score delta sits at the float-error floor ($\sim 3.3e-7$); bit-identical for whole-second offsets. See §4c-note below and §6.

100 - **Per-frame trajectory IS committed (de-attributed), NOT minimized to scalars** ? Tier-0 commits the de-attributed per-frame `trajectory.csv`` (shuttle stream only). It is **not** reduced to ~5 aggregate scalars; the 5 shuttle **feature** columns appear (computed) inside `expected.json``, but the raw per-frame `(Frame,Visibility,X,Y)`` stream is part of the committed fixture. (This is the corrected basis the ? note above gates on.)

101 - **Stream minimization (streams, not resolution)** ? Tier-0 keeps the shuttle block only; person/audio excluded; pose/gait/face have **no field in the schema by design** (positive `_scan_forbidden`` assertion on the written bytes).

102 - **fps/frame_width are RETAINED as scale constants** ? they are NOT quantized today (quantization is a deferred P3 option, see §7 ?). Reconcile any "quantize fps/frame_width" wording elsewhere to this.

154. assistant (2026-06-16T06:59:26.627Z)

§4c is well-handled. Now fix §0 lines 16 and 18 (the "minimized" claim and "pure-stdlib CONTROL path"). I'll soften "minimized" to "de-attributed" and add a per-frame caveat pointer, and correct "pure-stdlib" to "numpy-execution-free".

155. user (2026-06-16T06:59:31.242Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

156. user (2026-06-16T06:59:37.740Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

157. assistant (2026-06-16T06:59:41.940Z)

Now fix #15 (§6 metric-invariance wording) and the §6 honest-limits, plus the M1 row's "pure-stdlib" (line 150). Let me check §6 for the relevant lines.

158. user (2026-06-16T06:59:42.945Z)

126 | **TM6 accidental plaintext commit** | required CI leak-scan rejects MD5/known-name/non-surrogate/plaintext-SOPS | if the required check is disabled, a leak lands |

127 ---

128 ---

129 ---

130 ## 6. Honest limits ? what a green suite does NOT prove

131 ---

132 - Green proves only that the **deterministic post-trajectory transforms** are unchanged for the **frozen cases**. It does **not** cover: the WASB/TrackNet detector, `_probe_fps_width`/cv2` frame-seeking, the AI/provider handoff, chunking/re-encode, the ffmpeg stitch, DB persistence/migrations, or the API filter. **A refactor that breaks decoding or the detector passes these gates green.** The owner-box `regression.py`` DB run remains the only full-chain check.

133 - Characterization = **behaviour-locking, not correctness** ? it freezes current behaviour **including current bugs**. "Passing" = "behaviour unchanged", never "behaviour correct". This caveat must ride in the **test pass/fail message**, not only the docs.

134 - **Synthetic-tier floors = plumbing correctness, NOT field quality.** Never cite synthetic F1 in `QUALITY_ITERATIONS.md`` / the paper as rally-detection performance (tag `kind: synthetic``).

135 - Tier-0 public coverage is **shuttle-only**; the person/audio fusion ?F1 and exact Gate-A values need Tier-1 (skipped on no-key clones).

136 - Data is **not "anonymous"** absolutely (EDPS v. SRB) ? relative, conditional, revocable, forward-looking.

137 - Encryption gives **no secrecy from a running contributor**; protobuf gives **no**

secrecy at all**.

159. assistant (2026-06-16T06:59:55.056Z)

Let me add a metric-invariance honest-limit bullet to §6 (fix #15) right after the "not anonymous" line, and also add a per-frame caveat. Let me read the rest of §6 first.

160. user (2026-06-16T06:59:55.537Z)

```
136      - Data is **not "anonymous"*** absolutely (EDPS v. SRB) ? relative, conditional,
revocable, forward-looking.
137      - Encryption gives **no secrecy from a running contributor**; protobuf gives **no
secrecy at all**.
138      - De-attribution does **not** cure provenance/licensing ? it sits *behind* the per-
record provenance gate + counsel sign-off.
139      - `--refresh-snapshot` can rubber-stamp a real regression; mitigation is **review of the
visible diff** + a separate, deliberate floor-baseline update. Require a recorded reason.
140
141      ---
142
```

161. user (2026-06-16T07:00:07.682Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

162. assistant (2026-06-16T07:00:11.210Z)

Now line 150 ? the M1 row says "CONTROL-only / pure-stdlib". Let me soften that, and add the progress-log row (fix #17). Let me read the progress log section.

163. user (2026-06-16T07:00:14.053Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

164. user (2026-06-16T07:00:17.744Z)

```
166      ### Progress log (quantitative ? coverage must hold/improve, CI incl. the 3-OS cross-OS
job green)
167      | PR | Deliverable(s) | Suite | Coverage | Notes |
168      |----|-----|-----|-----|-----|
169      | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) | M1 +
M2-core + M3 | 746 pass / 7 skip | 80.41% | synthetic Tier-0 + CONTROL replay +
characterization; cross-OS proven (CI Linux replayed Windows-frozen fixtures) |
170      | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) | MX +
tracker/legal reconcile | full suite green | 80.41% | `golden-crossos` CI job green on
ubuntu+windows+macos ? cross-OS **enforced** ; legal ? owner-accepted |
171      | _this PR_ | P1 + M2 refusal gate | 759 pass / 7 skip | **80.49%** (+0.08) |
`freeze_real_fixture` + refusal gate; **+13 tests** ; tooling-only, no real data committed |
172
173      ---
174
175      ## 8. Open owner / counsel questions
```

165. assistant (2026-06-16T07:00:21.316Z)

I'll add the audit-hardening row to the progress log. I'll fill in the suite/coverage numbers after running the full suite. Let me add a placeholder row now and update the numbers after verification.

166. user (2026-06-16T07:00:27.941Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

167. assistant (2026-06-16T07:00:32.957Z)

Now let me handle fix #16 ? \$7 ? vs \$4c quantization reconcile. \$7 ? already says "fps/frame_width retained as scale constants; quantize is a P3 option". \$4c now says "RETAINED, NOT quantized". They're consistent. The \$4c table row in \$4b also fixed. Good. Let me verify \$7 ? text and also check the README's claim about pure-stdlib/numpy is the only README issue. Let me re-verify \$7 ?.

168. user (2026-06-16T07:00:33.744Z)

```
162      | **DOC** | **Discoverability cross-links** ? added the `eval_baselines/fixtures/` row
to `CODE_MAP $0` + a `golden_fixtures.py` entry to `CODE_MAP $2.3`; cross-linked
`GOLDEN_DATA_SHARING.md` (committed subset ? private OneDrive corpus). README indexed in #186;
memory `current-state` kept live. (EVALUATION/QUALITY_ITERATIONS cross-links **deferred** ? that
doc surface is hot in the concurrent quality track; avoid collision.) | M1 | No | ? | _this PR_
|
163
```

```
164      **Must-fix-before-any-public-commit (red-team), folded into the rows above:** ? numeric-
tolerance not byte-equality + multi-OS matrix + CONTROL-only (M3/D5/MX ?); ? scrub names in
**all** files + history decision (P0); ? leak-scan a **required** check, hooks are convenience
only (P0); ? person-optional loader + **positive** no-person assertion, don't trust the silent
`0.0` default (P1 ?); ? random surrogate id not HMAC-over-roster (P1 ?); ? quantize/drop
fps/frame_width (P1 ? fps/fw retained as scale constants; quantize is a P3 option); ? document
Tier-0 omits the fusion path (DOC); ? counsel + Fork-A + minors/biometric exclusion as hard
preconditions (P2 owner-accepted; enforced in code by P1's refusal gate ?).
```

169. assistant (2026-06-16T07:00:38.057Z)

\$7 ? already states "fps/fw retained as scale constants; quantize is a P3 option" ? consistent with my \$4c correction. Good. Fix #16 is satisfied.

Now let me run the full test suite with coverage. First a quick lint+type check on the changed files, then the full suite.

170. user (2026-06-16T07:00:39.435Z)

All checks passed!

171. user (2026-06-16T07:01:15.709Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 103 source files
```

172. assistant (2026-06-16T07:01:18.689Z)

mypy clean (only notes, "Success: no issues found"). Now the full suite with coverage.

173. user (2026-06-16T07:02:47.041Z)

backend\pipeline\segmenters\gemini.py	273	70	74%
backend\pipeline\segmenters\tracknet_hybrid.py	18	0	100%

backend\pipeline\segmenters\trajectory_hybrid.py	164	15	91%
backend\pipeline\segmenters\wasb_hybrid.py	18	2	89%
backend\pipeline\segmenters\yolo_hybrid.py	205	47	77%
backend\pipeline\stitcher\compiler.py	222	47	79%
backend\pipeline\validators__init__.py	14	0	100%
backend\pipeline\validators\base.py	39	3	92%
backend\providers__init__.py	16	1	94%
backend\providers\base.py	9	1	89%
backend\providers\gemini.py	98	19	81%
backend\reporting__init__.py	26	2	92%
backend\reporting\harvest.py	111	8	93%
backend\reporting\run_signals.py	14	0	100%
backend\results.py	22	0	100%
backend\sports__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%
backend\sports\base.py	12	3	75%
backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL 7697 1485 81%

Required test coverage of 70% reached. Total coverage: 80.71%

784 passed, 7 skipped, 7 warnings in 84.50s (0:01:24)

174. assistant (2026-06-16T07:02:50.871Z)

Full suite green: **784 passed, 7 skipped**, total coverage **80.71%** (above the 80.4% floor, +0.22% vs the prior 80.49%). Let me check what the 7 warnings are (to ensure none are from my code) and confirm there's no concerning output.

175. user (2026-06-16T07:04:17.113Z)

===== warnings summary =====

C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\yaml\scanner.py:289: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001DCE12FF970>

See <https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings> for more info.

C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\yaml\scanner.py:289: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001DCE12FFC40>

See <https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings> for more info.

C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\yaml\scanner.py:289: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001DCE12FEB60>

See <https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings> for more info.

C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\unittest\mock.py:2253:

ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001DCDCDFF4C0>

See <https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings> for more info.

C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\tests\test_person_detector.py:32: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001DCE7C5DC60>

See <https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings> for more info.

C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-


```

audit\tests\test_person_detector.py:32: ResourceWarning: unclosed database in
<sqlite3.Connection object at 0x000001DCE7C5D5D0>
See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more
info.
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-
audit\tests\test_person_detector.py:32: ResourceWarning: unclosed database in
<sqlite3.Connection object at 0x000001DCE7C5DA80>
See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more
info.
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
backend\eval\fusion_golden.py                111      23      79%
backend\eval\golden_fixtures.py             432      48      89%
784 passed, 7 skipped, 7 warnings in 82.09s (0:01:22)

```

176. assistant (2026-06-16T07:04:22.378Z)

The 7 warnings are pre-existing `ResourceWarning: unclosed database` from sqlite tests (test_person_detector, yaml scanner) ? unrelated to my changes. `golden\_fixtures.py` coverage is 89%.

Now let me update the progress-log row with the real numbers. The full suite shows 784 passed / 7 skipped, 80.71% coverage.

177. user (2026-06-16T07:04:25.206Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-audit\docs\GOLDEN_REGRESSION_FIXTURES.md has been updated successfully. (file state is current in your context ? no need to Read it back)

178. assistant (2026-06-16T07:04:30.199Z)

Now let me complete VERIFY step 4 ? demonstrate fix #1 with the exact repro the prompt asks for (`freeze_real_fixture(label="mens singles final", ...)`` rejected, and a `source\_note` with a name not persisted), into a tmp dir (not eval_baselines/fixtures/). Let me run a clean repro.

179. user (2026-06-16T07:04:42.432Z)

```

=== fix #1a: identifying label "mens singles final" ===
REJECTED: - label must match the strict slug charset '^[a-z0-9][a-z0-9_\-]*$' (lowercase
alnum + '_'/'-', no spaces/punctuation) ? a free-t
partial fixture dir created? False

=== fix #1b: source_note with a name is NOT persisted (safe note) ===
committed provenance keys: ['fps', 'frame_width', 'gt_hash', 'iou', 'kind', 'label',
'license', 'minors_free', 'note', 'owned', 'paths', 'schema_version', 'slug',
'time_origin_reset', 'windowing']
source_note key present? False
note constant present? True
label committed verbatim: mens_singles_final

=== fix #1 (bonus): identifying source_note (a name) trips the scan ===
REJECTED: path in source_note caught by defensive scan; partial dir? False

```

180. assistant (2026-06-16T07:04:47.892Z)

All of fix #1 demonstrated cleanly:

- `label="mens singles final"` (spaces) ? RefusalError, no partial dir.
- A safe `source\_note` ("captured by coach at the academy") is **not** a key in committed provenance; the boilerplate `note` constant remains; `label` committed verbatim as a valid slug.
- A path-bearing `source\_note` still trips the defensive scan and leaves no partial dir.

Now let me confirm the synthetic `--check` one final time (it's the critical guardrail) and confirm no fixtures dir got polluted in eval_baselines. Let me also verify I didn't accidentally create real fixtures in the repo's fixtures dir.

181. user (2026-06-16T07:04:52.405Z)

```
[ok] clean_single_rally: 1 candidates, 1 segments, F1=1.000000
[ok] multi_rally_with_deadtime: 2 candidates, 2 segments, F1=1.000000
[ok] occlusion_break: 2 candidates, 2 segments, F1=1.000000
[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ?
$6)
=== fixtures dir contents (must be only the 3 synthetic + manifest + README) ===
README.md
clean_single_rally
manifest.json
multi_rally_with_deadtime
occlusion_break
```

182. assistant (2026-06-16T07:04:56.976Z)

Clean ? only the 3 synthetic fixtures, manifest, README. No real fixtures or stray temp dirs (the ``os.replace`/`shutil.rmtree`` cleanup works; and any ``.tmp.`` dirs in `eval_baselines` would be a leak ? none present).

Let me also confirm the synthetic `expected.json`` files are byte-identical to what was committed (the strongest form of the guardrail). I'll re-freeze into a temp dir and compare bytes against the committed synthetic fixtures.

183. user (2026-06-16T07:05:02.540Z)

All committed synthetic expected/provenance/csv bytes reproduced exactly: True

184. assistant (2026-06-16T07:05:06.348Z)

Byte-identical confirmed ? the synthetic fixtures (including their `expected.json``) reproduce exactly, so I've hardened the freeze-real/edge paths and docs without altering committed synthetic bytes.

Let me do a final consolidated verification run of all four VERIFY commands as the prompt requires, capturing verbatim output.

185. user (2026-06-16T07:05:13.140Z)

```
===== VERIFY 1: synthetic --check =====
[ok] clean_single_rally: 1 candidates, 1 segments, F1=1.000000
[ok] multi_rally_with_deadtime: 2 candidates, 2 segments, F1=1.000000
[ok] occlusion_break: 2 candidates, 2 segments, F1=1.000000
[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ?
$6)
===== VERIFY 3a: ruff =====
All checks passed!
===== VERIFY 3b: mypy =====
Success: no issues found in 103 source files
```

186. assistant (2026-06-16T07:05:15.900Z)

All verifications pass. Let me do a final full-suite coverage run to capture the verbatim VERIFY 2 result cleanly.

187. user (2026-06-16T07:06:42.293Z)

```
backend\eval\golden_fixtures.py          432    48    89%
TOTAL                                   7697  1485    81%
Required test coverage of 70% reached. Total coverage: 80.71%
784 passed, 7 skipped, 7 warnings in 81.47s (0:01:21)
```

188. assistant (2026-06-16T07:07:03.046Z)

All work complete and verified. Here is the report.

Summary

Hardened the Golden Regression Fixtures feature per the adversarial audit. All 17 confirmed fixes implemented. The 3 committed synthetic fixtures' `expected.json` (and CSVs/provenance) remain **byte-identical** ? I hardened the freeze-real / edge paths + docs only.

Files changed

- `backend/eval/golden\_fixtures.py` ? code fixes #1?#9.
- `tests/test\_golden\_fixtures.py` ? fix #11 (by-kind provenance tags).
- `tests/test\_golden\_real\_fixtures.py` ? fix #2 (parametrized invariance) + fixes #1/#3/#4/#5/#6/#7/#10 negative tests.
- `.github/workflows/ci.yml` ? fix #12 (added `test\_golden\_real\_fixtures.py` to `golden-crossos`).
- `docs/GOLDEN\_REGRESSION\_FIXTURES.md` ? fixes #13?#17.
- `eval\_baselines/fixtures/README.md` ? fix #9 (numpy wording).

Fix notes worth flagging

- **#1** label now validated against `^[a-z0-9][a-z0-9\-\_]*$` (free-text like "mens singles final" ? `RefusalError``); `source_note`` is accepted-but-NOT-persisted (dropped from committed provenance, still scanned defensively).
- **#3** real-fixture freeze now writes to a temp dir and `os.replace``-promotes only after the privacy scan passes; negative reset labels are refused up-front ? a refused/failed freeze leaves **no** partial dir.
- **#2** (important caveat): Bit-identical reset-vs-no-reset scores are **not** achievable for arbitrary sub-second offsets, and I confirmed this is mathematically irreducible: the trajectory shifts by integer frames while labels live on the 6dp CSV grid, and 6dp-rounding is non-additive (`round6(8/30) ? round6(7/30) ? round6(1/30)``). I implemented the prompt's prescribed `t0_sec+label` 6dp-rounding (eliminates double-rounding), which keeps the **genuine/raw** metric delta at the float-error floor ( $\sim 3.3e-7$ , well within `_FLOAT_TOL``); it is exactly bit-identical for whole-second offsets. The only residual is a one-6dp-ULP ($1e-6$) difference in the **serialized** boundary error on sub-second offsets ? the serialization grid, not a metric divergence. The test asserts raw-score invariance within `_FLOAT_TOL`` (and exact snapshot equality for whole-second offsets), and I corrected the doc/docstring claims from "bit-identical" to "metric-invariant within `_FLOAT_TOL``; bit-identical for whole-second offsets" (honesty norm).

Verification (verbatim)

1. `--check``: 3/3 fixtures match ? synthetic replay unchanged. Re-freeze confirms all synthetic `expected.json`/`provenance.json`/CSV` bytes reproduce exactly.
2. Full suite: 784 passed, 7 skipped ? total coverage **80.71%** (? 80.4% floor; +0.22% vs prior 80.49%). `golden_fixtures.py`` at 89%. The 7 warnings are pre-existing `ResourceWarning: unclosed database`` from unrelated sqlite tests.
3. `ruff check .`` ? `All checks passed!``; `mypy backend training`` ? `Success: no issues found in 103 source files``.
4. Fix #1 demonstrated into a tmp dir (not `eval_baselines/fixtures/``): `label="mens singles final"`` rejected with no partial dir; a safe `source_note`` left no `source_note`` key in committed provenance (boilerplate `note`` kept, `label`` committed verbatim); a path-bearing `source_note`` tripped the defensive scan.

No real fixtures were committed; `eval_baselines/fixtures/`` contains only the 3 synthetic fixtures + manifest + README. No git commands were run.

Two items I want to be explicit about: the **#2** "bit-identical" claim is not literally achievable for sub-second offsets (I implemented the rounding and documented the honest within-tolerance guarantee instead); and the doc's coverage row now reads 80.71% (the prior `_this PR_ 80.49%`` row was left intact above the new audit-hardening row).